PlussAI NodeJ + Lymph API Documentation

Pluss Contributors

Jul 09, 2026

CONTENTS

1 API Reference **3**
1.1 Coverage Snapshot . . . 3

2 NodeJ Overview **5**
2.1 Estimator Family Additions . . . 5

3 Lymph Overview **9**
3.1 Estimator Factory Additions . . . 9

4 NodeJ Full API Reference **13**
4.1 Function and Method Reference . . . 13

5 Lymph Full API Reference **223**
5.1 Function and Method Reference . . . 223

6 Build Outputs **391**

This set follows a sklearn-style API reference pattern:

- signature + parameter-by-parameter reference
- return semantics + behavior notes
- 2-3 usage snippets where available
- verified output blocks captured from executed demos

CHAPTER

ONE

API REFERENCE

- *NodeJ Overview*
- *Lymph Overview*
- *NodeJ Full API Reference*
- *Lymph Full API Reference*

1.1 Coverage Snapshot

- NodeJ pages: **203** (methods: 169, data-types: 34)
- Lymph pages: **161** (methods: 135, data-types: 26)
- Combined pages: **364**

CHAPTER

TWO

NODEJ OVERVIEW

NodeJ is the explicit classical ML core in **PlussAI**.

Primary public modules:

- types / status / options
- dataset / data
- split
- prep transformers
- model families
- builders
- compose::Flow
- metrics / tune
- online
- io

The full method-by-method surface is documented in *NodeJ Full API Reference*.

2.1 Estimator Family Additions

2.1.1 API Signature

- `ModelSpec(task: Classification|Regression, family: Kernel, options: KernelFamilyOptions) -> ModelSpec`
- `ModelSpec(task: Transform, family: Kernel, options: KernelFamilyOptions{.form = OneClass}) -> ModelSpec`
- `ModelSpec(task: Classification|Regression, family: Bayes, options: BayesFamilyOptions) -> ModelSpec`

2.1.2 Notes

Kernel family options now cover C-style SVM, Nu-style SVM, and one-class kernel transform workflows. One-class flows use the transform lifecycle and can be saved/restored with fitted transform state.

Bayes family options now cover Gaussian, Bernoulli, Multinomial, and Complement likelihoods for classification. Multinomial and Complement modes treat features as count-like inputs and reject negative counts during learn/infer. Multinomial Bayes regression also rejects negative labels. Complement Bayes is classifier-only.

2.1.3 Inputs

- `KernelFamilyOptions` with `form = C`, `Nu`, or `OneClass`
- `BayesFamilyOptions` with `likelihood = Gaussian`, `Bernoulli`, `Multinomial`, or `Complement`

2.1.4 Output

- trained classifiers/regressors for supervised specs
- mapped inlier/outlier labels for one-class transform specs
- explicit `InvalidArgument` status for invalid multinomial/complement count inputs
- explicit `NotImplemented` status for complement Bayes regression

2.1.5 Options

- `KernelFamilyOptions::nu` controls Nu and one-class behavior.
- `BayesFamilyOptions::smoothing` applies to all Bayes likelihoods.

2.1.6 Examples

```
namespace nje = pluss::ai::nodej;

nje::ModelSpec one_class{
  .task = nje::TaskKind::Transform,
  .family = nje::ModelFamily::Kernel,
  .options = nje::KernelFamilyOptions{
      .kernel = nje::KernelKind::Rbf,
      .form = nje::KernelForm::OneClass,
      .gamma = 0.8,
      .nu = 0.5,
  },
};
```

```
nje::ModelSpec multinomial{
  .task = nje::TaskKind::Classification,
  .family = nje::ModelFamily::Bayes,
  .options = nje::BayesFamilyOptions{
      .likelihood = nje::BayesLikelihood::Multinomial,
      .smoothing = 1e-6,
  },
};
```

```
nje::ModelSpec complement{
  .task = nje::TaskKind::Classification,
  .family = nje::ModelFamily::Bayes,
  .options = nje::BayesFamilyOptions{
      .likelihood = nje::BayesLikelihood::Complement,
      .smoothing = 1e-6,
  },
};
```

2.1.7 Expected output

- one-class kernel specs produce transform flows.
- multinomial Bayes specs accept non-negative counts and reject negative counts.
- complement Bayes specs accept non-negative counts for classification and reject regression specs.

CHAPTER

THREE

LYMPH OVERVIEW

Lymph is the convenience layer over NodeJ, with high-level wrappers for:

- workflow-oriented training/inference
- DataFrame-oriented preprocessing and evaluation
- quick one-call builder/trainer helpers
- tuning and cross-validation helpers
- online wrappers

The full method-by-method surface is documented in *Lymph Full API Reference*.

3.1 Estimator Factory Additions

3.1.1 API Signature

- `kernel_factory::nu_svm_regressor(options: KernelFamilyOptions = {}) -> WorkflowModel`
- `kernel_factory::nu_svm_classifier(options: KernelFamilyOptions = {}) -> WorkflowModel`
- `kernel_factory::one_class_svm(options: KernelFamilyOptions = {}) -> WorkflowModel`
- `neighbor_factory::radius_neighbors_regressor(options: NeighborFamilyOptions = {}) -> WorkflowModel`
- `neighbor_factory::radius_neighbors_classifier(options: NeighborFamilyOptions = {}) -> WorkflowModel`
- `bayes_factory::bernoulli_nb_regressor(options: BayesFamilyOptions = {}) -> WorkflowModel`
- `bayes_factory::bernoulli_nb_classifier(options: BayesFamilyOptions = {}) -> WorkflowModel`
- `bayes_factory::multinomial_nb_regressor(options: BayesFamilyOptions = {}) -> WorkflowModel`
- `bayes_factory::multinomial_nb_classifier(options: BayesFamilyOptions = {}) -> WorkflowModel`
- `bayes_factory::complement_nb_classifier(options: BayesFamilyOptions = {}) -> WorkflowModel`

3.1.2 Notes

The factories mirror the expanded NodeJ family options while keeping Lymph's one-call workflow style. Radius-neighbor factories set the neighbor query mode to Radius. Nu SVM factories set the kernel form to Nu. `one_class_svm` creates a transform-style workflow model. Bernoulli, Multinomial, and Complement NB factories set the Bayes likelihood explicitly.

Multinomial and Complement NB expect non-negative count-like features. Negative counts fail with an explicit status instead of being clamped. Complement NB is classifier-only.

3.1.3 Inputs

- `KernelFamilyOptions`
- `NeighborFamilyOptions`
- `BayesFamilyOptions`
- feature or labeled batches for the selected workflow operation

3.1.4 Output

- `WorkflowModel` objects ready for `learn`, `infer`, `learn_map`, or `map`

3.1.5 Options

- kernel factories preserve supplied `kernel`, `gamma`, `c`, `epsilon`, and `nu` values while setting the intended form
- radius-neighbor factories preserve supplied `radius`, `radius_fallback_k`, `distance_weighted`, and `standardize`
- Bayes factories preserve supplied `smoothing` while setting the intended likelihood

3.1.6 Examples

```
namespace lym = pluss::ai::lymph;
namespace nje = pluss::ai::nodej;

auto one_class = lym::kernel_factory::one_class_svm(
    nje::KernelFamilyOptions{
        .kernel = nje::KernelKind::Rbf,
        .gamma = 0.8,
        .nu = 0.5,
    });

auto labels = one_class.learn_map(features);
```

```
auto radius = lym::neighbor_factory::radius_neighbors_classifier(
    nje::NeighborFamilyOptions{
        .radius = 0.75,
        .radius_fallback_k = 3,
        .distance_weighted = true,
    });

auto trained = radius.learn(labeled_batch);
auto predictions = trained.infer(features);
```

```
auto multinomial = lym::bayes_factory::multinomial_nb_classifier(
    nje::BayesFamilyOptions{.smoothing = 1e-6});

auto trained = multinomial.learn(count_batch);
auto predictions = trained.infer(count_features);
```

```
auto complement = lym::bayes_factory::complement_nb_classifier(
    nje::BayesFamilyOptions{.smoothing = 1e-6});

auto trained = complement.learn(count_batch);
auto predictions = trained.infer(count_features);
```

3.1.7 Expected output

- `one_class.learn_map` returns one inlier/outlier label per input sample.
- `radius.learn(...).infer(...)` uses all neighbors in the configured radius and falls back to nearest neighbors for empty neighborhoods.
- `multinomial.learn(...).infer(...)` returns predictions for valid non-negative counts.
- `complement.learn(...).infer(...)` returns classifier predictions for valid non-negative counts.

CHAPTER

FOUR

NODEJ FULL API REFERENCE

NodeJ method-centric reference generated from public headers. Each page follows **API Signature -> Inputs -> Output -> Options -> Examples (with expected outputs)**. Options/enums are documented under the methods that consume them.

- Method pages: **169**
- Data-type pages: **34**
- Overloads covered: **184**

4.1 Function and Method Reference

4.1.1 NodeJ Full API Reference: Method Index

Total method pages: **169** Total data-type pages: **34**

Data Types

`ClassReport`

API Signature

- `ClassReport`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `ClassReport` object used by NodeJ/Lymph workflows.

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Fields

Public fields on this type:

- `Series labels = Series::undefined()`
- `Series support = Series::undefined()`

- `Series precision = Series::undefined()`
- `Series recall = Series::undefined()`
- `Series f1 = Series::undefined()`
- `Series accuracy = Series::undefined()`
- `Series micro_average = Series::undefined()`
- `Series macro_average = Series::undefined()`
- `Series weighted_average = Series::undefined()`

Examples

```
ClassReport cfg{};
// cfg.labels = ...;
```

`ConfusionTable`

API Signature

- `ConfusionTable`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `ConfusionTable` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `Series labels = Series::undefined()`
- `Series counts = Series::undefined()`
- `ConfusionNormalize normalize = ConfusionNormalize::None`

Examples

```
ConfusionTable cfg{};
// cfg.labels = ...;
```

`CrossAssessReport`

API Signature

- `CrossAssessReport`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `CrossAssessReport` object used by NodeJ/Lymph workflows.

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Fields

Public fields on this type:

- `std::vector<FoldAssessResult> folds{}`
- `std::vector<MetricKind> metrics{}`
- `Series mean_metrics = Series::undefined()`
- `Series std_metrics = Series::undefined()`

Examples

```
CrossAssessReport cfg{};
```

`DependenceCurve`

API Signature

- `DependenceCurve`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `DependenceCurve` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `std::int64_t feature_index = -1`
- `Series grid = Series::undefined()`
- `Series mean_prediction = Series::undefined()`

Examples

```
DependenceCurve cfg{};
// cfg.feature_index = ...;
```

DependenceSurface

API Signature

- `DependenceSurface`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `DependenceSurface` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `std::int64_t feature_a = -1`
- `std::int64_t feature_b = -1`
- `Series grid_a = Series::undefined()`
- `Series grid_b = Series::undefined()`
- `Series mean_prediction = Series::undefined()`

Examples

```
DependenceSurface cfg{};
// cfg.feature_a = ...;
```

FeatureBatch

API Signature

- `FeatureBatch`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `FeatureBatch` object used by NodeJ/Lymph workflows.

Notes

Container for model input features and their observation layout. Use `layout` to declare whether rows or columns represent observations.

Fields

Public fields on this type:

- `Series x = Series::undefined()`
- `ObservationLayout layout = ObservationLayout::Rows`

Examples

Example 1

```
                                            1.0,  2.0, std::numeric_limits<double>
↪::quiet_NaN(),
                                            2.0,  2.5, 1.0,
                                            3.0,  std::numeric_limits<double>::quiet_
↪NaN(), 0.0,
                                            4.0,  3.5, 1.0,
                                            5.0,  4.0, 0.0,
                                            6.0,  4.5, 1.0},
                                        {6, 3},
                                        f64());
  const FeatureBatch dense{.x = x_raw, .layout = ObservationLayout::Rows};
  const Series labels = pluss::make_series(std::vector<double>{0.0, 0.0, 0.0, 1.0, 1.0,␣
↪1.0}, {6}, f64());

  SplitOptions split{};
  split.holdout_fraction = 0.33;
  split.random.seed = 7;

  auto holdout_or = pluss::ai::nodej::holdout_split(dense.sample_count(), split);
  if (!require_ok(holdout_or, "holdout_split")) return 1;
  std::cout << "holdout train idx count=" << holdout_or.value().train_indices.numel()
            << " test idx count=" << holdout_or.value().test_indices.numel() << "\n";

  auto strat_or = pluss::ai::nodej::strat_holdout_split(labels, split);
```

Expected output

```
NodeJ split + prep demo
holdout train idx count=4 test idx count=2
strat holdout train idx count=4 test idx count=2
kfold folds=3
imputed shape=[6, 3]
standardized shape=[6, 3]
```

(continues on next page)

(continued from previous page)

```
minmax shape=[6, 3]
robust shape=[6, 3]
neighbor-imputed shape=[6, 3]
iterative-imputed shape=[6, 3]
vector-normalized shape=[6, 3]
maxabs shape=[6, 3]
...
```

Example 2

```
  for (int i = 0; i < 30; ++i) {
    xvals.push_back(-3.0 + 0.1 * i);
    xvals.push_back(-2.5 + 0.05 * i);
  }
  for (int i = 0; i < 30; ++i) {
    xvals.push_back(2.0 + 0.08 * i);
    xvals.push_back(2.5 + 0.06 * i);
  }
  const FeatureBatch features{
      .x = pluss::make_series(xvals, {60, 2}, cfg),
      .layout = ObservationLayout::Rows,
  };

  ModelSpec kmeans_spec{
      .task = TaskKind::Transform,
      .family = ModelFamily::Cluster,
      .options = ClusterFamilyOptions{
          .method = ClusterMethod::KMeans,
          .clusters = 2,
          .max_iterations = 200,
          .restarts = 5,
```

Expected output

```
kmeans labels (first 10): 0 0 0 0 0 0 0 0 0 0
pca output shape: [80, 2]
nodej_unsupervised_demo complete
```

FeaturePickResult

API Signature

- `FeaturePickResult`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `FeaturePickResult` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `FeatureBatch reduced{}`
- `Series selected_feature_indices = Series::undefined()`
- `Series scores = Series::undefined()`
- `Series ranked_feature_indices = Series::undefined()`

Examples

```
FeaturePickResult cfg{};
```

`FeatureRankReport`

API Signature

- `FeatureRankReport`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `FeatureRankReport` object used by NodeJ/Lymph workflows.

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Fields

Public fields on this type:

- `Series scores = Series::undefined()`
- `Series ranked_feature_indices = Series::undefined()`

Examples

```
FeatureRankReport cfg{};
// cfg.scores = ...;
```

FlowBundle

API Signature

- `FlowBundle`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `FlowBundle` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `FlowBundleMetadata metadata{}`
- `ModelSpec model{}`
- `std::vector<TransformSpec> transforms{}`
- `bool trained = false`
- `bool has_snapshot = false`
- `std::vector<std::vector<Series>> transform_states{}`
- `std::vector<Series> model_state{}`
- `TrainOptions train_options{}`
- `LabeledBatch train_batch{}`

Examples

```
FlowBundle cfg{};
```

FlowBundleMetadata

API Signature

- `FlowBundleMetadata`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `FlowBundleMetadata` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `std::uint32_t schema_version = k_flow_bundle_schema`
- `std::string producer = "pluss.ai.nodej"`
- `std::string producer_version = "0.1.0"`
- `std::uint64_t created_unix_nanos = 0`

Examples

```
FlowBundleMetadata cfg{};
// cfg.schema_version = ...;
```

`FlowGraphInfo`

API Signature

- `FlowGraphInfo`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `FlowGraphInfo` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `TaskKind task = TaskKind::Transform`
- `bool trained = false`
- `bool has_model = false`
- `bool has_opaque_transforms = false`
- `std::vector<FlowStageInfo> stages{}`

Examples

```
FlowGraphInfo cfg{};
// cfg.task = ...;
```

`FlowStageInfo`

API Signature

- `FlowStageInfo`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `FlowStageInfo` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `std::int64_t index = 0`
- `std::string stage_key{}`
- `TransformKind kind = TransformKind::StandardScaler`
- `std::int64_t routed_feature_count = 0`
- `RouteRemainderPolicy remainder = RouteRemainderPolicy::PassThrough`
- `bool route_active = false`

Examples

```
FlowStageInfo cfg{};
// cfg.index = ...;
```

`FoldAssessResult`

API Signature

- `FoldAssessResult`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `FoldAssessResult` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `Series metrics = Series::undefined()`
- `Series train_indices = Series::undefined()`
- `Series test_indices = Series::undefined()`

Examples

```
FoldAssessResult cfg{};
// cfg.metrics = ...;
```

`IndexSplit`

API Signature

- `IndexSplit`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `IndexSplit` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `Series train_indices = Series::undefined()`
- `Series test_indices = Series::undefined()`

Examples

Example 1

```
  holdout_options.random.seed = 42;
  holdout_options.random.deterministic = true;

  StatusOr<IndexSplit> holdout_or = holdout_split(frame.ncases(), holdout_options);
  if (!holdout_or.ok()) {
    std::cerr << "holdout_split failed: " << holdout_or.status().message << "\n";
    return 1;
  }
  const IndexSplit holdout = holdout_or.move_value();

  const std::vector<std::int64_t> holdout_train_idx = holdout.train_indices.
↪to(pluss::pCPU, ScalarType::Int64)
                                                        .reshape({holdout.train_
↪indices.numel()})
                                                        .to_vec<std::int64_t>();
  const std::vector<std::int64_t> holdout_test_idx = holdout.test_indices.to(pluss::pCPU,
↪ ScalarType::Int64)
                                                      .reshape({holdout.test_indices.
↪numel()})
                                                      .to_vec<std::int64_t>();

  const DataFrame train_frame_full = frame.take(holdout_train_idx);
  const DataFrame test_frame_full = frame.take(holdout_test_idx);

  const DataFrame train_x = train_frame_full.features({
```

Expected output

```
IndexSplit configured
```

IsolationTree

API Signature

- `IsolationTree`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `IsolationTree` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `std::vector<std::int32_t> split_feature{}`

- `std::vector<double> split_value{}`
- `std::vector<std::int32_t> left_child{}`
- `std::vector<std::int32_t> right_child{}`
- `std::vector<std::int32_t> node_size{}`
- `std::vector<std::uint8_t> missing_go_left{}`

Examples

```
IsolationTree cfg{};
```

LabeledBatch

API Signature

- `LabeledBatch`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `LabeledBatch` object used by NodeJ/Lymph workflows.

Notes

Container for supervised batches: features (`x`) plus aligned labels (`y`) and optional `sample_weights`. Use this as the primary input to training, validation, and tuning workflows.

Fields

Public fields on this type:

- `FeatureBatch features{}`
- `Series y = Series::undefined()`
- `Series sample_weights = Series::undefined()`

Examples

Example 1

```
void print_series(const std::string& label, const Series& x) {
  const auto v = x.to(pluss::pCPU, ScalarType::Float64).reshape({x.numel()}).to_vec
↪<double>();
  std::cout << label << ":";
  for (double value : v) std::cout << " " << value;
  std::cout << "\n";
}

LabeledBatch make_xor_batch() {
```

(continues on next page)

(continued from previous page)

```
  const Series x = pluss::make_series(std::vector<double>{
                                          -1.0, -1.0,
                                          -1.0,  1.0,
                                           1.0, -1.0,
                                           1.0,  1.0,
                                          -0.9, -0.9,
                                          -0.9,  0.9,
                                           0.9, -0.9,
                                           0.9,  0.9,
                                          -1.2, -1.1,
                                          -1.1,  1.2,
                                           1.1, -1.2,
```

Expected output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪1.12262
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.169779
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.420312
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0248626
sweep trial objectives: 0 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0352983 0.743786
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 2

```
                                          -1.0, -0.5,
                                           1.0,  0.5,
                                           2.0,  1.0,
                                          -1.5, -0.6,
                                           1.5,  0.7},
                                      {6, 2},
                                      cfg);
  const Series y = pluss::make_series({0.0, 0.0, 1.0, 1.0, 0.0, 1.0}, {6}, cfg);
  const LabeledBatch batch{
      .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
      .y = y,
  };

  ModelSpec spec{
      .task = TaskKind::Classification,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
```

(continues on next page)

(continued from previous page)

```
        .objective = LinearObjective::Logistic,
        .regularization = RegularizationKind::L2,
        .regularization_strength = 1e-6,
        .fit_intercept = true,
```

Expected output

```
libc++abi: terminating due to uncaught exception of type pluss::Error: DataReader::with_
↪sizes: explicit sizes are only supported for host-buffer inputs
Triggered at: <path> (DataReader &pluss::detail::DataReader::with_sizes(std::span<const␣
↪std::int64_t>))
Native stack trace:
  0  pluss::detail::DataReader::with_sizes(std::__1::span<long long const,␣
↪18446744073709551615ul>)(data_reader.h:250)
  1  pluss::codem::Series pluss::detail::DataReader::to_series<pluss::codem::Config␣
↪const&>(pluss::detail::DataReader::Option, pluss::codem::Config const&) const(data_
↪reader.h:295)
  2  pluss::codem::Series pluss::codem::value<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_value.h:13)
  3  pluss::codem::Series pluss::make_series<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_factory.h:960)
  4  main(nodej_flow_bundle_demo.cpp:13)
  5  start + 6992 [dyld]


[exit_code] -6
```

Example 3

```
                                        -1.0, -0.6,
                                        1.0, 0.6,
                                        2.0, 0.8,
                                        3.0, 1.0,
                                        std::numeric_limits<double>::quiet_NaN(), 0.
↪7},
                                    {8, 2},
                                    f64());
    const Series y = pluss::make_series(std::vector<double>{0.0, 0.0, 0.0, 0.0, 1.0, 1.0,
↪ 1.0, 1.0}, {8}, f64());
    const LabeledBatch train{
        .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
        .y = y,
    };

    ModelSpec cls_spec{
        .task = TaskKind::Classification,
        .family = ModelFamily::Linear,
        .options = LinearFamilyOptions{
            .objective = LinearObjective::Logistic,
```

(continues on next page)

(continued from previous page)

```
            .regularization = RegularizationKind::L2,
            .regularization_strength = 1e-6,
            .fit_intercept = true,
```

Expected output

```
NodeJ Flow demo
classification flow metrics [accuracy, logloss]: 1 1.44251e-05
regression flow metrics [mse, r2]: 4.46559e-17 1
NodeJ Flow demo completed.
```

`ModelSpec`

API Signature

- `ModelSpec`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `ModelSpec` object used by NodeJ/Lymph workflows.

Notes

Model construction contract. Set task, family, and family-specific options before calling `make_classifier/make_regressor`.

Fields

Public fields on this type:

- `TaskKind task = TaskKind::Regression`
- `ModelFamily family = ModelFamily::Linear`
- `FamilyOptions options = LinearFamilyOptions{}`

Examples

Example 1

```
                                    {16},
                                    f64());
  return LabeledBatch{
      .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
      .y = y,
  };
}

ModelSpec linear_logistic_spec() {
```

(continues on next page)

(continued from previous page)

```
  return ModelSpec{
      .task = TaskKind::Classification,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::Logistic,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-6,
          .fit_intercept = true,
      },
  };
}
```

Expected output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪1.12262
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.169779
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.420312
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0248626
sweep trial objectives: 0 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0352983 0.743786
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 2

```
  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };
```

(continues on next page)

(continued from previous page)

```
  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

Example 3

```
                                      {6, 2},
                                      cfg);
  const Series y = pluss::make_series({0.0, 0.0, 1.0, 1.0, 0.0, 1.0}, {6}, cfg);
  const LabeledBatch batch{
      .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
      .y = y,
  };

  ModelSpec spec{
      .task = TaskKind::Classification,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::Logistic,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-6,
          .fit_intercept = true,
      },
  };

  auto flow_or = make_flow(spec);
  if (!flow_or.ok()) {
```

Expected output

```
libc++abi: terminating due to uncaught exception of type pluss::Error: DataReader::with_
↪sizes: explicit sizes are only supported for host-buffer inputs
Triggered at: <path> (DataReader &pluss::detail::DataReader::with_sizes(std::span<const␣
↪std::int64_t>))
Native stack trace:
  0  pluss::detail::DataReader::with_sizes(std::__1::span<long long const,␣
↪18446744073709551615ul>)(data_reader.h:250)
  1  pluss::codem::Series pluss::detail::DataReader::to_series<pluss::codem::Config␣
↪const&>(pluss::detail::DataReader::Option, pluss::codem::Config const&) const(data_
↪reader.h:295)
  2  pluss::codem::Series pluss::codem::value<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_value.h:13)
  3  pluss::codem::Series pluss::make_series<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
```

(continues on next page)

(continued from previous page)

```
↪const&)(series_factory.h:960)
  4  main(nodej_flow_bundle_demo.cpp:13)
  5  start + 6992 [dyld]


[exit_code] -6
```

PermuteImpactReport

API Signature

- `PermuteImpactReport`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `PermuteImpactReport` object used by NodeJ/Lymph workflows.

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Fields

Public fields on this type:

- `Series mean_drop = Series::undefined()`
- `Series std_drop = Series::undefined()`
- `Series per_repeat_drops = Series::undefined()`
- `MetricKind metric = MetricKind::Accuracy`

Examples

```
PermuteImpactReport cfg{};
// cfg.mean_drop = ...;
```

RandomState

API Signature

- `RandomState`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `RandomState` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `std::uint64_t seed = 0`
- `bool deterministic = true`

Examples

```
RandomState cfg{};
// cfg.seed = ...;
```

SparseCrossReport

API Signature

- `SparseCrossReport`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `SparseCrossReport` object used by NodeJ/Lymph workflows.

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Fields

Public fields on this type:

- `std::vector<SparseFoldReport> folds{}`
- `std::vector<MetricKind> metrics{}`
- `Series mean_metrics = Series::undefined()`
- `Series std_metrics = Series::undefined()`
- `Series mean_train_metrics = Series::undefined()`
- `Series std_train_metrics = Series::undefined()`
- `Series fit_time_seconds = Series::undefined()`
- `Series score_time_seconds = Series::undefined()`

Examples

```
SparseCrossReport cfg{};
```

SparseCsrMatrix

API Signature

- SparseCsrMatrix

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A SparseCsrMatrix object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- std::vector<std::int64_t> row_ptr{}
- std::vector<std::int64_t> col_ind{}
- std::vector<double> values{}
- std::int64_t rows = 0
- std::int64_t cols = 0

Examples

```
SparseCsrMatrix cfg{};
```

SparseFoldReport

API Signature

- SparseFoldReport

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A SparseFoldReport object used by NodeJ/Lymph workflows.

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Fields

Public fields on this type:

- `Series metrics = Series::undefined()`
- `Series train_metrics = Series::undefined()`
- `Series train_indices = Series::undefined()`
- `Series test_indices = Series::undefined()`
- `double fit_time_seconds = 0.0`
- `double score_time_seconds = 0.0`

Examples

```
SparseFoldReport cfg{};
// cfg.metrics = ...;
```

SparseTripletMatrix

API Signature

- `SparseTripletMatrix`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `SparseTripletMatrix` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `Series row_indices = Series::undefined()`
- `Series column_indices = Series::undefined()`
- `Series values = Series::undefined()`
- `std::int64_t rows = 0`
- `std::int64_t cols = 0`

Examples

```
SparseTripletMatrix cfg{};
// cfg.row_indices = ...;
```

SparseTuneLedger

API Signature

- SparseTuneLedger

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A SparseTuneLedger object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- std::vector<MetricKind> metrics{}
- std::vector<std::string> encoded_param_feature_names{}
- Series objective = Series::undefined()
- Series objective_rank = Series::undefined()
- Series status_code = Series::undefined()
- Series is_success = Series::undefined()
- Series mean_test_metrics = Series::undefined()
- Series std_test_metrics = Series::undefined()
- Series split_test_metrics = Series::undefined()
- Series fit_time_seconds = Series::undefined()
- Series score_time_seconds = Series::undefined()
- Series encoded_params = Series::undefined()

Examples

```
SparseTuneLedger cfg{};
```

SparseTuneReport

API Signature

- SparseTuneReport

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A SparseTuneReport object used by NodeJ/Lymph workflows.

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Fields

Public fields on this type:

- std::vector<SparseTuneTrial> trials{}
- std::int64_t best_index = -1
- MetricKind objective_metric = MetricKind::Accuracy
- SparseObjectivePolicy objective_policy = SparseObjectivePolicy::Auto
- std::vector<std::int64_t> objective_rank{}
- SparseTuneLedger ledger{}

Examples

```
SparseTuneReport cfg{};
```

SparseTuneTrial

API Signature

- SparseTuneTrial

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A SparseTuneTrial object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `LinearFamilyOptions options{}`
- `Status status = Status::success()`
- `SparseCrossReport assess{}`
- `double objective = std::numeric_limits<double>::quiet_NaN()`

Examples

```
SparseTuneTrial cfg{};
```

`SplitPlan`

API Signature

- `SplitPlan`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `SplitPlan` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `std::vector<IndexSplit> folds{}`

Examples

```
SplitPlan cfg{};
```

`Stage`

API Signature

- `Stage`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `Stage` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `std::vector<std::unique_ptr<TransformerBase>> transformers{}`
- `std::vector<std::int64_t> feature_route{}`

Examples

```
Stage cfg{};
```

`Status`

API Signature

- `Status`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `Status` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `StatusCode code = StatusCode::Ok`
- `std::string message{}`

Examples

Example 1

```
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
  }

  auto metrics_or = reg->assess(batch, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metrics_or, "assess")) return 1;
  print_metrics("linear metrics [mse, r2]", metrics_or.value());

  std::cout << "NodeJ DataFrame bridge demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

TransformSpec

API Signature

- `TransformSpec`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `TransformSpec` object used by NodeJ/Lymph workflows.

Notes

Transformer construction contract. Choose a transform kind and configure its options before adding it to a flow/workflow.

Fields

Public fields on this type:

- `TransformKind kind = TransformKind::StandardScaler`
- `TransformOptions options = prep::StandardScalerOptions{}`
- `std::vector<std::int64_t> feature_route{}`
- `RouteRemainderPolicy remainder = RouteRemainderPolicy::PassThrough`
- `std::string stage_key{}`

Examples

```
TransformSpec cfg{};
// cfg.kind = ...;
```

TuneLedger

API Signature

- `TuneLedger`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `TuneLedger` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `std::vector<MetricKind> metrics{}`
- `std::vector<std::string> encoded_param_feature_names{}`
- `Series objective = Series::undefined()`
- `Series objective_rank = Series::undefined()`
- `Series stage = Series::undefined()`
- `Series status_code = Series::undefined()`
- `Series is_success = Series::undefined()`
- `Series mean_test_metrics = Series::undefined()`
- `Series std_test_metrics = Series::undefined()`
- `Series split_test_metrics = Series::undefined()`
- `Series fit_time_seconds = Series::undefined()`

- Series score_time_seconds = Series::undefined()
- Series encoded_params = Series::undefined()

Examples

```
TuneLedger cfg{};
```

TuneReport

API Signature

- TuneReport

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A TuneReport object used by NodeJ/Lymph workflows.

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Fields

Public fields on this type:

- std::vector<TuneTrialResult> trials{}
- std::int64_t best_index = -1
- MetricKind objective_metric = MetricKind::Accuracy
- TuneObjectivePolicy objective_policy = TuneObjectivePolicy::Auto
- std::vector<std::int64_t> objective_rank{}
- TuneLedger ledger{}

Examples

Example 1

```
    train_options,
    assess_options,
    MetricKind::Rmse,
    TuneObjectivePolicy::Minimize);
if (!broad_sweep_or.ok()) {
  std::cerr << "sweep(broad) failed: " << broad_sweep_or.status().message << "\n";
  return 1;
}
TuneReport broad_sweep = broad_sweep_or.move_value();
```

(continues on next page)

(continued from previous page)

```
  std::vector<std::size_t> broad_success_indices{};
  for (std::size_t i = 0; i < broad_sweep.trials.size(); ++i) {
    if (!broad_sweep.trials[i].status.ok()) continue;
    if (!std::isfinite(broad_sweep.trials[i].objective)) continue;
    broad_success_indices.push_back(i);
  }
  if (broad_success_indices.empty()) {
    std::cerr << "broad sweep produced no successful trials.\n";
    return 1;
  }
```

Expected output

```
TuneReport configured
```

`TuneTrialResult`

API Signature

- `TuneTrialResult`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `TuneTrialResult` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `ModelSpec spec{}`
- `Status status = Status::success()`
- `CrossAssessReport assess{}`
- `double objective = std::numeric_limits<double>::quiet_NaN()`
- `std::int64_t stage = 0`

Examples

```
TuneTrialResult cfg{};
```

VariancePruneResult

API Signature

- `VariancePruneResult`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `VariancePruneResult` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `FeatureBatch reduced{}`
- `Series selected_feature_indices = Series::undefined()`
- `Series feature_variances = Series::undefined()`

Examples

```
VariancePruneResult cfg{};
```

Methods

apply_learn_map

API Signature

- `apply_learn_map(batch:  const FeatureBatch&, learn_options:  const TrainOptions&, infer_options:  const InferOptions&) -> StatusOr<FeatureBatch>`

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

- `batch` (`const FeatureBatch&`) Input feature data used by this call.
- `infer_options` (`const InferOptions&`) Configuration object controlling algorithm behavior and safety defaults.
- `learn_options` (`const TrainOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `StatusOr<FeatureBatch>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `FeatureBatch` (`struct`):
  - `Series x = Series::undefined()` Option field used by this method.
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.
- `InferOptions` (`struct`):
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.
  - `MissingPolicy missing = MissingPolicy::Reject` Option field used by this method.
  - `UnknownCategoryPolicy unknown_categorical = UnknownCategoryPolicy::Error` Controls fold/trial/count sizing.
  - `double positive_threshold = 0.5` Option field used by this method.
- `TrainOptions` (`struct`):
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.
  - `MissingPolicy missing = MissingPolicy::Reject` Option field used by this method.
  - `UnknownCategoryPolicy unknown_categorical = UnknownCategoryPolicy::Error` Controls fold/trial/count sizing.
  - `std::int64_t max_iterations = 1000` Maximum optimization iterations.
  - `double tolerance = 1e-6` Convergence tolerance.
  - `bool warm_start = false` Option field used by this method.
  - `bool track_diagnostics = true` Controls fold/trial/count sizing.
  - `RandomState random{}` Random seed for reproducible behavior.
  - `Series sample_weights = Series::undefined()` Optional column used as per-row training weight.
  - `ClassWeightMode class_weight = ClassWeightMode::None` Option field used by this method.
  - `Series class_weights = Series::undefined()` Option field used by this method.
  - `ScoreCalibration calibration = ScoreCalibration::None` Option field used by this method.
  - `std::int64_t calibration_iterations = 400` Controls fold/trial/count sizing.
  - `double calibration_tolerance = 1e-6` Controls fold/trial/count sizing.
  - `Series parameter_lower_bounds = Series::undefined()` Option field used by this method.
  - `Series parameter_upper_bounds = Series::undefined()` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = apply_learn_map(/* batch */, /* learn_options */, /* infer_options */);
if (!res.ok()) {
  out.error("apply_learn_map failed", res.status().message);
} else {
  out.info("apply_learn_map ok");
}
```

Expected output

```
apply_learn_map: ok
status.ok() == true
result.value() is available
```

apply_map

API Signature

- `apply_map(batch: const FeatureBatch&, options: const InferOptions&) -> StatusOr<FeatureBatch>`

Notes

Applies preprocessing/transformation behavior configured by the current options.

Inputs

- `batch` (`const FeatureBatch&`) Input feature data used by this call.
- `options` (`const InferOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `StatusOr<FeatureBatch>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `FeatureBatch` (`struct`):
  - `Series x = Series::undefined()` Option field used by this method.
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.
- `InferOptions` (`struct`):
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.
  - `MissingPolicy missing = MissingPolicy::Reject` Option field used by this method.

- `UnknownCategoryPolicy unknown_categorical = UnknownCategoryPolicy::Error` Controls fold/trial/count sizing.
- `double positive_threshold = 0.5` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = apply_map(/* batch */, /* options */);
if (!res.ok()) {
  out.error("apply_map failed", res.status().message);
} else {
  out.info("apply_map ok");
}
```

Expected output

```
apply_map: ok
status.ok() == true
result.value() is available
```

check_ready_for_infer

API Signature

- `check_ready_for_infer(where:  const char*) -> Status`

Notes

Computes predictions from prepared inputs using the currently learned state.

Inputs

- `where` (`const char*`) Method argument; see signature type for accepted form.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = check_ready_for_infer(/* where */);
if (!st.ok()) {
  out.error("check_ready_for_infer failed", st.message);
```

(continues on next page)

(continued from previous page)

```
} else {
  out.info("check_ready_for_infer ok");
}
```

Expected output

```
check_ready_for_infer: ok
status.ok() == true
```

classification_report

API Signature

- `classification_report(truth_labels: const Series&, predicted_labels: const Series&, label_order: std::initializer_list<double>, zero_division: double = 0.0) -> StatusOr<ClassReport>`
- `classification_report(truth_labels: const Series&, predicted_labels: const Series&, label_order: RangeView::Of<double>, zero_division: double = 0.0) -> StatusOr<ClassReport>`

Notes

Namespace-level API function.

Inputs

- `label_order` (`RangeView::Of<double> | std::initializer_list<double>`) Target values aligned to input observations.
- `predicted_labels` (`const Series&`) Target values aligned to input observations.
- `truth_labels` (`const Series&`) Target values aligned to input observations.
- `zero_division` (`double`), default: `0.0` Method argument; see signature type for accepted form.

Output

- Returns `StatusOr<ClassReport>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = classification_report(/* truth_labels */, /* predicted_labels */, /* label_
↪order */, /* zero_division */);
if (!res.ok()) {
  out.error("classification_report failed", res.status().message);
} else {
```

(continues on next page)

(continued from previous page)

```
  out.info("classification_report ok");
}
```

Expected output

```
classification_report: ok
status.ok() == true
result.value() is available
```

ClassifierBase::task_kind

API Signature

- `ClassifierBase::task_kind() -> TaskKind`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `TaskKind`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = ClassifierBase::task_kind();
out.info("task_kind", value);
```

Expected output

```
task_kind returned TaskKind
```

confusion_table

API Signature

- `confusion_table(truth_labels: const Series&, predicted_labels: const Series&, label_order: std::initializer_list<double>, normalize: ConfusionNormalize = ConfusionNormalize::None) -> StatusOr<ConfusionTable>`

- `confusion_table(truth_labels: const Series&, predicted_labels: const Series&, label_order: RangeView::Of<double>, normalize: ConfusionNormalize = ConfusionNormalize::None) -> StatusOr<ConfusionTable>`

Notes

Namespace-level API function.

Inputs

- `label_order` (`RangeView::Of<double> | std::initializer_list<double>`) Target values aligned to input observations.
- `normalize` (`ConfusionNormalize`), default: `ConfusionNormalize::None` Method argument; see signature type for accepted form.
- `predicted_labels` (`const Series&`) Target values aligned to input observations.
- `truth_labels` (`const Series&`) Target values aligned to input observations.

Output

- Returns `StatusOr<ConfusionTable>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `ConfusionNormalize` (`enum`):
  - `None`: Disables this behavior.
  - `ByTruth`: Accepted enum choice for this method.
  - `ByPrediction`: Accepted enum choice for this method.
  - `Global`: Accepted enum choice for this method.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto res = confusion_table(/* truth_labels */, /* predicted_labels */, /* label_order */,
↪ /* normalize */);
if (!res.ok()) {
  out.error("confusion_table failed", res.status().message);
} else {
  out.info("confusion_table ok");
}
```

Expected output

```
confusion_table: ok
status.ok() == true
result.value() is available
```

dependence_curve

API Signature

- dependence_curve(spec: const ModelSpec&, train: const LabeledBatch&, eval: const FeatureBatch&, feature_index: std::int64_t, grid_points: std::int64_t = 32) -> StatusOr<DependenceCurve>

Notes

Namespace-level API function.

Inputs

- eval (const FeatureBatch&) Method argument; see signature type for accepted form.
- feature_index (std::int64_t) Input feature data used by this call.
- grid_points (std::int64_t), default: 32 Method argument; see signature type for accepted form.
- spec (const ModelSpec&) Method argument; see signature type for accepted form.
- train (const LabeledBatch&) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns StatusOr<DependenceCurve>. Check .ok() first; on success read .value() for the computed result.

Options

Option semantics used by this method:

- FeatureBatch (struct):
  - Series x = Series::undefined() Option field used by this method.
  - ObservationLayout layout = ObservationLayout::Rows Declares whether observations are represented by rows or columns.
- LabeledBatch (struct):
  - FeatureBatch features{} Option field used by this method.
  - Series y = Series::undefined() Option field used by this method.
  - Series sample_weights = Series::undefined() Optional column used as per-row training weight.
- ModelSpec (struct):
  - TaskKind task = TaskKind::Regression Selects task type (for example classification vs regression).
  - ModelFamily family = ModelFamily::Linear Selects the model family implementation.
  - FamilyOptions options = LinearFamilyOptions{} Family-specific options payload used by the selected model/transform family.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = dependence_curve(/* spec */, /* train */, /* eval */, /* feature_index */, /
↪* grid_points */);
if (!res.ok()) {
  out.error("dependence_curve failed", res.status().message);
} else {
  out.info("dependence_curve ok");
}
```

Expected output

```
dependence_curve: ok
status.ok() == true
result.value() is available
```

dependence_surface

API Signature

- `dependence_surface(spec: const ModelSpec&, train: const LabeledBatch&, eval: const FeatureBatch&, feature_a: std::int64_t, feature_b: std::int64_t, grid_points_a: std::int64_t = 16, grid_points_b: std::int64_t = 16) -> StatusOr<DependenceSurface>`

Notes

Namespace-level API function.

Inputs

- `eval` (`const FeatureBatch&`) Method argument; see signature type for accepted form.
- `feature_a` (`std::int64_t`) Input feature data used by this call.
- `feature_b` (`std::int64_t`) Input feature data used by this call.
- `grid_points_a` (`std::int64_t`), default: `16` Method argument; see signature type for accepted form.
- `grid_points_b` (`std::int64_t`), default: `16` Method argument; see signature type for accepted form.
- `spec` (`const ModelSpec&`) Method argument; see signature type for accepted form.
- `train` (`const LabeledBatch&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `StatusOr<DependenceSurface>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `FeatureBatch` (`struct`):
  - `Series x = Series::undefined()` Option field used by this method.

  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.

- `LabeledBatch` (`struct`):
  - `FeatureBatch features{}` Option field used by this method.
  - `Series y = Series::undefined()` Option field used by this method.
  - `Series sample_weights = Series::undefined()` Optional column used as per-row training weight.
- `ModelSpec` (`struct`):
  - `TaskKind task = TaskKind::Regression` Selects task type (for example classification vs regression).
  - `ModelFamily family = ModelFamily::Linear` Selects the model family implementation.
  - `FamilyOptions options = LinearFamilyOptions{}` Family-specific options payload used by the selected model/transform family.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = dependence_surface(/* spec */, /* train */, /* eval */, /* feature_a */, /*␣
↪feature_b */, /* grid_points_a */, /* grid_points_b */);
if (!res.ok()) {
  out.error("dependence_surface failed", res.status().message);
} else {
  out.info("dependence_surface ok");
}
```

Expected output

```
dependence_surface: ok
status.ok() == true
result.value() is available
```

EstimatorBase::dump_state

API Signature

- `EstimatorBase::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = EstimatorBase::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

EstimatorBase::load_state

API Signature

- `EstimatorBase::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto st = EstimatorBase::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

EstimatorBase::task_kind

API Signature

- `EstimatorBase::task_kind() -> TaskKind`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `TaskKind`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = EstimatorBase::task_kind();
out.info("task_kind", value);
```

Expected output

```
task_kind returned TaskKind
```

`EstimatorBase::trained`

API Signature

- `EstimatorBase::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = EstimatorBase::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

FeatureBatch::defined

API Signature

- `FeatureBatch::defined() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

Option semantics used by this method:

- `FeatureBatch` (`struct`):
  - `Series x = Series::undefined()` Option field used by this method.
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = FeatureBatch::defined();
out.info("defined", value);
```

Expected output

```
defined: true
```

Example 2 (Demo Pattern)

```
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

Example 3 (Demo Pattern)

```
    TrainOptions warm_opts{};
    warm_opts.max_iterations = 80;

    auto model = pluss::ai::lymph::online_factory::linear_classifier(online_options);
    model.learn_(warm, warm_opts).advance_(stream);

    const auto metrics = model.assess(full);
    const Series metric_values = metrics.accuracy.defined() ? metrics.accuracy : metrics.
↪values;

    std::cout << "lymph online seen_samples=" << model.seen_samples() << '\n';
    std::cout << "lymph online metrics [accuracy]: ";
    print_vec(metric_values);
    return 0;
  } catch (const std::exception& ex) {
    std::cerr << "lymph_online_demo failed: " << ex.what() << '\n';
    return 1;
  }
}
```

Expected output

```
lymph online seen_samples=1660
lymph online metrics [accuracy]: 0.9875
```

FeatureBatch::feature_count

API Signature

- `FeatureBatch::feature_count() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

Option semantics used by this method:

- `FeatureBatch` (`struct`):
  - `Series x = Series::undefined()` Option field used by this method.
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = FeatureBatch::feature_count();
out.info("feature_count", value);
```

Expected output

```
feature_count: <numeric_result>
```

Example 2 (Demo Pattern)

```
  options.note_policy = DataNotePolicy::Hash;
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
```

(continues on next page)

(continued from previous page)

```
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

Example 3 (Demo Pattern)

```
}

Config i64() {
  return Config{}.with_dtype(ScalarType::Int64).with_platform(pluss::pCPU);
}

LabeledBatch slice_rows(const LabeledBatch& batch, std::int64_t start, std::int64_t␣
↪count) {
  const Series idx = pluss::arange(start, start + count, i64());
  const Series x = batch.features.x.index_select(0, idx).reshape({count, batch.features.
↪feature_count()});
  const Series y = batch.y.index_select(0, idx).reshape({count});
  return LabeledBatch{
      .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
      .y = y,
  };
}

LabeledBatch make_class_batch() {
  std::vector<double> xv{};
  std::vector<double> yv{};
  xv.reserve(2 * 90);
  yv.reserve(90);
```

Expected output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

`FeatureBatch::sample_count`

API Signature

- `FeatureBatch::sample_count() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

Option semantics used by this method:

- `FeatureBatch` (`struct`):
  - `Series x = Series::undefined()` Option field used by this method.
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = FeatureBatch::sample_count();
out.info("sample_count", value);
```

Expected output

```
sample_count: <numeric_result>
```

Example 2 (Demo Pattern)

```
  options.sample_weight_feature = "weight";
  options.note_policy = DataNotePolicy::Hash;
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";
```

(continues on next page)

(continued from previous page)

```
  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

Example 3 (Demo Pattern)

```
                                          f64());
  const FeatureBatch dense{.x = x_raw, .layout = ObservationLayout::Rows};
  const Series labels = pluss::make_series(std::vector<double>{0.0, 0.0, 0.0, 1.0, 1.0,␣
↪1.0}, {6}, f64());

  SplitOptions split{};
  split.holdout_fraction = 0.33;
  split.random.seed = 7;

  auto holdout_or = pluss::ai::nodej::holdout_split(dense.sample_count(), split);
  if (!require_ok(holdout_or, "holdout_split")) return 1;
  std::cout << "holdout train idx count=" << holdout_or.value().train_indices.numel()
            << " test idx count=" << holdout_or.value().test_indices.numel() << "\n";

  auto strat_or = pluss::ai::nodej::strat_holdout_split(labels, split);
  if (!require_ok(strat_or, "strat_holdout_split")) return 1;
  std::cout << "strat holdout train idx count=" << strat_or.value().train_indices.numel()
            << " test idx count=" << strat_or.value().test_indices.numel() << "\n";

  split.folds = 3;
  auto fold_or = pluss::ai::nodej::fold_plan(dense.sample_count(), split);
  if (!require_ok(fold_or, "fold_plan")) return 1;
```

Expected output

```
NodeJ split + prep demo
holdout train idx count=4 test idx count=2
strat holdout train idx count=4 test idx count=2
kfold folds=3
imputed shape=[6, 3]
standardized shape=[6, 3]
minmax shape=[6, 3]
```

(continues on next page)

(continued from previous page)

```
robust shape=[6, 3]
neighbor-imputed shape=[6, 3]
iterative-imputed shape=[6, 3]
vector-normalized shape=[6, 3]
maxabs shape=[6, 3]
...
```

FeatureBatch::validate_features

API Signature

- `FeatureBatch::validate_features(batch:  const FeatureBatch&) -> Status`

Notes

Validates shape/layout/typing invariants for this data container before downstream use.

Inputs

- `batch` (`const FeatureBatch&`) Input feature data used by this call.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

Option semantics used by this method:

- `FeatureBatch` (`struct`):
  - `Series x = Series::undefined()` Option field used by this method.
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto st = FeatureBatch::validate_features(/* batch */);
if (!st.ok()) {
  out.error("validate_features failed", st.message);
} else {
  out.info("validate_features ok");
}
```

Expected output

```
validate_features: ok
status.ok() == true
```

`FeatureBatch::validate_labeled`

API Signature

- `FeatureBatch::validate_labeled(batch:  const LabeledBatch&) -> Status`

Notes

Validates shape/layout/typing invariants for this data container before downstream use.

Inputs

- `batch` (`const LabeledBatch&`) Input feature data used by this call.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

Option semantics used by this method:

- `FeatureBatch` (`struct`):
  - `Series x = Series::undefined()` Option field used by this method.
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.
- `LabeledBatch` (`struct`):
  - `FeatureBatch features{}` Option field used by this method.
  - `Series y = Series::undefined()` Option field used by this method.
  - `Series sample_weights = Series::undefined()` Optional column used as per-row training weight.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = FeatureBatch::validate_labeled(/* batch */);
if (!st.ok()) {
  out.error("validate_labeled failed", st.message);
} else {
  out.info("validate_labeled ok");
}
```

Expected output

```
validate_labeled: ok
status.ok() == true
```

Example 2 (Demo Pattern)

```
  train_batch.y = train_y.to(train_standard_scaled.platform(), ScalarType::Float64).
↪reshape({train_standard_scaled.sizes()[0]});
  train_batch.sample_weights = Series::undefined();

  LabeledBatch test_batch{};
  test_batch.features = FeatureBatch{.x = test_standard_scaled, .layout =␣
↪ObservationLayout::Rows};
  test_batch.y = test_y.to(test_standard_scaled.platform(), ScalarType::Float64).reshape(
↪{test_standard_scaled.sizes()[0]});
  test_batch.sample_weights = Series::undefined();

  Status train_batch_check = validate_labeled(train_batch);
  if (!train_batch_check.ok()) {
    std::cerr << "validate_labeled(train_batch) failed: " << train_batch_check.message <
↪< "\n";
    return 1;
  }
  Status test_batch_check = validate_labeled(test_batch);
  if (!test_batch_check.ok()) {
    std::cerr << "validate_labeled(test_batch) failed: " << test_batch_check.message <<
↪"\n";
    return 1;
  }

  TrainOptions train_options{};
  train_options.max_iterations = 3000;
```

Expected output

```
validate_labeled: ok
status.ok() == true
```

`FeatureTransformer::add_pipeline`

API Signature

- `FeatureTransformer::add_pipeline(transformers: std::vector<std::unique_ptr<TransformerBase>>, feature_route: RangeView::Int) -> Status`

Notes

Instance method on `self`.

Inputs

- `feature_route` (`RangeView::Int`) Input feature data used by this call.
- `transformers` (`std::vector<std::unique_ptr<TransformerBase>>`) List-like argument; pass values in the order expected by the signature.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = FeatureTransformer::add_pipeline(/* transformers */, /* feature_route */);
if (!st.ok()) {
  out.error("add_pipeline failed", st.message);
} else {
  out.info("add_pipeline ok");
}
```

Expected output

```
add_pipeline: ok
status.ok() == true
```

`FeatureTransformer::add_stage`

API Signature

- `FeatureTransformer::add_stage(transformer: std::unique_ptr<TransformerBase>, feature_route: RangeView::Int) -> Status`

Notes

Instance method on `self`.

Inputs

- `feature_route` (`RangeView::Int`) Input feature data used by this call.
- `transformer` (`std::unique_ptr<TransformerBase>`) Method argument; see signature type for accepted form.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = FeatureTransformer::add_stage(/* transformer */, /* feature_route */);
if (!st.ok()) {
  out.error("add_stage failed", st.message);
} else {
  out.info("add_stage ok");
}
```

Expected output

```
add_stage: ok
status.ok() == true
```

FeatureTransformer::clear_stages

API Signature

- `FeatureTransformer::clear_stages() -> void`

Notes

Instance method on `self`.

Inputs

None.

Output

- No direct return value; operation applies side effects to object state.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
FeatureTransformer::clear_stages();
out.info("clear_stages completed");
```

Expected output

```
clear_stages completed
```

`FeatureTransformer::dump_state`

API Signature

- `FeatureTransformer::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = FeatureTransformer::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
```

(continues on next page)

(continued from previous page)

```
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

`FeatureTransformer::FeatureTransformer`

API Signature

- `FeatureTransformer(remainder: FeatureRemainderPolicy = FeatureRemainderPolicy::Drop) -> auto`

Notes

Instance method on `self`.

Inputs

- `remainder` (`FeatureRemainderPolicy`), default: `FeatureRemainderPolicy::Drop` Method argument; see signature type for accepted form.

Output

- Returns method result for the selected overload.

Options

Option semantics used by this method:

- `FeatureRemainderPolicy` (`enum`):
  - `Drop`: Accepted enum choice for this method.
  - `PassThrough`: Accepted enum choice for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = FeatureTransformer(/* remainder */);
out.info("FeatureTransformer", value);
```

Expected output

```
FeatureTransformer returned result
```

`FeatureTransformer::load_state`

API Signature

- `FeatureTransformer::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = FeatureTransformer::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

FeatureTransformer::remainder_policy

API Signature

- `FeatureTransformer::remainder_policy() -> FeatureRemainderPolicy`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `FeatureRemainderPolicy`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = FeatureTransformer::remainder_policy();
out.info("remainder_policy", value);
```

Expected output

```
remainder_policy returned FeatureRemainderPolicy
```

FeatureTransformer::set_remainder_policy

API Signature

- `FeatureTransformer::set_remainder_policy(policy: FeatureRemainderPolicy) -> void`

Notes

Sets or updates internal configuration/state used by later train/infer/evaluate calls.

Inputs

- `policy` (`FeatureRemainderPolicy`) Target values aligned to input observations.

Output

- No direct return value; operation applies side effects to object state.

Options

Option semantics used by this method:

- `FeatureRemainderPolicy` (`enum`):
  - `Drop`: Accepted enum choice for this method.
  - `PassThrough`: Accepted enum choice for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
FeatureTransformer::set_remainder_policy(/* policy */);
out.info("set_remainder_policy completed");
```

Expected output

```
set_remainder_policy completed
```

`FeatureTransformer::stage_count`

API Signature

- `FeatureTransformer::stage_count() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = FeatureTransformer::stage_count();
out.info("stage_count", value);
```

Expected output

```
stage_count: <numeric_result>
```

FeatureTransformer::trained

API Signature

- `FeatureTransformer::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = FeatureTransformer::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

Flow::add_column_stage

API Signature

- `Flow::add_column_stage(stage_key: std::string, kind: TransformKind, options: const TransformOptions&, feature_route: std::initializer_list<std::int64_t>, remainder: RouteRemainderPolicy = RouteRemainderPolicy::PassThrough) -> Status`

Notes

Instance method on `self`.

Inputs

- `feature_route` (`std::initializer_list<std::int64_t>`) Input feature data used by this call.
- `kind` (`TransformKind`) Algorithm-size/count argument used to shape fit or split behavior.

- `options` (`const TransformOptions&`) Configuration object controlling algorithm behavior and safety defaults.
- `remainder` (`RouteRemainderPolicy`), default: `RouteRemainderPolicy::PassThrough` Method argument; see signature type for accepted form.
- `stage_key` (`std::string`) Target values aligned to input observations.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

Option semantics used by this method:

- `RouteRemainderPolicy` (`enum`):
  - `PassThrough`: Accepted enum choice for this method.
  - `Drop`: Accepted enum choice for this method.
- `TransformKind` (`enum`):
  - `StandardScaler`: Accepted enum choice for this method.
  - `MinMaxScaler`: Accepted enum choice for this method.
  - `RobustScaler`: Accepted enum choice for this method.
  - `SimpleImputer`: Accepted enum choice for this method.
  - `NeighborImputer`: Accepted enum choice for this method.
  - `IterativeImputer`: Accepted enum choice for this method.
  - `OneHotEncoder`: Accepted enum choice for this method.
  - `FrequencyEncoder`: Accepted enum choice for this method.
  - `VectorNormalizer`: Accepted enum choice for this method.
  - `MaxAbsScaler`: Accepted enum choice for this method.
  - `QuantileMap`: Accepted enum choice for this method.
  - `PowerMap`: Accepted enum choice for this method.
  - `ThresholdBinarizer`: Accepted enum choice for this method.
  - `PolynomialLift`: Accepted enum choice for this method.
  - `OrdinalEncoder`: Accepted enum choice for this method.
  - `LabelEncoder`: Accepted enum choice for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = Flow::add_column_stage(/* stage_key */, /* kind */, /* options */, /* feature_
↪route */, /* remainder */);
if (!st.ok()) {
```

(continues on next page)

(continued from previous page)

```
  out.error("add_column_stage failed", st.message);
} else {
  out.info("add_column_stage ok");
}
```

Expected output

```
add_column_stage: ok
status.ok() == true
```

`Flow::add_feature_transformer`

API Signature

- `Flow::add_feature_transformer(transformer:  prep::FeatureTransformer) -> Status`

Notes

Instance method on `self`.

Inputs

- `transformer` (`prep::FeatureTransformer`) Method argument; see signature type for accepted form.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = Flow::add_feature_transformer(/* transformer */);
if (!st.ok()) {
  out.error("add_feature_transformer failed", st.message);
} else {
  out.info("add_feature_transformer ok");
}
```

Expected output

```
add_feature_transformer: ok
status.ok() == true
```

Flow::add_function_transformer

API Signature

- `Flow::add_function_transformer(transformer: prep::FunctionTransformer) -> Status`

Notes

Instance method on `self`.

Inputs

- `transformer` (`prep::FunctionTransformer`) Method argument; see signature type for accepted form.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = Flow::add_function_transformer(/* transformer */);
if (!st.ok()) {
  out.error("add_function_transformer failed", st.message);
} else {
  out.info("add_function_transformer ok");
}
```

Expected output

```
add_function_transformer: ok
status.ok() == true
```

Flow::append_transform

API Signature

- `Flow::append_transform(transformer: std::unique_ptr<TransformerBase>) -> Status`

Notes

Instance method on `self`.

Inputs

- `transformer` (`std::unique_ptr<TransformerBase>`) Method argument; see signature type for accepted form.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = Flow::append_transform(/* transformer */);
if (!st.ok()) {
  out.error("append_transform failed", st.message);
} else {
  out.info("append_transform ok");
}
```

Expected output

```
append_transform: ok
status.ok() == true
```

`Flow::clear_model`

API Signature

- `Flow::clear_model() -> void`

Notes

Instance method on `self`.

Inputs

None.

Output

- No direct return value; operation applies side effects to object state.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
Flow::clear_model();
out.info("clear_model completed");
```

Expected output

```
clear_model completed
```

Flow::clear_transforms

API Signature

- `Flow::clear_transforms() -> void`

Notes

Instance method on `self`.

Inputs

None.

Output

- No direct return value; operation applies side effects to object state.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
Flow::clear_transforms();
out.info("clear_transforms completed");
```

Expected output

```
clear_transforms completed
```

Flow::dump_model_state

API Signature

- `Flow::dump_model_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = Flow::dump_model_state();
if (!res.ok()) {
  out.error("dump_model_state failed", res.status().message);
} else {
  out.info("dump_model_state ok");
}
```

Expected output

```
dump_model_state: ok
status.ok() == true
result.value() is available
```

`Flow::dump_transform_states`

API Signature

- `Flow::dump_transform_states() -> StatusOr<std::vector<std::vector<Series>>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<std::vector<Series>>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = Flow::dump_transform_states();
if (!res.ok()) {
  out.error("dump_transform_states failed", res.status().message);
} else {
  out.info("dump_transform_states ok");
}
```

Expected output

```
dump_transform_states: ok
status.ok() == true
result.value() is available
```

`Flow::flow_summary`

API Signature

- `Flow::flow_summary() -> std::string`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::string`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Flow::flow_summary();
out.info("flow_summary", value);
```

Expected output

```
flow_summary returned std::string
```

Flow::graph_info

API Signature

- `Flow::graph_info() -> StatusOr<FlowGraphInfo>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<FlowGraphInfo>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = Flow::graph_info();
if (!res.ok()) {
  out.error("graph_info failed", res.status().message);
} else {
  out.info("graph_info ok");
}
```

Expected output

```
graph_info: ok
status.ok() == true
result.value() is available
```

Flow::has_model

API Signature

- `Flow::has_model() -> bool`

Notes

Boolean introspection helper that reports whether a capability/state is available.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Flow::has_model();
out.info("has_model", value);
```

Expected output

```
has_model: true
```

`Flow::has_model_spec`

API Signature

- `Flow::has_model_spec() -> bool`

Notes

Boolean introspection helper that reports whether a capability/state is available.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Flow::has_model_spec();
out.info("has_model_spec", value);
```

Expected output

```
has_model_spec: true
```

Flow::has_opaque_transforms

API Signature

- `Flow::has_opaque_transforms() -> bool`

Notes

Boolean introspection helper that reports whether a capability/state is available.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Flow::has_opaque_transforms();
out.info("has_opaque_transforms", value);
```

Expected output

```
has_opaque_transforms: true
```

Flow::load_blueprint

API Signature

- `Flow::load_blueprint(text:  std::string_view) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `text` (`std::string_view`) Input feature data used by this call.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = Flow::load_blueprint(/* text */);
if (!st.ok()) {
  out.error("load_blueprint failed", st.message);
} else {
  out.info("load_blueprint ok");
}
```

Expected output

```
load_blueprint: ok
status.ok() == true
```

`Flow::load_model_state`

API Signature

- `Flow::load_model_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = Flow::load_model_state(/* state */);
```

(continues on next page)

(continued from previous page)

```
if (!st.ok()) {
  out.error("load_model_state failed", st.message);
} else {
  out.info("load_model_state ok");
}
```

Expected output

```
load_model_state: ok
status.ok() == true
```

Flow::load_transform_states

API Signature

- `Flow::load_transform_states(states:  RangeView::Of<std::vector<Series>>) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `states` (`RangeView::Of<std::vector<Series>>`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = Flow::load_transform_states(/* states */);
if (!st.ok()) {
  out.error("load_transform_states failed", st.message);
} else {
  out.info("load_transform_states ok");
}
```

Expected output

```
load_transform_states: ok
status.ok() == true
```

`Flow::mark_trained`

API Signature

- `Flow::mark_trained(trained:  bool) -> void`

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

- `trained` (`bool`) Method argument; see signature type for accepted form.

Output

- No direct return value; operation applies side effects to object state.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
Flow::mark_trained(/* trained */);
out.info("mark_trained completed");
```

Expected output

```
mark_trained completed
```

`Flow::model_spec`

API Signature

- `Flow::model_spec() -> const std::optional<ModelSpec>&`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `const std::optional<ModelSpec>&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Flow::model_spec();
out.info("model_spec", value);
```

Expected output

```
model_spec returned const std::optional<ModelSpec>&
```

`Flow::nested_param`

API Signature

- `Flow::nested_param(path:  std::string_view) -> StatusOr<double>`

Notes

Instance method on `self`.

Inputs

- `path` (`std::string_view`) Path to persisted model/data artifact.

Output

- Returns `StatusOr<double>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = Flow::nested_param(/* path */);
if (!res.ok()) {
  out.error("nested_param failed", res.status().message);
} else {
  out.info("nested_param ok");
}
```

Expected output

```
nested_param: ok
status.ok() == true
result.value() is available
```

Flow::nested_param_keys

API Signature

- `Flow::nested_param_keys() -> std::vector<std::string>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::vector<std::string>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Flow::nested_param_keys();
out.info("nested_param_keys", value);
```

Expected output

```
nested_param_keys returned std::vector<std::string>
```

Flow::save_blueprint

API Signature

- `Flow::save_blueprint() -> StatusOr<std::string>`

Notes

Persists the current fitted artifact so it can be loaded for inference later.

Inputs

None.

Output

- Returns `StatusOr<std::string>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = Flow::save_blueprint();
if (!res.ok()) {
  out.error("save_blueprint failed", res.status().message);
} else {
  out.info("save_blueprint ok");
}
```

Expected output

```
save_blueprint: ok
status.ok() == true
result.value() is available
```

`Flow::set_classifier`

API Signature

- `Flow::set_classifier(model:  std::unique_ptr<ClassifierBase>) -> Status`

Notes

Sets or updates internal configuration/state used by later train/infer/evaluate calls.

Inputs

- `model` (`std::unique_ptr<ClassifierBase>`) Method argument; see signature type for accepted form.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = Flow::set_classifier(/* model */);
if (!st.ok()) {
  out.error("set_classifier failed", st.message);
} else {
  out.info("set_classifier ok");
}
```

Expected output

```
set_classifier: ok
status.ok() == true
```

Flow::set_model

API Signature

- `Flow::set_model(spec:  const ModelSpec&) -> Status`

Notes

Sets or updates internal configuration/state used by later train/infer/evaluate calls.

Inputs

- `spec` (`const ModelSpec&`) Method argument; see signature type for accepted form.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

Option semantics used by this method:

- `ModelSpec` (`struct`):
  - `TaskKind task = TaskKind::Regression` Selects task type (for example classification vs regression).
  - `ModelFamily family = ModelFamily::Linear` Selects the model family implementation.
  - `FamilyOptions options = LinearFamilyOptions{}` Family-specific options payload used by the selected model/transform family.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = Flow::set_model(/* spec */);
if (!st.ok()) {
  out.error("set_model failed", st.message);
} else {
  out.info("set_model ok");
}
```

Expected output

```
set_model: ok
status.ok() == true
```

Flow::set_nested_param

API Signature

- `Flow::set_nested_param(path: std::string_view, value: double) -> Status`

Notes

Sets or updates internal configuration/state used by later train/infer/evaluate calls.

Inputs

- `path` (`std::string_view`) Path to persisted model/data artifact.
- `value` (`double`) Method argument; see signature type for accepted form.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto st = Flow::set_nested_param(/* path */, /* value */);
if (!st.ok()) {
  out.error("set_nested_param failed", st.message);
} else {
  out.info("set_nested_param ok");
}
```

Expected output

```
set_nested_param: ok
status.ok() == true
```

Flow::set_regressor

API Signature

- `Flow::set_regressor(model: std::unique_ptr<RegressorBase>) -> Status`

Notes

Sets or updates internal configuration/state used by later train/infer/evaluate calls.

Inputs

- `model` (`std::unique_ptr<RegressorBase>`) Method argument; see signature type for accepted form.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = Flow::set_regressor(/* model */);
if (!st.ok()) {
  out.error("set_regressor failed", st.message);
} else {
  out.info("set_regressor ok");
}
```

Expected output

```
set_regressor: ok
status.ok() == true
```

`Flow::set_stage_key`

API Signature

- `Flow::set_stage_key(stage_index: std::int64_t, stage_key: std::string) -> Status`

Notes

Sets or updates internal configuration/state used by later train/infer/evaluate calls.

Inputs

- `stage_index` (`std::int64_t`) Input feature data used by this call.
- `stage_key` (`std::string`) Target values aligned to input observations.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = Flow::set_stage_key(/* stage_index */, /* stage_key */);
if (!st.ok()) {
  out.error("set_stage_key failed", st.message);
} else {
  out.info("set_stage_key ok");
}
```

Expected output

```
set_stage_key: ok
status.ok() == true
```

Flow::set_stage_remainder

API Signature

- `Flow::set_stage_remainder(stage_index:  std::int64_t, remainder: RouteRemainderPolicy) -> Status`

Notes

Sets or updates internal configuration/state used by later train/infer/evaluate calls.

Inputs

- `remainder` (`RouteRemainderPolicy`) Method argument; see signature type for accepted form.
- `stage_index` (`std::int64_t`) Input feature data used by this call.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

Option semantics used by this method:

- `RouteRemainderPolicy` (`enum`):
  - `PassThrough`: Accepted enum choice for this method.
  - `Drop`: Accepted enum choice for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = Flow::set_stage_remainder(/* stage_index */, /* remainder */);
if (!st.ok()) {
  out.error("set_stage_remainder failed", st.message);
} else {
  out.info("set_stage_remainder ok");
}
```

Expected output

```
set_stage_remainder: ok
status.ok() == true
```

`Flow::set_stage_route`

API Signature

- `Flow::set_stage_route(stage_index: std::int64_t, feature_route: RangeView::Int) -> Status`
- `Flow::set_stage_route(stage_index: std::int64_t, feature_route: std::initializer_list<std::int64_t>) -> Status`

Notes

Sets or updates internal configuration/state used by later train/infer/evaluate calls.

Inputs

- `feature_route` (`RangeView::Int | std::initializer_list<std::int64_t>`) Input feature data used by this call.
- `stage_index` (`std::int64_t`) Input feature data used by this call.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = Flow::set_stage_route(/* stage_index */, /* feature_route */);
if (!st.ok()) {
  out.error("set_stage_route failed", st.message);
```

(continues on next page)

(continued from previous page)

```
} else {
  out.info("set_stage_route ok");
}
```

Expected output

```
set_stage_route: ok
status.ok() == true
```

`Flow::stage_count`

API Signature

- `Flow::stage_count() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Flow::stage_count();
out.info("stage_count", value);
```

Expected output

```
stage_count: <numeric_result>
```

`Flow::task_kind`

API Signature

- `Flow::task_kind() -> TaskKind`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `TaskKind`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Flow::task_kind();
out.info("task_kind", value);
```

Expected output

```
task_kind returned TaskKind
```

`Flow::trained`

API Signature

- `Flow::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Flow::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

Flow::transform_specs

API Signature

- `Flow::transform_specs() -> const std::vector<TransformSpec>&`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `const std::vector<TransformSpec>&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Flow::transform_specs();
out.info("transform_specs", value);
```

Expected output

```
transform_specs returned const std::vector<TransformSpec>&
```

FrequencyEncoder::dump_state

API Signature

- `FrequencyEncoder::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = FrequencyEncoder::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();
```

(continues on next page)

(continued from previous page)

```
  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

`FrequencyEncoder::load_state`

API Signature

- `FrequencyEncoder::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = FrequencyEncoder::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

FrequencyEncoder::trained

API Signature

- `FrequencyEncoder::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = FrequencyEncoder::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

FunctionTransformer::dump_state

API Signature

- `FunctionTransformer::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = FunctionTransformer::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
```

(continues on next page)

(continued from previous page)

```
result.value() is available
```

FunctionTransformer::load_state

API Signature

- `FunctionTransformer::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto st = FunctionTransformer::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

FunctionTransformer::trained

API Signature

- `FunctionTransformer::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = FunctionTransformer::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

`IterativeImputer::dump_state`

API Signature

- `IterativeImputer::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = IterativeImputer::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

IterativeImputer::load_state

API Signature

- `IterativeImputer::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = IterativeImputer::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

`IterativeImputer::trained`

API Signature

- `IterativeImputer::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = IterativeImputer::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

labeled_batch_from_dataframe

API Signature

- `labeled_batch_from_dataframe(frame: const pluss::table::DataFrame&, options: const DataFrameBatchOptions&) -> StatusOr<LabeledBatch>`

Notes

Builds a supervised `LabeledBatch` from a DataFrame. Use `options` to pick feature columns, label column, weight column, and categorical handling policy.

Inputs

- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `options` (`const DataFrameBatchOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `StatusOr<LabeledBatch>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `DataFrameBatchOptions` (`struct`):
  - `std::vector<std::string> feature_names{}` Explicit list of feature columns to read from the input frame.
  - `std::optional<std::string> label_feature{std::nullopt}` Column used as the supervised target (`y`).
  - `std::optional<std::string> sample_weight_feature{std::nullopt}` Optional column used as per-row training weight.
  - `ObservationLayout layout{ObservationLayout::Rows}` Declares whether observations are represented by rows or columns.

- `DataThreadPolicy thread_policy{DataThreadPolicy::CastTemporalToSeries}` Controls how numeric/temporal frame columns are adapted into `Series`.
- `DataNotePolicy note_policy{DataNotePolicy::Reject}` Controls how note/categorical columns are handled during adaptation.
- `std::uint64_t note_hash_seed{14695981039346656037ULL}` Random seed for reproducible behavior.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = labeled_batch_from_dataframe(/* frame */, /* options */);
if (!res.ok()) {
  out.error("labeled_batch_from_dataframe failed", res.status().message);
} else {
  out.info("labeled_batch_from_dataframe ok");
}
```

Expected output

```
labeled_batch_from_dataframe: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  DataFrameBatchOptions options{};
  options.feature_names = {"x0", "x1", "category", "ts", "target", "weight"};
  options.label_feature = "target";
  options.sample_weight_feature = "weight";
  options.note_policy = DataNotePolicy::Hash;
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
```

(continues on next page)

(continued from previous page)

```
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

LabeledBatch::has_labels

API Signature

- `LabeledBatch::has_labels() -> bool`

Notes

Boolean introspection helper that reports whether a capability/state is available.

Inputs

None.

Output

- Returns `bool`.

Options

Option semantics used by this method:

- `LabeledBatch` (`struct`):
  - `FeatureBatch features{}` Option field used by this method.
  - `Series y = Series::undefined()` Option field used by this method.
  - `Series sample_weights = Series::undefined()` Optional column used as per-row training weight.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto value = LabeledBatch::has_labels();
out.info("has_labels", value);
```

Expected output

```
has_labels: true
```

LabeledBatch::has_weights

API Signature

- `LabeledBatch::has_weights() -> bool`

Notes

Boolean introspection helper that reports whether a capability/state is available.

Inputs

None.

Output

- Returns `bool`.

Options

Option semantics used by this method:

- `LabeledBatch` (`struct`):
  - `FeatureBatch features{}` Option field used by this method.
  - `Series y = Series::undefined()` Option field used by this method.
  - `Series sample_weights = Series::undefined()` Optional column used as per-row training weight.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = LabeledBatch::has_weights();
out.info("has_weights", value);
```

Expected output

```
has_weights: true
```

`LabelEncoder::dump_state`

API Signature

- `LabelEncoder::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = LabelEncoder::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
```

(continues on next page)

(continued from previous page)

```
result.value() is available
```

LabelEncoder::load_state

API Signature

- `LabelEncoder::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = LabelEncoder::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

LabelEncoder::trained

API Signature

- `LabelEncoder::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = LabelEncoder::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

`LexiconMapper::feature_count`

API Signature

- `LexiconMapper::feature_count() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = LexiconMapper::feature_count();
out.info("feature_count", value);
```

Expected output

```
feature_count: <numeric_result>
```

Example 2 (Demo Pattern)

```
  options.note_policy = DataNotePolicy::Hash;
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

Example 3 (Demo Pattern)

```
}

Config i64() {
  return Config{}.with_dtype(ScalarType::Int64).with_platform(pluss::pCPU);
}

LabeledBatch slice_rows(const LabeledBatch& batch, std::int64_t start, std::int64_t␣
↪count) {
  const Series idx = pluss::arange(start, start + count, i64());
  const Series x = batch.features.x.index_select(0, idx).reshape({count, batch.features.
↪feature_count()});
  const Series y = batch.y.index_select(0, idx).reshape({count});
  return LabeledBatch{
      .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
      .y = y,
```

(continues on next page)

(continued from previous page)

```
  };
}

LabeledBatch make_class_batch() {
  std::vector<double> xv{};
  std::vector<double> yv{};
  xv.reserve(2 * 90);
  yv.reserve(90);
```

Expected output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

LexiconMapper::learn

API Signature

- `LexiconMapper::learn(documents: std::initializer_list<std::string>) -> Status`
- `LexiconMapper::learn(documents: RangeView::Of<std::string>) -> Status`

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

- `documents` (`RangeView::Of<std::string> | std::initializer_list<std::string>`) List-like argument; pass values in the order expected by the signature.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = LexiconMapper::learn(/* documents */);
if (!st.ok()) {
  out.error("learn failed", st.message);
} else {
  out.info("learn ok");
}
```

Expected output

```
learn: ok
status.ok() == true
```

Example 2 (Demo Pattern)

```
  auto bounded_or = make_classifier(linear_logistic_spec());
  if (!require_ok(bounded_or, "make_classifier(linear bounded)")) return 1;
  auto bounded = bounded_or.move_value();

  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }

  auto bounded_scores_or = bounded->infer_scores(batch.features);
  if (!require_ok(bounded_scores_or, "infer_scores(bounded)")) return 1;
  std::cout << "max |bounded score|: " << bounded_scores_or.value().abs().max().item
↪<double>() << "\n";

  std::cout << "NodeJ CV + tune demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪1.12262
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.169779
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.420312
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0248626
sweep trial objectives: 0 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0352983 0.743786
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 3 (Demo Pattern)

```
          .regularization_strength = 1e-8,
          .fit_intercept = true,
```

(continues on next page)

(continued from previous page)

```
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
  }

  auto metrics_or = reg->assess(batch, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metrics_or, "assess")) return 1;
  print_metrics("linear metrics [mse, r2]", metrics_or.value());

  std::cout << "NodeJ DataFrame bridge demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

`LexiconMapper::learned`

API Signature

- `LexiconMapper::learned() -> bool`

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = LexiconMapper::learned();
out.info("learned", value);
```

Expected output

```
learned: true
```

LexiconMapper::map_dense

API Signature

- `LexiconMapper::map_dense(documents: std::initializer_list<std::string>) -> StatusOr<FeatureBatch>`
- `LexiconMapper::map_dense(documents: RangeView::Of<std::string>) -> StatusOr<FeatureBatch>`

Notes

Applies preprocessing/transformation behavior configured by the current options.

Inputs

- `documents` (`RangeView::Of<std::string> | std::initializer_list<std::string>`) List-like argument; pass values in the order expected by the signature.

Output

- Returns `StatusOr<FeatureBatch>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = LexiconMapper::map_dense(/* documents */);
if (!res.ok()) {
  out.error("map_dense failed", res.status().message);
} else {
  out.info("map_dense ok");
}
```

Expected output

```
map_dense: ok
status.ok() == true
result.value() is available
```

LexiconMapper::map_sparse

API Signature

- `LexiconMapper::map_sparse(documents: std::initializer_list<std::string>) -> StatusOr<SparseTripletMatrix>`
- `LexiconMapper::map_sparse(documents: RangeView::Of<std::string>) -> StatusOr<SparseTripletMatrix>`

Notes

Applies preprocessing/transformation behavior configured by the current options.

Inputs

- `documents` (`RangeView::Of<std::string> | std::initializer_list<std::string>`) List-like argument; pass values in the order expected by the signature.

Output

- Returns `StatusOr<SparseTripletMatrix>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto res = LexiconMapper::map_sparse(/* documents */);
if (!res.ok()) {
  out.error("map_sparse failed", res.status().message);
} else {
  out.info("map_sparse ok");
}
```

Expected output

```
map_sparse: ok
status.ok() == true
result.value() is available
```

LexiconMapper::vocabulary

API Signature

- `LexiconMapper::vocabulary() -> const std::vector<std::string>&`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `const std::vector<std::string>&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = LexiconMapper::vocabulary();
out.info("vocabulary", value);
```

Expected output

```
vocabulary returned const std::vector<std::string>&
```

load

API Signature

- `load(path:  const std::string&) -> StatusOr<FlowBundle>`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `path` (`const std::string&`) Path to persisted model/data artifact.

Output

- Returns `StatusOr<FlowBundle>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = load(/* path */);
if (!res.ok()) {
  out.error("load failed", res.status().message);
} else {
  out.info("load ok");
}
```

Expected output

```
load: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
    auto best = std::move(tuned.best_workflow);
    best.learn_(train_x, train_y, train_options);
    const auto test_metrics = best.assess(test_x, test_y, assess_options);
    std::cout << "test metrics [rmse, r2, mae]: " << test_metrics.rmse << " " << test_
↪metrics.r2 << " " << test_metrics.mae << "\n";

    const Series pred = best.infer(test_x);
    const fs::path best_model_path = model_dir / "hour_best_workflow.bin";
    best.save(best_model_path.string(), train_x, train_y, train_options);
    auto restored = nja::Workflow::load(best_model_path.string());
    const Series pred_restored = restored.infer(test_x);
    const double restored_max_abs_diff = nja::max_abs_diff(pred, pred_restored);
    std::cout << "restored prediction max_abs_diff: " << restored_max_abs_diff << "\n";

    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});
    const Series y_pred = pred.to(pluss::pCPU, ScalarType::Float64).reshape({pred.
↪numel()});
    DataFrame pred_frame({
        {"y_true", Thread(y_true)},
        {"y_pred", Thread(y_pred)},
        {"residual", Thread(y_true.sub(y_pred))},
    });
    pred_frame.save_csv((out_dir / "hour_workflow_tuning_predictions.csv").string());
```

Expected output

```
CV objective metric: rmse
candidate=linear_ridge objective=102.872 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
```

(continues on next page)

(continued from previous page)

```
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[102.87158] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.41650024] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[77.611594]
candidate=decision_tree objective=44.9727 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[44.972693] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.88843998] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[27.61723]
candidate=random_forest objective=81.1235 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[81.123538] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.63710967] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
...
```

load_and_restore

API Signature

- `load_and_restore(path:  const std::string&) -> StatusOr<Flow>`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `path` (`const std::string&`) Path to persisted model/data artifact.

Output

- Returns `StatusOr<Flow>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = load_and_restore(/* path */);
if (!res.ok()) {
  out.error("load_and_restore failed", res.status().message);
} else {
```

(continues on next page)

(continued from previous page)

```
  out.info("load_and_restore ok");
}
```

Expected output

```
load_and_restore: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  const std::string bundle_path = model_dir + "/nodej_flow_bundle_demo.bin";
  auto save_status = nio::save(bundle_path, flow, batch, train);
  if (!save_status.ok()) {
    std::cerr << "save failed: " << save_status.message << "\n";
    return 1;
  }
  std::cout << "saved flow bundle to: " << bundle_path << "\n";

  auto restored_or = nio::load_and_restore(bundle_path);
  if (!restored_or.ok()) {
    std::cerr << "load_and_restore failed: " << restored_or.status().message << "\n";
    return 1;
  }
  auto pred_after_or = restored_or.value().infer(batch.features);
  if (!pred_after_or.ok()) {
    std::cerr << "restored flow infer failed: " << pred_after_or.status().message << "\n
↪";
    return 1;
  }

  const auto before = pred_before_or.value().to(pluss::pCPU, pluss::ScalarType::Float64).
↪to_vec<double>();
  const auto after = pred_after_or.value().to(pluss::pCPU, pluss::ScalarType::Float64).
↪to_vec<double>();
```

Expected output

```
libc++abi: terminating due to uncaught exception of type pluss::Error: DataReader::with_
↪sizes: explicit sizes are only supported for host-buffer inputs
Triggered at: <path> (DataReader &pluss::detail::DataReader::with_sizes(std::span<const␣
↪std::int64_t>))
Native stack trace:
  0  pluss::detail::DataReader::with_sizes(std::__1::span<long long const,␣
↪18446744073709551615ul>)(data_reader.h:250)
  1  pluss::codem::Series pluss::detail::DataReader::to_series<pluss::codem::Config␣
↪const&>(pluss::detail::DataReader::Option, pluss::codem::Config const&) const(data_
↪reader.h:295)
  2  pluss::codem::Series pluss::codem::value<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_value.h:13)
  3  pluss::codem::Series pluss::make_series<pluss::codem::Config const&>
```

(continues on next page)

(continued from previous page)

```
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_factory.h:960)
  4  main(nodej_flow_bundle_demo.cpp:13)
  5  start + 6992 [dyld]


[exit_code] -6
```

make_classifier

API Signature

- make_classifier(spec: const ModelSpec&) -> StatusOr<std::unique_ptr<ClassifierBase>>

Notes

Factory entry point that validates spec/options and returns a ready-to-train component.

Inputs

- spec (const ModelSpec&) Method argument; see signature type for accepted form.

Output

- Returns StatusOr<std::unique_ptr<ClassifierBase>>. Check .ok() first; on success read .value() for the computed result.

Options

Option semantics used by this method:

- ModelSpec (struct):
  - TaskKind task = TaskKind::Regression Selects task type (for example classification vs regression).
  - ModelFamily family = ModelFamily::Linear Selects the model family implementation.
  - FamilyOptions options = LinearFamilyOptions{} Family-specific options payload used by the selected model/transform family.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = make_classifier(/* spec */);
if (!res.ok()) {
  out.error("make_classifier failed", res.status().message);
} else {
  out.info("make_classifier ok");
}
```

Expected output

```
make_classifier: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
                          MetricKind::QuantileLoss,
                          TuneObjectivePolicy::Auto);
  if (!require_ok(reg_sweep_or, "sweep(regression quantile)")) return 1;
  std::cout << "regression quantile sweep objectives:";
  for (const auto& trial : reg_sweep_or.value().trials) std::cout << " " << trial.
↪objective;
  std::cout << "\n";
  std::cout << "regression quantile sweep best index: " << reg_sweep_or.value().best_
↪index << "\n";

  auto bounded_or = make_classifier(linear_logistic_spec());
  if (!require_ok(bounded_or, "make_classifier(linear bounded)")) return 1;
  auto bounded = bounded_or.move_value();

  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }

  auto bounded_scores_or = bounded->infer_scores(batch.features);
```

Expected output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪1.12262
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.169779
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.420312
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0248626
sweep trial objectives: 0 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0352983 0.743786
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 3 (Demo Pattern)

```
void print_metric(const std::string& label, const Series& m) {
  const auto v = m.to(pluss::pCPU, ScalarType::Float64).to_vec<double>();
  std::cout << label << ":";
  for (double x : v) std::cout << " " << x;
  std::cout << "\n";
}

void run_classifier_family(const std::string& name, const ModelSpec& spec, const␣
↪LabeledBatch& train) {
  auto model_or = make_classifier(spec);
  if (!require_ok(model_or, name + " make_classifier")) return;
  std::unique_ptr<ClassifierBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;

  auto pred_or = model->infer(train.features);
  if (!require_ok(pred_or, name + " infer")) return;
  auto metric_or = model->assess(train, AssessOptions{.metrics = {MetricKind::Accuracy,␣
↪MetricKind::LogLoss}});
  if (!require_ok(metric_or, name + " assess")) return;
  print_metric("  " + name + " (acc, logloss)", metric_or.value());
}

void run_regressor_family(const std::string& name, const ModelSpec& spec, const␣
↪LabeledBatch& train) {
```

Expected output

```
NodeJ model families demo
Classification families:
  linear (acc, logloss): 1 4.47877e-06
  linear_hinge (acc, logloss): 1 0.208572
  tree (acc, logloss): 1 9.99978e-13
  ensemble (acc, logloss): 1 1.00003e-12
  neighbor (acc, logloss): 1 9.99978e-13
  kernel (acc, logloss): 1 0.318438
  bayes (acc, logloss): 1 9.99978e-13
Regression families:
  linear (mse, r2): 1.52381e-19 1
  tree (mse, r2): 0 1
...
```

make_flow

API Signature

- make_flow(spec: const ModelSpec&) -> StatusOr<Flow>

Notes

Constructs a `Flow` from model spec/options; use it to chain transforms and model training.

Inputs

- `spec` (`const ModelSpec&`) Method argument; see signature type for accepted form.

Output

- Returns `StatusOr<Flow>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `ModelSpec` (`struct`):
  - `TaskKind task = TaskKind::Regression` Selects task type (for example classification vs regression).
  - `ModelFamily family = ModelFamily::Linear` Selects the model family implementation.
  - `FamilyOptions options = LinearFamilyOptions{}` Family-specific options payload used by the selected model/transform family.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = make_flow(/* spec */);
if (!res.ok()) {
  out.error("make_flow failed", res.status().message);
} else {
  out.info("make_flow ok");
}
```

Expected output

```
make_flow: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
      .options = LinearFamilyOptions{
          .objective = LinearObjective::Logistic,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-6,
          .fit_intercept = true,
      },
  };

  auto flow_or = make_flow(spec);
  if (!flow_or.ok()) {
```

(continues on next page)

(continued from previous page)

```
    std::cerr << "make_flow failed: " << flow_or.status().message << "\n";
    return 1;
  }
  Flow flow = flow_or.move_value();
  if (!flow.add_standard_scaler().ok()) {
    std::cerr << "add_standard_scaler failed\n";
    return 1;
  }
  TrainOptions train{};
  train.max_iterations = 2500;
  train.tolerance = 1e-8;
```

Expected output

```
libc++abi: terminating due to uncaught exception of type pluss::Error: DataReader::with_
↪sizes: explicit sizes are only supported for host-buffer inputs
Triggered at: <path> (DataReader &pluss::detail::DataReader::with_sizes(std::span<const␣
↪std::int64_t>))
Native stack trace:
  0  pluss::detail::DataReader::with_sizes(std::__1::span<long long const,␣
↪18446744073709551615ul>)(data_reader.h:250)
  1  pluss::codem::Series pluss::detail::DataReader::to_series<pluss::codem::Config␣
↪const&>(pluss::detail::DataReader::Option, pluss::codem::Config const&) const(data_
↪reader.h:295)
  2  pluss::codem::Series pluss::codem::value<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_value.h:13)
  3  pluss::codem::Series pluss::make_series<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_factory.h:960)
  4  main(nodej_flow_bundle_demo.cpp:13)
  5  start + 6992 [dyld]


[exit_code] -6
```

Example 3 (Demo Pattern)

```
        .options = LinearFamilyOptions{
            .objective = LinearObjective::Logistic,
            .regularization = RegularizationKind::L2,
            .regularization_strength = 1e-6,
            .fit_intercept = true,
        },
    };

    auto flow_or = make_flow(cls_spec);
    if (!require_ok(flow_or, "make_flow(classification)")) return 1;
    Flow cls = flow_or.move_value();
    if (!cls.add_simple_imputer(prep::SimpleImputerOptions{.strategy =␣
↪prep::SimpleImputerStrategy::Mean}).ok()) return 1;
```

(continues on next page)

(continued from previous page)

```
    if (!cls.add_standard_scaler().ok()) return 1;

    TrainOptions opts{};
    opts.max_iterations = 3000;
    opts.tolerance = 1e-8;
    if (!cls.learn(train, opts).ok()) {
      std::cerr << "classification flow learn failed\n";
      return 1;
    }
```

Expected output

```
NodeJ Flow demo
classification flow metrics [accuracy, logloss]: 1 1.44251e-05
regression flow metrics [mse, r2]: 4.46559e-17 1
NodeJ Flow demo completed.
```

make_regressor

API Signature

- `make_regressor(spec:  const ModelSpec&) -> StatusOr<std::unique_ptr<RegressorBase>>`

Notes

Factory entry point that validates spec/options and returns a ready-to-train component.

Inputs

- `spec` (`const ModelSpec&`) Method argument; see signature type for accepted form.

Output

- Returns `StatusOr<std::unique_ptr<RegressorBase>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `ModelSpec` (`struct`):
  - `TaskKind task = TaskKind::Regression` Selects task type (for example classification vs regression).
  - `ModelFamily family = ModelFamily::Linear` Selects the model family implementation.
  - `FamilyOptions options = LinearFamilyOptions{}` Family-specific options payload used by the selected model/transform family.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = make_regressor(/* spec */);
if (!res.ok()) {
  out.error("make_regressor failed", res.status().message);
} else {
  out.info("make_regressor ok");
}
```

Expected output

```
make_regressor: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
  }

  auto metrics_or = reg->assess(batch, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metrics_or, "assess")) return 1;
  print_metrics("linear metrics [mse, r2]", metrics_or.value());
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

Example 3 (Demo Pattern)

```
  auto pred_or = model->infer(train.features);
  if (!require_ok(pred_or, name + " infer")) return;
  auto metric_or = model->assess(train, AssessOptions{.metrics = {MetricKind::Accuracy,␣
↪MetricKind::LogLoss}});
  if (!require_ok(metric_or, name + " assess")) return;
```

(continues on next page)

(continued from previous page)

```
  print_metric("  " + name + " (acc, logloss)", metric_or.value());
}

void run_regressor_family(const std::string& name, const ModelSpec& spec, const␣
↪LabeledBatch& train) {
  auto model_or = make_regressor(spec);
  if (!require_ok(model_or, name + " make_regressor")) return;
  std::unique_ptr<RegressorBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;

  auto pred_or = model->infer(train.features);
  if (!require_ok(pred_or, name + " infer")) return;
  auto metric_or = model->assess(train, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metric_or, name + " assess")) return;
  print_metric("  " + name + " (mse, r2)", metric_or.value());
}

} // namespace
```

Expected output

```
NodeJ model families demo
Classification families:
  linear (acc, logloss): 1 4.47877e-06
  linear_hinge (acc, logloss): 1 0.208572
  tree (acc, logloss): 1 9.99978e-13
  ensemble (acc, logloss): 1 1.00003e-12
  neighbor (acc, logloss): 1 9.99978e-13
  kernel (acc, logloss): 1 0.318438
  bayes (acc, logloss): 1 9.99978e-13
Regression families:
  linear (mse, r2): 1.52381e-19 1
  tree (mse, r2): 0 1
...
```

make_sparse_triplet

API Signature

- make_sparse_triplet(rows: std::int64_t, cols: std::int64_t, row_indices: std::initializer_list<std::int64_t>, column_indices: std::initializer_list<std::int64_t>, values: std::initializer_list<double>) -> StatusOr<SparseTripletMatrix>
- make_sparse_triplet(rows: std::int64_t, cols: std::int64_t, row_indices: RangeView::Int, column_indices: RangeView::Int, values: RangeView::Of<double>) -> StatusOr<SparseTripletMatrix>

Notes

Namespace-level API function.

Inputs

- `cols` (`std::int64_t`) Method argument; see signature type for accepted form.
- `column_indices` (`RangeView::Int | std::initializer_list<std::int64_t>`) List-like argument; pass values in the order expected by the signature.
- `row_indices` (`RangeView::Int | std::initializer_list<std::int64_t>`) List-like argument; pass values in the order expected by the signature.
- `rows` (`std::int64_t`) Method argument; see signature type for accepted form.
- `values` (`RangeView::Of<double> | std::initializer_list<double>`) List-like argument; pass values in the order expected by the signature.

Output

- Returns `StatusOr<SparseTripletMatrix>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = make_sparse_triplet(/* rows */, /* cols */, /* row_indices */, /* column_
↪indices */, /* values */);
if (!res.ok()) {
  out.error("make_sparse_triplet failed", res.status().message);
} else {
  out.info("make_sparse_triplet ok");
}
```

Expected output

```
make_sparse_triplet: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
    return 1;
  }
  std::cout << "dense objective: " << tune_or.value().ledger.objective << "\n";
  std::cout << "dense ranks: " << tune_or.value().ledger.objective_rank << "\n";
  std::cout << "dense encoded params shape: "
            << tune_or.value().ledger.encoded_params.sizes()[0] << "x"
```

(continues on next page)

(continued from previous page)

```
            << tune_or.value().ledger.encoded_params.sizes()[1] << "\n";

  const auto sparse_or = make_sparse_triplet(
      6,
      3,
      {0, 1, 2, 3, 4, 5, 0, 1, 2, 3, 4, 5},
      {0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1},
      {-2.0, -1.0, 0.0, 1.0, 2.0, 3.0, 0.0, 0.0, 1.0, 1.0, 1.0, 1.0});
  if (!sparse_or.ok()) {
    std::cerr << sparse_or.status().message << "\n";
    return 1;
  }
  const std::vector<LinearFamilyOptions> sparse_candidates{
      LinearFamilyOptions{
          .objective = LinearObjective::Logistic,
```

Expected output

```
class scores: pluss::Series(shape=[3], strides=[1], dtype=float64, platform=cpu, numel=3,
↪ contiguous=true)
data=[0, 13.5, 0]
class rank: pluss::Series(shape=[3], strides=[1], dtype=int64, platform=cpu, numel=3,␣
↪contiguous=true)
data=[1, 0, 2]
picked feature ids: pluss::Series(shape=[2], strides=[1], dtype=int64, platform=cpu,␣
↪numel=2, contiguous=true)
data=[1, 0]
picked shape: 6x2
regression picked ids: pluss::Series(shape=[2], strides=[1], dtype=int64, platform=cpu,␣
↪numel=2, contiguous=true)
data=[1, 2]
dense objective: pluss::Series(shape=[2], strides=[1], dtype=float64, platform=cpu,␣
↪numel=2, contiguous=true)
data=[0.66666667, 0.5]
dense ranks: pluss::Series(shape=[2], strides=[1], dtype=int64, platform=cpu, numel=2,␣
↪contiguous=true)
...
```

make_transformer

API Signature

- make_transformer(spec: const ModelSpec&) -> StatusOr<std::unique_ptr<TransformerBase>>

Notes

Factory entry point that validates spec/options and returns a ready-to-train component.

Inputs

- `spec` (`const ModelSpec&`) Method argument; see signature type for accepted form.

Output

- Returns `StatusOr<std::unique_ptr<TransformerBase>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `ModelSpec` (`struct`):
  - `TaskKind task = TaskKind::Regression` Selects task type (for example classification vs regression).
  - `ModelFamily family = ModelFamily::Linear` Selects the model family implementation.
  - `FamilyOptions options = LinearFamilyOptions{}` Family-specific options payload used by the selected model/transform family.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = make_transformer(/* spec */);
if (!res.ok()) {
  out.error("make_transformer failed", res.status().message);
} else {
  out.info("make_transformer ok");
}
```

Expected output

```
make_transformer: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
      .options = ClusterFamilyOptions{
          .method = ClusterMethod::KMeans,
          .clusters = 2,
          .max_iterations = 200,
          .restarts = 5,
          .tolerance = 1e-5,
      },
  };
  auto kmeans_or = make_transformer(kmeans_spec);
```

(continues on next page)

(continued from previous page)

```
  if (!kmeans_or.ok()) {
    std::cerr << "make_transformer(kmeans) failed: " << kmeans_or.status().message << "\n
↪";
    return 1;
  }
  auto kmeans = kmeans_or.move_value();
  if (!kmeans->learn(features).ok()) {
    std::cerr << "kmeans learn failed\n";
    return 1;
  }
  auto labels_or = kmeans->map(features);
  if (!labels_or.ok()) {
    std::cerr << "kmeans map failed: " << labels_or.status().message << "\n";
```

Expected output

```
kmeans labels (first 10): 0 0 0 0 0 0 0 0 0 0
pca output shape: [80, 2]
nodej_unsupervised_demo complete
```

make_workflow

API Signature

- `make_workflow(spec:  const ModelSpec&) -> StatusOr<Workflow>`

Notes

Constructs a high-level workflow object with model and transform stages.

Inputs

- `spec` (`const ModelSpec&`) Method argument; see signature type for accepted form.

Output

- Returns `StatusOr<Workflow>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `ModelSpec` (`struct`):
  - `TaskKind task = TaskKind::Regression` Selects task type (for example classification vs regression).
  - `ModelFamily family = ModelFamily::Linear` Selects the model family implementation.
  - `FamilyOptions options = LinearFamilyOptions{}` Family-specific options payload used by the selected model/transform family.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = make_workflow(/* spec */);
if (!res.ok()) {
  out.error("make_workflow failed", res.status().message);
} else {
  out.info("make_workflow ok");
}
```

Expected output

```
make_workflow: ok
status.ok() == true
result.value() is available
```

MaxAbsScaler::dump_state

API Signature

- `MaxAbsScaler::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = MaxAbsScaler::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

`MaxAbsScaler::load_state`

API Signature

- `MaxAbsScaler::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = MaxAbsScaler::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

MaxAbsScaler::trained

API Signature

- `MaxAbsScaler::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = MaxAbsScaler::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

MinMaxScaler::dump_state

API Signature

- `MinMaxScaler::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = MinMaxScaler::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
```

(continues on next page)

(continued from previous page)

```
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

MinMaxScaler::load_state

API Signature

- `MinMaxScaler::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = MinMaxScaler::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

MinMaxScaler::trained

API Signature

- `MinMaxScaler::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = MinMaxScaler::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

NeighborImputer::dump_state

API Signature

- `NeighborImputer::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = NeighborImputer::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
```

(continues on next page)

(continued from previous page)

```
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

`NeighborImputer::load_state`

API Signature

- `NeighborImputer::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = NeighborImputer::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
```

(continues on next page)

(continued from previous page)

```
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

NeighborImputer::trained

API Signature

- `NeighborImputer::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = NeighborImputer::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

OneHotEncoder::dump_state

API Signature

- `OneHotEncoder::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = OneHotEncoder::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();
```

(continues on next page)

(continued from previous page)

```
  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

OneHotEncoder::load_state

API Signature

- `OneHotEncoder::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = OneHotEncoder::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

`OneHotEncoder::trained`

API Signature

- `OneHotEncoder::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OneHotEncoder::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

`OnlineLinearClassifier::seen_samples`

API Signature

- `OnlineLinearClassifier::seen_samples() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OnlineLinearClassifier::seen_samples();
out.info("seen_samples", value);
```

Expected output

```
seen_samples: <numeric_result>
```

Example 2 (Demo Pattern)

```
  pluss::ai::nodej::AssessOptions assess{};
  assess.metrics = {MetricKind::Accuracy, MetricKind::LogLoss};
  auto metric_or = model.assess(full, assess);
  if (!metric_or.ok()) {
    std::cerr << "assess failed: " << metric_or.status().message << '\n';
    return 1;
  }

  std::cout << "online seen_samples=" << model.seen_samples() << '\n';
  std::cout << "online metrics [accuracy, logloss]: ";
  print_vec(metric_or.value());
  return 0;
}
```

Expected output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

Example 3 (Demo Pattern)

```
    warm_opts.max_iterations = 80;

    auto model = pluss::ai::lymph::online_factory::linear_classifier(online_options);
    model.learn_(warm, warm_opts).advance_(stream);

    const auto metrics = model.assess(full);
    const Series metric_values = metrics.accuracy.defined() ? metrics.accuracy : metrics.
↪values;

    std::cout << "lymph online seen_samples=" << model.seen_samples() << '\n';
    std::cout << "lymph online metrics [accuracy]: ";
    print_vec(metric_values);
    return 0;
```

(continues on next page)

(continued from previous page)

```
  } catch (const std::exception& ex) {
    std::cerr << "lymph_online_demo failed: " << ex.what() << '\n';
    return 1;
  }
}
```

Expected output

```
lymph online seen_samples=1660
lymph online metrics [accuracy]: 0.9875
```

`OnlineLinearClassifier::trained`

API Signature

- `OnlineLinearClassifier::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OnlineLinearClassifier::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

`OnlineLinearRegressor::seen_samples`

API Signature

- `OnlineLinearRegressor::seen_samples() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OnlineLinearRegressor::seen_samples();
out.info("seen_samples", value);
```

Expected output

```
seen_samples: <numeric_result>
```

Example 2 (Demo Pattern)

```
  pluss::ai::nodej::AssessOptions assess{};
  assess.metrics = {MetricKind::Accuracy, MetricKind::LogLoss};
  auto metric_or = model.assess(full, assess);
  if (!metric_or.ok()) {
    std::cerr << "assess failed: " << metric_or.status().message << '\n';
    return 1;
  }

  std::cout << "online seen_samples=" << model.seen_samples() << '\n';
  std::cout << "online metrics [accuracy, logloss]: ";
  print_vec(metric_or.value());
  return 0;
}
```

Expected output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

Example 3 (Demo Pattern)

```
    warm_opts.max_iterations = 80;

    auto model = pluss::ai::lymph::online_factory::linear_classifier(online_options);
    model.learn_(warm, warm_opts).advance_(stream);

    const auto metrics = model.assess(full);
    const Series metric_values = metrics.accuracy.defined() ? metrics.accuracy : metrics.
↪values;

    std::cout << "lymph online seen_samples=" << model.seen_samples() << '\n';
    std::cout << "lymph online metrics [accuracy]: ";
    print_vec(metric_values);
    return 0;
  } catch (const std::exception& ex) {
    std::cerr << "lymph_online_demo failed: " << ex.what() << '\n';
    return 1;
  }
}
```

Expected output

```
lymph online seen_samples=1660
lymph online metrics [accuracy]: 0.9875
```

`OnlineLinearRegressor::trained`

API Signature

- `OnlineLinearRegressor::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OnlineLinearRegressor::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

`OrdinalEncoder::dump_state`

API Signature

- `OrdinalEncoder::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = OrdinalEncoder::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

`OrdinalEncoder::load_state`

API Signature

- `OrdinalEncoder::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = OrdinalEncoder::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

`OrdinalEncoder::trained`

API Signature

- `OrdinalEncoder::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OrdinalEncoder::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

`OutlierIsolator::decision_offset`

API Signature

- `OutlierIsolator::decision_offset() -> double`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `double`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OutlierIsolator::decision_offset();
out.info("decision_offset", value);
```

Expected output

```
decision_offset: <numeric_result>
```

`OutlierIsolator::learned`

API Signature

- `OutlierIsolator::learned() -> bool`

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OutlierIsolator::learned();
out.info("learned", value);
```

Expected output

```
learned: true
```

`OutlierIsolator::max_features_used`

API Signature

- `OutlierIsolator::max_features_used() -> std::int64_t`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OutlierIsolator::max_features_used();
out.info("max_features_used", value);
```

Expected output

```
max_features_used: <numeric_result>
```

`OutlierIsolator::max_samples_used`

API Signature

- `OutlierIsolator::max_samples_used() -> std::int64_t`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OutlierIsolator::max_samples_used();
out.info("max_samples_used", value);
```

Expected output

```
max_samples_used: <numeric_result>
```

`PolynomialLift::dump_state`

API Signature

- `PolynomialLift::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = PolynomialLift::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
```

(continues on next page)

(continued from previous page)

```
result.value() is available
```

PolynomialLift::load_state

API Signature

- `PolynomialLift::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto st = PolynomialLift::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

PolynomialLift::trained

API Signature

- `PolynomialLift::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = PolynomialLift::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

`PowerMap::dump_state`

API Signature

- `PowerMap::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = PowerMap::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

PowerMap::load_state

API Signature

- `PowerMap::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = PowerMap::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

`PowerMap::trained`

API Signature

- `PowerMap::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = PowerMap::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

QuantileMap::dump_state

API Signature

- `QuantileMap::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = QuantileMap::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

`QuantileMap::load_state`

API Signature

- `QuantileMap::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = QuantileMap::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

QuantileMap::trained

API Signature

- `QuantileMap::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = QuantileMap::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

RegressorBase::task_kind

API Signature

- `RegressorBase::task_kind() -> TaskKind`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `TaskKind`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = RegressorBase::task_kind();
out.info("task_kind", value);
```

Expected output

```
task_kind returned TaskKind
```

restore

API Signature

- `restore(bundle:  const FlowBundle&) -> StatusOr<Flow>`

Notes

Namespace-level API function.

Inputs

- `bundle` (`const FlowBundle&`) Method argument; see signature type for accepted form.

Output

- Returns `StatusOr<Flow>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `FlowBundle` (`struct`):
  - `FlowBundleMetadata metadata{}` Learning-rate step size.
  - `ModelSpec model{}` Option field used by this method.
  - `std::vector<TransformSpec> transforms{}` Option field used by this method.
  - `bool trained = false` Option field used by this method.
  - `bool has_snapshot = false` Option field used by this method.
  - `std::vector<std::vector<Series>> transform_states{}` Option field used by this method.
  - `std::vector<Series> model_state{}` Option field used by this method.
  - `TrainOptions train_options{}` Controls fold/trial/count sizing.
  - `LabeledBatch train_batch{}` Controls fold/trial/count sizing.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = restore(/* bundle */);
if (!res.ok()) {
  out.error("restore failed", res.status().message);
} else {
  out.info("restore ok");
}
```

Expected output

```
restore: ok
status.ok() == true
result.value() is available
```

RobustScaler::dump_state

API Signature

- `RobustScaler::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = RobustScaler::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
```

(continues on next page)

(continued from previous page)

```
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

RobustScaler::load_state

API Signature

- `RobustScaler::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = RobustScaler::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

RobustScaler::trained

API Signature

- `RobustScaler::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = RobustScaler::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

SimpleImputer::dump_state

API Signature

- `SimpleImputer::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = SimpleImputer::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
```

(continues on next page)

(continued from previous page)

```
result.value() is available
```

SimpleImputer::load_state

API Signature

- `SimpleImputer::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto st = SimpleImputer::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

SimpleImputer::trained

API Signature

- `SimpleImputer::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = SimpleImputer::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

slice_feature_batch

API Signature

- `slice_feature_batch(in: const FeatureBatch&, indices: const Series&) -> StatusOr<FeatureBatch>`

Notes

Returns a row-subset of a feature-only batch using the provided index selection.

Inputs

- `in` (`const FeatureBatch&`) Method argument; see signature type for accepted form.
- `indices` (`const Series&`) Pluss numeric vector/tensor input.

Output

- Returns `StatusOr<FeatureBatch>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `FeatureBatch` (`struct`):
  - `Series x = Series::undefined()` Option field used by this method.
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = slice_feature_batch(/* in */, /* indices */);
if (!res.ok()) {
  out.error("slice_feature_batch failed", res.status().message);
} else {
  out.info("slice_feature_batch ok");
}
```

Expected output

```
slice_feature_batch: ok
status.ok() == true
result.value() is available
```

slice_labeled_batch

API Signature

- `slice_labeled_batch(in: const LabeledBatch&, indices: const Series&) -> StatusOr<LabeledBatch>`

Notes

Returns a row-subset of a labeled batch using the provided index selection.

Inputs

- `in` (`const LabeledBatch&`) Method argument; see signature type for accepted form.
- `indices` (`const Series&`) Pluss numeric vector/tensor input.

Output

- Returns `StatusOr<LabeledBatch>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `LabeledBatch` (`struct`):
  - `FeatureBatch features{}` Option field used by this method.
  - `Series y = Series::undefined()` Option field used by this method.
  - `Series sample_weights = Series::undefined()` Optional column used as per-row training weight.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = slice_labeled_batch(/* in */, /* indices */);
if (!res.ok()) {
  out.error("slice_labeled_batch failed", res.status().message);
} else {
  out.info("slice_labeled_batch ok");
}
```

Expected output

```
slice_labeled_batch: ok
status.ok() == true
result.value() is available
```

sparse_triplet_to_csr

API Signature

- `sparse_triplet_to_csr(sparse: const SparseTripletMatrix&) -> StatusOr<SparseCsrMatrix>`

Notes

Namespace-level API function.

Inputs

- `sparse` (`const SparseTripletMatrix&`) Method argument; see signature type for accepted form.

Output

- Returns `StatusOr<SparseCsrMatrix>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `SparseTripletMatrix` (`struct`):
  - `Series row_indices = Series::undefined()` Option field used by this method.
  - `Series column_indices = Series::undefined()` Controls fold/trial/count sizing.
  - `Series values = Series::undefined()` Option field used by this method.
  - `std::int64_t rows = 0` Option field used by this method.
  - `std::int64_t cols = 0` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = sparse_triplet_to_csr(/* sparse */);
if (!res.ok()) {
  out.error("sparse_triplet_to_csr failed", res.status().message);
} else {
  out.info("sparse_triplet_to_csr ok");
}
```

Expected output

```
sparse_triplet_to_csr: ok
status.ok() == true
result.value() is available
```

SparseLinearBinaryClassifier::dump_state

API Signature

- `SparseLinearBinaryClassifier::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = SparseLinearBinaryClassifier::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

SparseLinearBinaryClassifier::feature_count

API Signature

- `SparseLinearBinaryClassifier::feature_count() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = SparseLinearBinaryClassifier::feature_count();
out.info("feature_count", value);
```

Expected output

```
feature_count: <numeric_result>
```

Example 2 (Demo Pattern)

```
  options.note_policy = DataNotePolicy::Hash;
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

Example 3 (Demo Pattern)

```
}

Config i64() {
  return Config{}.with_dtype(ScalarType::Int64).with_platform(pluss::pCPU);
}

LabeledBatch slice_rows(const LabeledBatch& batch, std::int64_t start, std::int64_t␣
↪count) {
  const Series idx = pluss::arange(start, start + count, i64());
  const Series x = batch.features.x.index_select(0, idx).reshape({count, batch.features.
↪feature_count()});
  const Series y = batch.y.index_select(0, idx).reshape({count});
  return LabeledBatch{
      .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
      .y = y,
  };
}

LabeledBatch make_class_batch() {
  std::vector<double> xv{};
  std::vector<double> yv{};
  xv.reserve(2 * 90);
  yv.reserve(90);
```

Expected output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

SparseLinearBinaryClassifier::infer_scores

API Signature

- `SparseLinearBinaryClassifier::infer_scores(x: const SparseCsrMatrix&) -> StatusOr<Series>`
- `SparseLinearBinaryClassifier::infer_scores(x: const SparseTripletMatrix&) -> StatusOr<Series>`

Notes

Computes predictions from prepared inputs using the currently learned state.

Inputs

- `x` (`const SparseCsrMatrix& | const SparseTripletMatrix&`) Input feature data used by this call.

Output

- Returns `StatusOr<Series>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `SparseCsrMatrix` (`struct`):
  - `std::vector<std::int64_t> row_ptr{}` Option field used by this method.
  - `std::vector<std::int64_t> col_ind{}` Option field used by this method.
  - `std::vector<double> values{}` Option field used by this method.
  - `std::int64_t rows = 0` Option field used by this method.
  - `std::int64_t cols = 0` Option field used by this method.
- `SparseTripletMatrix` (`struct`):
  - `Series row_indices = Series::undefined()` Option field used by this method.
  - `Series column_indices = Series::undefined()` Controls fold/trial/count sizing.
  - `Series values = Series::undefined()` Option field used by this method.
  - `std::int64_t rows = 0` Option field used by this method.
  - `std::int64_t cols = 0` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = SparseLinearBinaryClassifier::infer_scores(/* x */);
if (!res.ok()) {
  out.error("infer_scores failed", res.status().message);
} else {
  out.info("infer_scores ok");
}
```

Expected output

```
infer_scores: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }

  auto bounded_scores_or = bounded->infer_scores(batch.features);
```

(continues on next page)

(continued from previous page)

```
  if (!require_ok(bounded_scores_or, "infer_scores(bounded)")) return 1;
  std::cout << "max |bounded score|: " << bounded_scores_or.value().abs().max().item
↪<double>() << "\n";

  std::cout << "NodeJ CV + tune demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪1.12262
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.169779
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.420312
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0248626
sweep trial objectives: 0 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0352983 0.743786
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 3 (Demo Pattern)

```
  }
  auto cls = cls_or.move_value();
  const auto learn_status = cls->learn(batch);
  if (!learn_status.ok()) {
    std::cerr << "learn failed: " << learn_status.message << '\n';
    return 1;
  }

  auto score_or = cls->infer_scores(batch.features);
  if (!score_or.ok()) {
    std::cerr << "infer_scores failed: " << score_or.status().message << '\n';
    return 1;
  }
  std::cout << "scores shape: [" << score_or.value().sizes()[0] << ", " << score_or.
↪value().sizes()[1] << "]\n";

  pluss::ai::nodej::AssessOptions assess{};
  assess.metrics = {MetricKind::Accuracy, MetricKind::F1, MetricKind::LogLoss};
  auto metric_or = cls->assess(batch, assess);
  if (!metric_or.ok()) {
    std::cerr << "assess failed: " << metric_or.status().message << '\n';
    return 1;
```

Expected output

```
scores shape: [12, 3]
metrics [accuracy, f1, logloss]: 1 1 1.24859e-10
flow metrics [accuracy, f1, logloss]: 1 1 1.48016e-09
```

SparseLinearBinaryClassifier::load_state

API Signature

- `SparseLinearBinaryClassifier::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto st = SparseLinearBinaryClassifier::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

SparseLinearBinaryClassifier::trained

API Signature

- `SparseLinearBinaryClassifier::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = SparseLinearBinaryClassifier::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

SparseLinearMultiClassifier::class_count

API Signature

- `SparseLinearMultiClassifier::class_count() -> std::int64_t`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = SparseLinearMultiClassifier::class_count();
out.info("class_count", value);
```

Expected output

```
class_count: <numeric_result>
```

SparseLinearMultiClassifier::dump_state

API Signature

- `SparseLinearMultiClassifier::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = SparseLinearMultiClassifier::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

SparseLinearMultiClassifier::feature_count

API Signature

- `SparseLinearMultiClassifier::feature_count() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = SparseLinearMultiClassifier::feature_count();
out.info("feature_count", value);
```

Expected output

```
feature_count: <numeric_result>
```

Example 2 (Demo Pattern)

```
  options.note_policy = DataNotePolicy::Hash;
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

Example 3 (Demo Pattern)

```
}

Config i64() {
```

(continues on next page)

(continued from previous page)

```
  return Config{}.with_dtype(ScalarType::Int64).with_platform(pluss::pCPU);
}

LabeledBatch slice_rows(const LabeledBatch& batch, std::int64_t start, std::int64_t␣
↪count) {
  const Series idx = pluss::arange(start, start + count, i64());
  const Series x = batch.features.x.index_select(0, idx).reshape({count, batch.features.
↪feature_count()});
  const Series y = batch.y.index_select(0, idx).reshape({count});
  return LabeledBatch{
      .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
      .y = y,
  };
}

LabeledBatch make_class_batch() {
  std::vector<double> xv{};
  std::vector<double> yv{};
  xv.reserve(2 * 90);
  yv.reserve(90);
```

Expected output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

SparseLinearMultiClassifier::infer

API Signature

- `SparseLinearMultiClassifier::infer(x: const SparseCsrMatrix&) -> StatusOr<Series>`
- `SparseLinearMultiClassifier::infer(x: const SparseTripletMatrix&) -> StatusOr<Series>`

Notes

Computes predictions from prepared inputs using the currently learned state.

Inputs

- `x` (`const SparseCsrMatrix& | const SparseTripletMatrix&`) Input feature data used by this call.

Output

- Returns `StatusOr<Series>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `SparseCsrMatrix` (`struct`):
  - `std::vector<std::int64_t> row_ptr{}` Option field used by this method.

  - `std::vector<std::int64_t> col_ind{}` Option field used by this method.
  - `std::vector<double> values{}` Option field used by this method.
  - `std::int64_t rows = 0` Option field used by this method.
  - `std::int64_t cols = 0` Option field used by this method.
- `SparseTripletMatrix` (`struct`):
  - `Series row_indices = Series::undefined()` Option field used by this method.
  - `Series column_indices = Series::undefined()` Controls fold/trial/count sizing.
  - `Series values = Series::undefined()` Option field used by this method.
  - `std::int64_t rows = 0` Option field used by this method.
  - `std::int64_t cols = 0` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = SparseLinearMultiClassifier::infer(/* x */);
if (!res.ok()) {
  out.error("infer failed", res.status().message);
} else {
  out.info("infer ok");
}
```

Expected output

```
infer: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  train.tolerance = 1e-8;
  train.random.seed = 77;
  train.random.deterministic = true;
  if (!flow.learn(batch, train).ok()) {
    std::cerr << "flow learn failed\n";
    return 1;
  }

  auto pred_before_or = flow.infer(batch.features);
  if (!pred_before_or.ok()) {
    std::cerr << "flow infer before save failed: " << pred_before_or.status().message <<
↪"\n";
    return 1;
  }
  const std::string model_dir = "<repo_root>/PlussAI/examples/NodeJ/trained_models";
  std::error_code mkdir_ec;
  std::filesystem::create_directories(model_dir, mkdir_ec);
```

(continues on next page)

(continued from previous page)

```
  if (mkdir_ec) {
    std::cerr << "failed to create model directory: " << model_dir << " (" << mkdir_ec.
↪message() << ")\n";
    return 1;
  }
  const std::string bundle_path = model_dir + "/nodej_flow_bundle_demo.bin";
```

Expected output

```
libc++abi: terminating due to uncaught exception of type pluss::Error: DataReader::with_
↪sizes: explicit sizes are only supported for host-buffer inputs
Triggered at: <path> (DataReader &pluss::detail::DataReader::with_sizes(std::span<const␣
↪std::int64_t>))
Native stack trace:
  0  pluss::detail::DataReader::with_sizes(std::__1::span<long long const,␣
↪18446744073709551615ul>)(data_reader.h:250)
  1  pluss::codem::Series pluss::detail::DataReader::to_series<pluss::codem::Config␣
↪const&>(pluss::detail::DataReader::Option, pluss::codem::Config const&) const(data_
↪reader.h:295)
  2  pluss::codem::Series pluss::codem::value<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_value.h:13)
  3  pluss::codem::Series pluss::make_series<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_factory.h:960)
  4  main(nodej_flow_bundle_demo.cpp:13)
  5  start + 6992 [dyld]


[exit_code] -6
```

Example 3 (Demo Pattern)

```
}

void run_classifier_family(const std::string& name, const ModelSpec& spec, const␣
↪LabeledBatch& train) {
  auto model_or = make_classifier(spec);
  if (!require_ok(model_or, name + " make_classifier")) return;
  std::unique_ptr<ClassifierBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;

  auto pred_or = model->infer(train.features);
  if (!require_ok(pred_or, name + " infer")) return;
  auto metric_or = model->assess(train, AssessOptions{.metrics = {MetricKind::Accuracy,␣
↪MetricKind::LogLoss}});
  if (!require_ok(metric_or, name + " assess")) return;
  print_metric("  " + name + " (acc, logloss)", metric_or.value());
}

void run_regressor_family(const std::string& name, const ModelSpec& spec, const␣
```

(continues on next page)

(continued from previous page)

```
↪LabeledBatch& train) {
  auto model_or = make_regressor(spec);
  if (!require_ok(model_or, name + " make_regressor")) return;
  std::unique_ptr<RegressorBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;
```

Expected output

```
NodeJ model families demo
Classification families:
  linear (acc, logloss): 1 4.47877e-06
  linear_hinge (acc, logloss): 1 0.208572
  tree (acc, logloss): 1 9.99978e-13
  ensemble (acc, logloss): 1 1.00003e-12
  neighbor (acc, logloss): 1 9.99978e-13
  kernel (acc, logloss): 1 0.318438
  bayes (acc, logloss): 1 9.99978e-13
Regression families:
  linear (mse, r2): 1.52381e-19 1
  tree (mse, r2): 0 1
...
```

SparseLinearMultiClassifier::infer_scores

API Signature

- `SparseLinearMultiClassifier::infer_scores(x: const SparseCsrMatrix&) -> StatusOr<Series>`
- `SparseLinearMultiClassifier::infer_scores(x: const SparseTripletMatrix&) -> StatusOr<Series>`

Notes

Computes predictions from prepared inputs using the currently learned state.

Inputs

- `x` (`const SparseCsrMatrix& | const SparseTripletMatrix&`) Input feature data used by this call.

Output

- Returns `StatusOr<Series>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `SparseCsrMatrix` (`struct`):
  - `std::vector<std::int64_t> row_ptr{}` Option field used by this method.
  - `std::vector<std::int64_t> col_ind{}` Option field used by this method.
  - `std::vector<double> values{}` Option field used by this method.

  - `std::int64_t rows = 0` Option field used by this method.
  - `std::int64_t cols = 0` Option field used by this method.
- `SparseTripletMatrix` (struct):
  - `Series row_indices = Series::undefined()` Option field used by this method.
  - `Series column_indices = Series::undefined()` Controls fold/trial/count sizing.
  - `Series values = Series::undefined()` Option field used by this method.
  - `std::int64_t rows = 0` Option field used by this method.
  - `std::int64_t cols = 0` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = SparseLinearMultiClassifier::infer_scores(/* x */);
if (!res.ok()) {
  out.error("infer_scores failed", res.status().message);
} else {
  out.info("infer_scores ok");
}
```

Expected output

```
infer_scores: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }

  auto bounded_scores_or = bounded->infer_scores(batch.features);
  if (!require_ok(bounded_scores_or, "infer_scores(bounded)")) return 1;
  std::cout << "max |bounded score|: " << bounded_scores_or.value().abs().max().item
↪<double>() << "\n";

  std::cout << "NodeJ CV + tune demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪1.12262
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.169779
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.420312
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0248626
sweep trial objectives: 0 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0352983 0.743786
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 3 (Demo Pattern)

```
  }
  auto cls = cls_or.move_value();
  const auto learn_status = cls->learn(batch);
  if (!learn_status.ok()) {
    std::cerr << "learn failed: " << learn_status.message << '\n';
    return 1;
  }

  auto score_or = cls->infer_scores(batch.features);
  if (!score_or.ok()) {
    std::cerr << "infer_scores failed: " << score_or.status().message << '\n';
    return 1;
  }
  std::cout << "scores shape: [" << score_or.value().sizes()[0] << ", " << score_or.
↪value().sizes()[1] << "]\n";

  pluss::ai::nodej::AssessOptions assess{};
  assess.metrics = {MetricKind::Accuracy, MetricKind::F1, MetricKind::LogLoss};
  auto metric_or = cls->assess(batch, assess);
  if (!metric_or.ok()) {
    std::cerr << "assess failed: " << metric_or.status().message << '\n';
    return 1;
```

Expected output

```
scores shape: [12, 3]
metrics [accuracy, f1, logloss]: 1 1 1.24859e-10
flow metrics [accuracy, f1, logloss]: 1 1 1.48016e-09
```

SparseLinearMultiClassifier::load_state

API Signature

- `SparseLinearMultiClassifier::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = SparseLinearMultiClassifier::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

SparseLinearMultiClassifier::trained

API Signature

- `SparseLinearMultiClassifier::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = SparseLinearMultiClassifier::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

SparseLinearRegressor::dump_state

API Signature

- `SparseLinearRegressor::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = SparseLinearRegressor::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

SparseLinearRegressor::feature_count

API Signature

- `SparseLinearRegressor::feature_count() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = SparseLinearRegressor::feature_count();
out.info("feature_count", value);
```

Expected output

```
feature_count: <numeric_result>
```

Example 2 (Demo Pattern)

```
  options.note_policy = DataNotePolicy::Hash;
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
```

(continues on next page)

(continued from previous page)

```
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

Example 3 (Demo Pattern)

```
}

Config i64() {
  return Config{}.with_dtype(ScalarType::Int64).with_platform(pluss::pCPU);
}

LabeledBatch slice_rows(const LabeledBatch& batch, std::int64_t start, std::int64_t␣
↪count) {
  const Series idx = pluss::arange(start, start + count, i64());
  const Series x = batch.features.x.index_select(0, idx).reshape({count, batch.features.
↪feature_count()});
  const Series y = batch.y.index_select(0, idx).reshape({count});
  return LabeledBatch{
      .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
      .y = y,
  };
}

LabeledBatch make_class_batch() {
  std::vector<double> xv{};
  std::vector<double> yv{};
  xv.reserve(2 * 90);
  yv.reserve(90);
```

Expected output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

SparseLinearRegressor::infer

API Signature

- SparseLinearRegressor::infer(x: const SparseCsrMatrix&) -> StatusOr<Series>
- SparseLinearRegressor::infer(x: const SparseTripletMatrix&) -> StatusOr<Series>

Notes

Computes predictions from prepared inputs using the currently learned state.

Inputs

- x (const SparseCsrMatrix& | const SparseTripletMatrix&) Input feature data used by this call.

Output

- Returns `StatusOr<Series>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `SparseCsrMatrix` (`struct`):
  - `std::vector<std::int64_t> row_ptr{}` Option field used by this method.
  - `std::vector<std::int64_t> col_ind{}` Option field used by this method.
  - `std::vector<double> values{}` Option field used by this method.
  - `std::int64_t rows = 0` Option field used by this method.
  - `std::int64_t cols = 0` Option field used by this method.
- `SparseTripletMatrix` (`struct`):
  - `Series row_indices = Series::undefined()` Option field used by this method.
  - `Series column_indices = Series::undefined()` Controls fold/trial/count sizing.
  - `Series values = Series::undefined()` Option field used by this method.
  - `std::int64_t rows = 0` Option field used by this method.
  - `std::int64_t cols = 0` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = SparseLinearRegressor::infer(/* x */);
if (!res.ok()) {
  out.error("infer failed", res.status().message);
} else {
  out.info("infer ok");
}
```

Expected output

```
infer: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  train.tolerance = 1e-8;
  train.random.seed = 77;
  train.random.deterministic = true;
  if (!flow.learn(batch, train).ok()) {
    std::cerr << "flow learn failed\n";
    return 1;
  }
```

(continues on next page)

(continued from previous page)

```
  auto pred_before_or = flow.infer(batch.features);
  if (!pred_before_or.ok()) {
    std::cerr << "flow infer before save failed: " << pred_before_or.status().message <<
↪"\n";
    return 1;
  }
  const std::string model_dir = "<repo_root>/PlussAI/examples/NodeJ/trained_models";
  std::error_code mkdir_ec;
  std::filesystem::create_directories(model_dir, mkdir_ec);
  if (mkdir_ec) {
    std::cerr << "failed to create model directory: " << model_dir << " (" << mkdir_ec.
↪message() << ")\n";
    return 1;
  }
  const std::string bundle_path = model_dir + "/nodej_flow_bundle_demo.bin";
```

Expected output

```
libc++abi: terminating due to uncaught exception of type pluss::Error: DataReader::with_
↪sizes: explicit sizes are only supported for host-buffer inputs
Triggered at: <path> (DataReader &pluss::detail::DataReader::with_sizes(std::span<const␣
↪std::int64_t>))
Native stack trace:
  0  pluss::detail::DataReader::with_sizes(std::__1::span<long long const,␣
↪18446744073709551615ul>)(data_reader.h:250)
  1  pluss::codem::Series pluss::detail::DataReader::to_series<pluss::codem::Config␣
↪const&>(pluss::detail::DataReader::Option, pluss::codem::Config const&) const(data_
↪reader.h:295)
  2  pluss::codem::Series pluss::codem::value<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_value.h:13)
  3  pluss::codem::Series pluss::make_series<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_factory.h:960)
  4  main(nodej_flow_bundle_demo.cpp:13)
  5  start + 6992 [dyld]


[exit_code] -6
```

Example 3 (Demo Pattern)

```
}

void run_classifier_family(const std::string& name, const ModelSpec& spec, const␣
↪LabeledBatch& train) {
  auto model_or = make_classifier(spec);
  if (!require_ok(model_or, name + " make_classifier")) return;
  std::unique_ptr<ClassifierBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;
```

(continues on next page)

(continued from previous page)

```
  auto pred_or = model->infer(train.features);
  if (!require_ok(pred_or, name + " infer")) return;
  auto metric_or = model->assess(train, AssessOptions{.metrics = {MetricKind::Accuracy,␣
↪MetricKind::LogLoss}});
  if (!require_ok(metric_or, name + " assess")) return;
  print_metric("  " + name + " (acc, logloss)", metric_or.value());
}

void run_regressor_family(const std::string& name, const ModelSpec& spec, const␣
↪LabeledBatch& train) {
  auto model_or = make_regressor(spec);
  if (!require_ok(model_or, name + " make_regressor")) return;
  std::unique_ptr<RegressorBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;
```

Expected output

```
NodeJ model families demo
Classification families:
  linear (acc, logloss): 1 4.47877e-06
  linear_hinge (acc, logloss): 1 0.208572
  tree (acc, logloss): 1 9.99978e-13
  ensemble (acc, logloss): 1 1.00003e-12
  neighbor (acc, logloss): 1 9.99978e-13
  kernel (acc, logloss): 1 0.318438
  bayes (acc, logloss): 1 9.99978e-13
Regression families:
  linear (mse, r2): 1.52381e-19 1
  tree (mse, r2): 0 1
...
```

SparseLinearRegressor::load_state

API Signature

- `SparseLinearRegressor::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = SparseLinearRegressor::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

SparseLinearRegressor::trained

API Signature

- `SparseLinearRegressor::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = SparseLinearRegressor::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

StandardScaler::dump_state

API Signature

- `StandardScaler::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = StandardScaler::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();
```

(continues on next page)

(continued from previous page)

```
  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

`StandardScaler::load_state`

API Signature

- `StandardScaler::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = StandardScaler::load_state(/* state */);
```

(continues on next page)

(continued from previous page)

```
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

`StandardScaler::trained`

API Signature

- `StandardScaler::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = StandardScaler::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

`Status::ok`

API Signature

- `Status::ok() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

Option semantics used by this method:

- `Status` (`struct`):
  - `StatusCode code = StatusCode::Ok` Option field used by this method.
  - `std::string message{}` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Status::ok();
out.info("ok", value);
```

Expected output

```
ok: true
```

`Status::success`

API Signature

- `Status::success() -> Status`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

Option semantics used by this method:

- `Status` (`struct`):
  - `StatusCode code = StatusCode::Ok` Option field used by this method.
  - `std::string message{}` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = Status::success();
if (!st.ok()) {
  out.error("success failed", st.message);
} else {
  out.info("success ok");
}
```

Expected output

```
success: ok
status.ok() == true
```

`StatusOr::move_value`

API Signature

- `StatusOr::move_value() -> T&&`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `T&&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = StatusOr::move_value();
out.info("move_value", value);
```

Expected output

```
move_value returned T&&
```

Example 2 (Demo Pattern)

```
  if (!require_ok(reg_sweep_or, "sweep(regression quantile)")) return 1;
  std::cout << "regression quantile sweep objectives:";
  for (const auto& trial : reg_sweep_or.value().trials) std::cout << " " << trial.
↪objective;
  std::cout << "\n";
  std::cout << "regression quantile sweep best index: " << reg_sweep_or.value().best_
↪index << "\n";

  auto bounded_or = make_classifier(linear_logistic_spec());
  if (!require_ok(bounded_or, "make_classifier(linear bounded)")) return 1;
  auto bounded = bounded_or.move_value();

  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }

  auto bounded_scores_or = bounded->infer_scores(batch.features);
  if (!require_ok(bounded_scores_or, "infer_scores(bounded)")) return 1;
  std::cout << "max |bounded score|: " << bounded_scores_or.value().abs().max().item
↪<double>() << "\n";
```

Expected output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪1.12262
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.169779
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.420312
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0248626
sweep trial objectives: 0 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0352983 0.743786
```

(continues on next page)

(continued from previous page)

```
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 3 (Demo Pattern)

```
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
  }

  auto metrics_or = reg->assess(batch, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metrics_or, "assess")) return 1;
  print_metrics("linear metrics [mse, r2]", metrics_or.value());

  std::cout << "NodeJ DataFrame bridge demo completed.\n";
  return 0;
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

StatusOr::ok

API Signature

- `StatusOr::ok() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = StatusOr::ok();
out.info("ok", value);
```

Expected output

```
ok: true
```

`StatusOr::status`

API Signature

- `StatusOr::status() -> const Status&`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `const Status&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = StatusOr::status();
out.info("status", value);
```

Expected output

```
status returned const Status&
```

StatusOr::StatusOr

API Signature

- `StatusOr(arg1: Status status) : status_(std::move(status)) -> auto`
- `StatusOr(arg1: T value) : status_(Status::success()), arg2: value_(std::move(value)), true: has_value_() -> auto`
- `status_(arg1: std::move(status)), arg2: value_(std::move(value)), arg3: has_value_(status_.ok()) -> StatusOr(Status status, T value) :`

Notes

Instance method on `self`.

Inputs

- `arg1` (`Status status) : status_(std::move(status) | T value) : status_(Status::success()) | std::move(status))`) Method argument; see signature type for accepted form.
- `arg2` (`value_(std::move(value))`) Method argument; see signature type for accepted form.
- `arg3` (`has_value_(status_.ok())`) Method argument; see signature type for accepted form.
- `true` (`has_value_()`) Method argument; see signature type for accepted form.

Output

- Returns `StatusOr(Status status, T value) :`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = StatusOr(/* arg1 */);
out.info("StatusOr", value);
```

Expected output

```
StatusOr returned result
```

StatusOr::value

API Signature

- `StatusOr::value() -> const T&`
- `StatusOr::value() -> T&`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `T&`.
- Returns `const T&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = StatusOr::value();
out.info("value", value);
```

Expected output

```
value returned const T&
```

Example 2 (Demo Pattern)

```
      MetricKind::Precision,
      MetricKind::Recall,
      MetricKind::F1,
      MetricKind::RocAuc,
      MetricKind::LogLoss};

  auto linear_cv_or = cross_assess(linear_logistic_spec(), batch, split, train, assess);
  if (!require_ok(linear_cv_or, "cross_assess(linear)")) return 1;
  print_series("linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc,␣
↪logloss]", linear_cv_or.value().mean_metrics);
  print_series("linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc,␣
↪logloss]", linear_cv_or.value().std_metrics);

  auto kernel_cv_or = cross_assess(rbf_kernel_spec(), batch, split, train, assess);
  if (!require_ok(kernel_cv_or, "cross_assess(kernel)")) return 1;
  print_series("kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc,␣
↪logloss]", kernel_cv_or.value().mean_metrics);
  print_series("kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc,␣
↪logloss]", kernel_cv_or.value().std_metrics);

  const std::vector<ModelSpec> candidates{linear_logistic_spec(), rbf_kernel_spec()};
```

(continues on next page)

(continued from previous page)

```
  AssessOptions tune_assess{};
  tune_assess.metrics = {MetricKind::Accuracy};

  auto sweep_or = sweep(candidates,
```

Expected output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪1.12262
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.169779
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.420312
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0248626
sweep trial objectives: 0 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0352983 0.743786
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 3 (Demo Pattern)

```
  options.label_feature = "target";
  options.sample_weight_feature = "weight";
  options.note_policy = DataNotePolicy::Hash;
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
```

(continues on next page)

(continued from previous page)

```
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

take_features

API Signature

- take_features(batch: const FeatureBatch&, feature_indices: std::initializer_list<std::int64_t>) -> StatusOr<FeatureBatch>
- take_features(batch: const FeatureBatch&, feature_indices: RangeView::Int) -> StatusOr<FeatureBatch>

Notes

Namespace-level API function.

Inputs

- batch (const FeatureBatch&) Input feature data used by this call.
- feature_indices (RangeView::Int | std::initializer_list<std::int64_t>) Input feature data used by this call.

Output

- Returns StatusOr<FeatureBatch>. Check .ok() first; on success read .value() for the computed result.

Options

Option semantics used by this method:

- FeatureBatch (struct):
  - Series x = Series::undefined() Option field used by this method.
  - ObservationLayout layout = ObservationLayout::Rows Declares whether observations are represented by rows or columns.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto res = take_features(/* batch */, /* feature_indices */);
if (!res.ok()) {
  out.error("take_features failed", res.status().message);
} else {
  out.info("take_features ok");
}
```

Expected output

```
take_features: ok
status.ok() == true
result.value() is available
```

ThresholdBinarizer::dump_state

API Signature

- `ThresholdBinarizer::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto res = ThresholdBinarizer::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```cpp
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
```

(continues on next page)

(continued from previous page)

```
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

ThresholdBinarizer::load_state

API Signature

- `ThresholdBinarizer::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = ThresholdBinarizer::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

`ThresholdBinarizer::trained`

API Signature

- `ThresholdBinarizer::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = ThresholdBinarizer::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

`TransformerBase::dump_state`

API Signature

- `TransformerBase::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = TransformerBase::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
```

(continues on next page)

(continued from previous page)

```
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

`TransformerBase::load_state`

API Signature

- `TransformerBase::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = TransformerBase::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
```

(continues on next page)

(continued from previous page)

```
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

TransformerBase::trained

API Signature

- `TransformerBase::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = TransformerBase::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

variance_prune

API Signature

- `variance_prune(batch:  const FeatureBatch&, variance_floor:  double = 1e-12) -> StatusOr<VariancePruneResult>`

Notes

Namespace-level API function.

Inputs

- `batch` (`const FeatureBatch&`) Input feature data used by this call.
- `variance_floor` (`double`), default: `1e-12` Method argument; see signature type for accepted form.

Output

- Returns `StatusOr<VariancePruneResult>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

Option semantics used by this method:

- `FeatureBatch` (`struct`):
  - `Series x = Series::undefined()` Option field used by this method.
  - `ObservationLayout layout = ObservationLayout::Rows` Declares whether observations are represented by rows or columns.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = variance_prune(/* batch */, /* variance_floor */);
if (!res.ok()) {
  out.error("variance_prune failed", res.status().message);
} else {
  out.info("variance_prune ok");
}
```

Expected output

```
variance_prune: ok
status.ok() == true
result.value() is available
```

`VectorNormalizer::dump_state`

API Signature

- `VectorNormalizer::dump_state() -> StatusOr<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StatusOr<std::vector<Series>>`. Check `.ok()` first; on success read `.value()` for the computed result.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto res = VectorNormalizer::dump_state();
if (!res.ok()) {
  out.error("dump_state failed", res.status().message);
} else {
  out.info("dump_state ok");
}
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
```

(continues on next page)

(continued from previous page)

```
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state: ok
status.ok() == true
result.value() is available
```

VectorNormalizer::load_state

API Signature

- `VectorNormalizer::load_state(state:  RangeView::Series) -> Status`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `state` (`RangeView::Series`) Randomization control for reproducible behavior.

Output

- Returns a status object indicating success/failure and an error message on failure.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto st = VectorNormalizer::load_state(/* state */);
if (!st.ok()) {
  out.error("load_state failed", st.message);
} else {
  out.info("load_state ok");
}
```

Expected output

```
load_state: ok
status.ok() == true
```

`VectorNormalizer::trained`

API Signature

- `VectorNormalizer::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = VectorNormalizer::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

CHAPTER

FIVE

LYMPH FULL API REFERENCE

Lymph method-centric reference generated from public headers. Each page follows **API Signature -> Inputs -> Output -> Options -> Examples (with expected outputs)**. Options/enums are documented under the methods that consume them.

- Method pages: **135**
- Data-type pages: **26**
- Overloads covered: **180**

5.1 Function and Method Reference

5.1.1 Lymph Full API Reference: Method Index

Total method pages: **135** Total data-type pages: **26**

Data Types

`BatchHoldoutSplit`

API Signature

- `BatchHoldoutSplit`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `BatchHoldoutSplit` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `LabeledBatch train_set{}`
- `LabeledBatch test_set{}`

- `IndexSplit indices{}`

Examples

```
BatchHoldoutSplit cfg{};
```

`bayes_factory`

API Signature

- `bayes_factory`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `bayes_factory` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
bayes_factory cfg{};
```

`Branch`

API Signature

- `Branch`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `Branch` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `std::vector<TransformSpec> transforms{}`
- `std::vector<std::string> feature_names{}`
- `std::vector<std::int64_t> feature_route{}`

Examples

```
Branch cfg{};
```

Candidate

API Signature

- `Candidate`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `Candidate` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `std::string name{}`
- `Workflow workflow{}`
- `Candidate& operator=(Candidate&&) noexcept = default`
- `Candidate& operator=(const Candidate&) = delete`

Examples

Example 1

```
  const auto t_cv1 = Clock::now();
  out.writef("cross_validate block seconds={}", secs(t_cv0, t_cv1)).endln();

  out.writeln("linear cv", linear_cv.mean.rmse, linear_cv.mean.r2);
  out.writeln("tree cv", tree_cv.mean.rmse, tree_cv.mean.r2);
  out.writeln("forest cv", forest_cv.mean.rmse, forest_cv.mean.r2);
  //========== End Cell 12 ========

  //========== Cell 13: Tune Candidate Families ========
  // Equivalent in spirit to Grid/Random search over model hyperparameters.
  std::vector<lymph::selection::Candidate> candidates{};
  {
    auto wf = lymph::Workflow{
        .model = lymph::linear_factory::linear_
↪regression(pluss::ai::nodej::LinearFamilyOptions{
            .objective = pluss::ai::nodej::LinearObjective::LeastSquares,
            .regularization = pluss::ai::nodej::RegularizationKind::L2,
```

(continues on next page)

(continued from previous page)

```
            .regularization_strength = 1e-6,
            .fit_intercept = true,
        }),
    };
    candidates.emplace_back("linear_l2_1e6", std::move(wf));
```

Expected output

```
Candidate configured
```

CategoryIndexerSummary

API Signature

- `CategoryIndexerSummary`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `CategoryIndexerSummary` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `bool learned = false`
- `std::int64_t feature_count = 0`
- `bool include_unknown_bucket = true`
- `bool one_based = true`
- `std::vector<std::string> feature_names{}`
- `std::vector<std::string> mapped_names{}`
- `std::vector<std::vector<std::string>> categories{}`

Examples

```
CategoryIndexerSummary cfg{};
// cfg.learned = ...;
```

cluster_factory

API Signature

- `cluster_factory`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `cluster_factory` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
cluster_factory cfg{};
```

CrossValidationReport

API Signature

- `CrossValidationReport`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `CrossValidationReport` object used by NodeJ/Lymph workflows.

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Fields

Public fields on this type:

- `CrossAssessReport report{}`
- `MetricReport mean{}`
- `MetricReport stdev{}`

Examples

```
CrossValidationReport cfg{};
```

decompose_factory

API Signature

- decompose_factory

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A decompose_factory object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
decompose_factory cfg{};
```

ensemble_factory

API Signature

- ensemble_factory

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A ensemble_factory object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
ensemble_factory cfg{};
```

FrameHoldoutSplit

API Signature

- FrameHoldoutSplit

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `FrameHoldoutSplit` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `pluss::table::DataFrame train_set{}`
- `pluss::table::DataFrame test_set{}`
- `IndexSplit indices{}`

Examples

```
FrameHoldoutSplit cfg{};
```

`kernel_factory`

API Signature

- `kernel_factory`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `kernel_factory` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
kernel_factory cfg{};
```

`linear_factory`

API Signature

- `linear_factory`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `linear_factory` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
linear_factory cfg{};
```

`LinearParameters`

API Signature

- `LinearParameters`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `LinearParameters` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `Series coeff = Series::undefined()`
- `Series intercept = Series::undefined()`
- `bool fit_intercept = false`

Examples

```
LinearParameters cfg{};
// cfg.coeff = ...;
```

`MetricReport`

API Signature

- `MetricReport`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `MetricReport` object used by NodeJ/Lymph workflows.

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Fields

Public fields on this type:

- `std::vector<MetricKind> requested{}`
- `Series values = Series::undefined()`
- `Series accuracy = Series::undefined()`
- `Series balanced_accuracy = Series::undefined()`
- `Series precision = Series::undefined()`
- `Series recall = Series::undefined()`
- `Series f1 = Series::undefined()`
- `Series roc_auc = Series::undefined()`
- `Series logloss = Series::undefined()`
- `Series mse = Series::undefined()`
- `Series rmse = Series::undefined()`
- `Series mae = Series::undefined()`
- `Series r2 = Series::undefined()`
- `Series quantile_loss = Series::undefined()`

Examples

```
MetricReport cfg{};
```

neighbor_factory

API Signature

- `neighbor_factory`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `neighbor_factory` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
neighbor_factory cfg{};
```

`online_factory`

API Signature

- `online_factory`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `online_factory` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
online_factory cfg{};
```

`OutlierIsolatorSummary`

API Signature

- `OutlierIsolatorSummary`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `OutlierIsolatorSummary` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `bool learned = false`
- `std::int64_t feature_count = 0`
- `std::int64_t estimators = 0`
- `std::int64_t max_samples = 0`
- `std::int64_t max_features = 0`
- `bool bootstrap = false`
- `bool contamination_auto = true`
- `double contamination = 0.1`
- `double offset = -0.5`
- `std::vector<std::string> feature_names{}`

Examples

```
OutlierIsolatorSummary cfg{};
// cfg.learned = ...;
```

SeriesCompareReport

API Signature

- `SeriesCompareReport`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `SeriesCompareReport` object used by NodeJ/Lymph workflows.

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Fields

Public fields on this type:

- `Series abs_error = Series::undefined()`
- `Series max_abs_diff = Series::undefined()`
- `Series mae = Series::undefined()`

Examples

```
SeriesCompareReport cfg{};
// cfg.abs_error = ...;
```

SimpleImputerSummary

API Signature

- SimpleImputerSummary

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A SimpleImputerSummary object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- SimpleImputerStrategy strategy = SimpleImputerStrategy::Mean
- bool learned = false
- std::int64_t feature_count = 0
- double constant_value = 0.0
- double empty_feature_fallback = 0.0
- std::vector<std::string> feature_names{}
- Series statistics = Series::undefined()

Examples

```
SimpleImputerSummary cfg{};
// cfg.strategy = ...;
```

SpecTuneResult

API Signature

- SpecTuneResult

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `SpecTuneResult` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `::pluss::ai::nodej::TuneReport report{}`
- `ModelSpec best_spec{}`

Examples

```
SpecTuneResult cfg{};
```

StandardScalerSummary

API Signature

- `StandardScalerSummary`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `StandardScalerSummary` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `std::vector<std::string> feature_names{}`
- `Series mean = Series::undefined()`
- `Series scale = Series::undefined()`

Examples

```
StandardScalerSummary cfg{};
```

TransformerSummary

API Signature

- TransformerSummary

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A TransformerSummary object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- std::vector<std::string> feature_names{}
- std::vector<Series> state{}

Examples

```
TransformerSummary cfg{};
```

tree_factory

API Signature

- tree_factory

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A tree_factory object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Examples

```
tree_factory cfg{};
```

`Trial`

API Signature

- `Trial`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `Trial` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `std::string name{}`
- `double objective = std::numeric_limits<double>::quiet_NaN()`
- `CrossValidationReport cv{}`

Examples

```
Trial cfg{};
```

`TuneReport`

API Signature

- `TuneReport`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `TuneReport` object used by NodeJ/Lymph workflows.

Notes

Structured report object carrying metrics/diagnostics from training/evaluation routines.

Fields

Public fields on this type:

- `std::vector<Trial> trials{}`
- `std::int64_t best_index = -1`
- `std::string best_name{}`
- `MetricKind objective_metric = MetricKind::Accuracy`
- `TuneObjectivePolicy objective_policy = TuneObjectivePolicy::Auto`

Examples

Example 1

```
    train_options,
    assess_options,
    MetricKind::Rmse,
    TuneObjectivePolicy::Minimize);
if (!broad_sweep_or.ok()) {
  std::cerr << "sweep(broad) failed: " << broad_sweep_or.status().message << "\n";
  return 1;
}
TuneReport broad_sweep = broad_sweep_or.move_value();

std::vector<std::size_t> broad_success_indices{};
for (std::size_t i = 0; i < broad_sweep.trials.size(); ++i) {
  if (!broad_sweep.trials[i].status.ok()) continue;
  if (!std::isfinite(broad_sweep.trials[i].objective)) continue;
  broad_success_indices.push_back(i);
}
if (broad_success_indices.empty()) {
  std::cerr << "broad sweep produced no successful trials.\n";
  return 1;
}
```

Expected output

```
TuneReport configured
```

TuneResult

API Signature

- `TuneResult`

Inputs

- This is a public data type. Populate its fields before passing into APIs.

Output

- A `TuneResult` object used by NodeJ/Lymph workflows.

Notes

Public user-facing type used directly in NodeJ/Lymph workflows.

Fields

Public fields on this type:

- `TuneReport report{}`
- `Workflow best_workflow{}`

Examples

```
TuneResult cfg{};
```

Methods

`as_stratify_series`

API Signature

- `as_stratify_series(thread:  const pluss::table::Thread&) -> Series`
- `as_stratify_series(thread:  const pluss::table::Thread&) -> Series`

Notes

Namespace-level API function.

Inputs

- `thread` (`const pluss::table::Thread&`) Pluss numeric vector/tensor input.

Output

- Returns `Series`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = as_stratify_series(/* thread */);
out.info("as_stratify_series", value);
```

Expected output

```
as_stratify_series returned Series
```

best_trial

API Signature

- `best_trial(report:  const TuneReport&) -> const TuneTrial&`

Notes

Namespace-level API function.

Inputs

- `report` (`const TuneReport&`) Method argument; see signature type for accepted form.

Output

- Returns `const TuneTrial&`.

Options

Option semantics used by this method:

- `TuneReport` (`struct`):
  - `std::vector<Trial> trials{}` Option field used by this method.
  - `std::int64_t best_index = -1` Option field used by this method.
  - `std::string best_name{}` Option field used by this method.
  - `MetricKind objective_metric = MetricKind::Accuracy` Selects optimization or evaluation objective.
  - `TuneObjectivePolicy objective_policy = TuneObjectivePolicy::Auto` Selects optimization or evaluation objective.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = best_trial(/* report */);
out.info("best_trial", value);
```

Expected output

```
best_trial returned const TuneTrial&
```

BinaryLinearModel::dump_state

API Signature

- `BinaryLinearModel::dump_state() -> std::vector<Series>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::vector<Series>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = BinaryLinearModel::dump_state();
out.info("dump_state", value);
```

Expected output

```
dump_state returned std::vector<Series>
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();
```

(continues on next page)

(continued from previous page)

```
  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state returned std::vector<Series>
```

BinaryLinearModel::feature_count

API Signature

- `BinaryLinearModel::feature_count() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = BinaryLinearModel::feature_count();
out.info("feature_count", value);
```

Expected output

```
feature_count: <numeric_result>
```

Example 2 (Demo Pattern)

```
  options.note_policy = DataNotePolicy::Hash;
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

```

(continues on next page)

(continued from previous page)

```
  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

Example 3 (Demo Pattern)

```
}

Config i64() {
  return Config{}.with_dtype(ScalarType::Int64).with_platform(pluss::pCPU);
}

LabeledBatch slice_rows(const LabeledBatch& batch, std::int64_t start, std::int64_t␣
↪count) {
  const Series idx = pluss::arange(start, start + count, i64());
  const Series x = batch.features.x.index_select(0, idx).reshape({count, batch.features.
↪feature_count()});
  const Series y = batch.y.index_select(0, idx).reshape({count});
  return LabeledBatch{
      .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
      .y = y,
  };
}

LabeledBatch make_class_batch() {
  std::vector<double> xv{};
  std::vector<double> yv{};
  xv.reserve(2 * 90);
  yv.reserve(90);
```

Expected output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

BinaryLinearModel::infer_scores

API Signature

- `BinaryLinearModel::infer_scores(x: const Csr&) -> Series`
- `BinaryLinearModel::infer_scores(x: const Triplet&) -> Series`

Notes

Computes predictions from prepared inputs using the currently learned state.

Inputs

- `x` (`const Csr& | const Triplet&`) Input feature data used by this call.

Output

- Returns `Series`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = BinaryLinearModel::infer_scores(/* x */);
out.info("infer_scores", value);
```

Expected output

```
infer_scores returned Series
```

Example 2 (Demo Pattern)

```
  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }
```

(continues on next page)

(continued from previous page)

```
  auto bounded_scores_or = bounded->infer_scores(batch.features);
  if (!require_ok(bounded_scores_or, "infer_scores(bounded)")) return 1;
  std::cout << "max |bounded score|: " << bounded_scores_or.value().abs().max().item
↪<double>() << "\n";

  std::cout << "NodeJ CV + tune demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪1.12262
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.169779
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.420312
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0248626
sweep trial objectives: 0 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0352983 0.743786
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 3 (Demo Pattern)

```
  }
  auto cls = cls_or.move_value();
  const auto learn_status = cls->learn(batch);
  if (!learn_status.ok()) {
    std::cerr << "learn failed: " << learn_status.message << '\n';
    return 1;
  }

  auto score_or = cls->infer_scores(batch.features);
  if (!score_or.ok()) {
    std::cerr << "infer_scores failed: " << score_or.status().message << '\n';
    return 1;
  }
  std::cout << "scores shape: [" << score_or.value().sizes()[0] << ", " << score_or.
↪value().sizes()[1] << "]\n";

  pluss::ai::nodej::AssessOptions assess{};
  assess.metrics = {MetricKind::Accuracy, MetricKind::F1, MetricKind::LogLoss};
  auto metric_or = cls->assess(batch, assess);
  if (!metric_or.ok()) {
    std::cerr << "assess failed: " << metric_or.status().message << '\n';
    return 1;
```

Expected output

```
scores shape: [12, 3]
metrics [accuracy, f1, logloss]: 1 1 1.24859e-10
flow metrics [accuracy, f1, logloss]: 1 1 1.48016e-09
```

BinaryLinearModel::trained

API Signature

- `BinaryLinearModel::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = BinaryLinearModel::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

branch_transformer

API Signature

- `branch_transformer(frame: const pluss::table::DataFrame&, data_options: const DataFrameBatchOptions&, branches: RangeView::Of<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer`
- `branch_transformer(frame: const pluss::table::DataFrame&, data_options: const DataFrameBatchOptions&, branches: std::initializer_list<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer`

- branch_transformer(frame: const pluss::table::DataFrame&, branches: RangeView::Of<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer
- branch_transformer(frame: const pluss::table::DataFrame&, branches: std::initializer_list<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer
- branch_transformer(feature_order: RangeView::Of<std::string>, branches: RangeView::Of<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer
- branch_transformer(feature_order: std::initializer_list<std::string>, branches: std::initializer_list<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer

Notes

Namespace-level API function.

Inputs

- branches (RangeView::Of<workflow::feature::Branch> | std::initializer_list<workflow::feature::Branch) List-like argument; pass values in the order expected by the signature.
- data_options (const DataFrameBatchOptions&) Input feature data used by this call.
- feature_order (RangeView::Of<std::string> | std::initializer_list<std::string>) Input feature data used by this call.
- frame (const pluss::table::DataFrame&) Input feature data used by this call.
- remainder (prep::FeatureRemainderPolicy), default: prep::FeatureRemainderPolicy::PassThrough Method argument; see signature type for accepted form.

Output

- Returns ::pluss::ai::nodej::prep::FeatureTransformer.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = branch_transformer(/* frame */, /* data_options */, /* branches */, /*␣
↪remainder */);
out.info("branch_transformer", value);
```

Expected output

```
branch_transformer returned ::pluss::ai::nodej::prep::FeatureTransformer
```

Candidate::Candidate

API Signature

- `name(arg1:  std::move(in_name)), arg2:  workflow(std::move(in_workflow)) -> Candidate(std::string in_name, Workflow in_workflow) :`

Notes

Instance method on `self`.

Inputs

- `arg1` (`std::move(in_name)`)) Method argument; see signature type for accepted form.
- `arg2` (`workflow(std::move(in_workflow))`) Method argument; see signature type for accepted form.

Output

- Returns `Candidate(std::string in_name, Workflow in_workflow) :`.

Options

Option semantics used by this method:

- `Candidate` (`struct`):
  - `std::string name{}` Option field used by this method.
  - `Workflow workflow{}` Option field used by this method.
  - `Candidate& operator=(Candidate&&) noexcept = default` Option field used by this method.
  - `Candidate& operator=(const Candidate&) = delete` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = name(/* arg1 */, /* arg2 */);
out.info("Candidate", value);
```

Expected output

```
Candidate returned Candidate(std::string in_name, Workflow in_workflow) :
```

Example 2 (Demo Pattern)

```
    nja::CrossValidationOptions cv_options{};
    cv_options.plan = CVPlanKind::KFold;
    cv_options.folds = 5;
    cv_options.shuffle = true;
```

(continues on next page)

(continued from previous page)

```
    cv_options.random_state = 73;
    cv_options.deterministic = true;

    std::vector<nja::selection::Candidate> candidates{};
    {
      auto wf = nja::Workflow{
          .model = nja::linear_factory::linear_regression(LinearFamilyOptions{          .
↪objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-6,
          .fit_intercept = true,
      }),
          .stages = {
              nja::workflow::stage::simple_imputer(nja::prep::SimpleImputerOptions{.
↪strategy = nja::prep::SimpleImputerStrategy::Median}),
              nja::workflow::stage::standard_scaler(),
          },
      };
```

Expected output

```
CV objective metric: rmse
candidate=linear_ridge objective=102.872 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[102.87158] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.41650024] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[77.611594]
candidate=decision_tree objective=44.9727 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[44.972693] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.88843998] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[27.61723]
candidate=random_forest objective=81.1235 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[81.123538] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.63710967] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
...
```

CategoryIndexer::features

API Signature

- `CategoryIndexer::features(names: RangeView::Of<std::string_view>) -> CategoryIndexer&`
- `CategoryIndexer::features(names: std::initializer_list<std::string_view>) -> CategoryIndexer&`

Notes

Instance method on `self`.

Inputs

- `names` (`RangeView::Of<std::string_view> | std::initializer_list<std::string_view>`) List-like argument; pass values in the order expected by the signature.

Output

- Returns `CategoryIndexer&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = CategoryIndexer::features(/* names */);
out.info("features", value);
```

Expected output

```
features returned CategoryIndexer&
```

Example 2 (Demo Pattern)

```
    const auto split = nja::holdout_split(frame, nja::HoldoutSplitOptions{
                                                     .test_size = 0.2,
                                                     .random_state = 42,
                                                     .deterministic = true,
                                                     .shuffle = true,
                                                 });
    const DataFrame train_x = split.train_set.features(
        {"season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
↪"weather_condition", "temp", "atemp", "humidity", "windspeed"});
    const DataFrame test_x = split.test_set.features(
        {"season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
↪"weather_condition", "temp", "atemp", "humidity", "windspeed"});
    const Series train_y = split.train_set.features("total_count").as_series();
    const Series test_y = split.test_set.features("total_count").as_series();

    TrainOptions train_options{};
    train_options.max_iterations = 2500;
    train_options.tolerance = 1e-8;
    train_options.random.seed = 7;
    train_options.random.deterministic = true;
```

Expected output

```
CV objective metric: rmse
candidate=linear_ridge objective=102.872 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[102.87158] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.41650024] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[77.611594]
candidate=decision_tree objective=44.9727 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[44.972693] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.88843998] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[27.61723]
candidate=random_forest objective=81.1235 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[81.123538] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.63710967] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
...
```

Example 3 (Demo Pattern)

```
  std::cout << "numeric features: ";
  for (const auto& n : numeric) std::cout << n << " ";
  std::cout << "\n";
  std::cout << "categorical features: ";
  for (const auto& n : categorical) std::cout << n << " ";
  std::cout << "\n";
  auto ord = lymph::prep::OrdinalEncoder({.include_unknown_bucket = true, .one_based =␣
↪true});
  ord.features({"ocean_proximity"});
  const auto ocean_ord = ord.learn_map(housing.features({"ocean_proximity"}));
  auto lbl = lymph::prep::LabelEncoder({.include_unknown_bucket = true, .one_based =␣
↪false});
  const auto ocean_lbl = lbl.learn_map(housing.features("ocean_proximity"));
  std::cout << "ordinal mapped names: ";
  for (const auto& name : ord.mapped_names_) std::cout << name << " ";
  std::cout << "\n";
  std::cout << "label mapped names: ";
  for (const auto& name : lbl.mapped_names_) std::cout << name << " ";
  std::cout << "\n";
  std::cout << "ordinal sample: " << ocean_ord << "\n";
  std::cout << "label sample: " << ocean_lbl << "\n";
  std::cout << "pred: " << pred << "\n";
```

Expected output

```
numeric features: median_income target
categorical features: ocean_proximity
```

(continues on next page)

(continued from previous page)

```
ordinal mapped names: ocean_proximity_ordinal
label mapped names: ocean_proximity_label
ordinal sample: ======================
     | ocean_proximity
----------------------
0    | 2
1    | 1
2    | 2
3    | 1
4    | 2
...
```

CategoryIndexer::learned

API Signature

- `CategoryIndexer::learned() -> bool`

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto value = CategoryIndexer::learned();
out.info("learned", value);
```

Expected output

```
learned: true
```

CategoryIndexer::summary

API Signature

- `CategoryIndexer::summary() -> CategoryIndexerSummary`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `CategoryIndexerSummary`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = CategoryIndexer::summary();
out.info("summary", value);
```

Expected output

```
summary returned CategoryIndexerSummary
```

Example 2 (Demo Pattern)

```
    //========== standalone lymph preprocessing API ========
    // This section demonstrates direct prep estimators (learn/map/learn_map)
    // without going through Workflow composition.
    auto simple_imp = lymph::prep::simple_imputer(lymph::prep::SimpleImputerOptions{
        .strategy = lymph::prep::SimpleImputerStrategy::Median,
    });
    const DataFrame train_simple_imputed = simple_imp.learn_map(train_x);
    const DataFrame test_simple_imputed = simple_imp.map(test_x);
    const auto simple_imp_summary = simple_imp.summary();

    auto knn_imp = lymph::prep::neighbor_imputer(lymph::prep::NeighborImputerOptions{
        .neighbors = 7,
        .weighting = lymph::prep::NeighborImputerWeighting::Distance,
        .distance_floor = 1e-12,
        .empty_feature_fallback = 0.0,
    });
    const DataFrame train_knn_imputed = knn_imp.learn_map(train_x);
    const DataFrame test_knn_imputed = knn_imp.map(test_x);
    const auto knn_imp_summary = knn_imp.summary();

    auto iter_imp = lymph::prep::iterative_imputer(lymph::prep::IterativeImputerOptions{
```

Expected output

```
summary returned CategoryIndexerSummary
```

classification_summary

API Signature

- classification_summary(truth_labels: const Series&, predicted_labels: const Series&, label_order: RangeView::Of<double>, zero_division: double = 0.0) -> ClassReport
- classification_summary(truth_labels: const Series&, predicted_labels: const Series&, label_order: std::initializer_list<double>, zero_division: double = 0.0) -> ClassReport

Notes

Namespace-level API function.

Inputs

- label_order (RangeView::Of<double> | std::initializer_list<double>) Target values aligned to input observations.
- predicted_labels (const Series&) Target values aligned to input observations.
- truth_labels (const Series&) Target values aligned to input observations.
- zero_division (double), default: 0.0 Method argument; see signature type for accepted form.

Output

- Returns ClassReport.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = classification_summary(/* truth_labels */, /* predicted_labels */, /* label_
↪order */, /* zero_division */);
out.info("classification_summary", value);
```

Expected output

```
metrics: <metric_1> <metric_2> ...
evaluation completed
```

compare_series

API Signature

- compare_series(lhs: const Series&, rhs: const Series&) -> SeriesCompareReport

Notes

Namespace-level API function.

Inputs

- lhs (const Series&) Pluss numeric vector/tensor input.
- rhs (const Series&) Pluss numeric vector/tensor input.

Output

- Returns SeriesCompareReport.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = compare_series(/* lhs */, /* rhs */);
out.info("compare_series", value);
```

Expected output

```
metrics: <metric_1> <metric_2> ...
evaluation completed
```

Example 2 (Demo Pattern)

```
  out.writeln("approx 95% rmse interval", std::vector<double>{rmse_lo, rmse_hi}, "rmse_
↪point", rmse_mean);
  //========== End Cell 14 ========

  //========== Cell 15: Save, Load, Re-Infer ========
  const fs::path final_model_path = model_dir / "lymph_housing_ch2_best.bin";
  final_workflow.save(final_model_path.string(), train_prepared, train_y, train_options);
  auto restored = lymph::Workflow::load(final_model_path.string());
  const Series restored_pred = restored.infer(test_prepared);
  const auto parity = lymph::compare_series(final_pred, restored_pred);
  out.writeln("restored parity", parity.max_abs_diff, parity.mae);

  DataFrame prediction_frame({
      {"y_true", test_y},
```

(continues on next page)

(continued from previous page)

```
    {"y_pred", Thread(final_pred)},
    {"y_pred_restored", Thread(restored_pred)},
});
prediction_frame.save_csv((output_dir / "housing_ch2_test_predictions.csv").string());
out.writeln("saved predictions", (output_dir / "housing_ch2_test_predictions.csv").
↪string());
out.writeln("saved model", final_model_path.string());
//========== End Cell 15 ========
```

Expected output

```
metrics: <metric_1> <metric_2> ...
evaluation completed
```

confusion_matrix

API Signature

- `confusion_matrix(truth_labels: const Series&, predicted_labels: const Series&, label_order: RangeView::Of<double>, normalize: ConfusionNormalize = ConfusionNormalize::None) -> ConfusionTable`
- `confusion_matrix(truth_labels: const Series&, predicted_labels: const Series&, label_order: std::initializer_list<double>, normalize: ConfusionNormalize = ConfusionNormalize::None) -> ConfusionTable`

Notes

Namespace-level API function.

Inputs

- `label_order` (`RangeView::Of<double> | std::initializer_list<double>`) Target values aligned to input observations.
- `normalize` (`ConfusionNormalize`), default: `ConfusionNormalize::None` Method argument; see signature type for accepted form.
- `predicted_labels` (`const Series&`) Target values aligned to input observations.
- `truth_labels` (`const Series&`) Target values aligned to input observations.

Output

- Returns `ConfusionTable`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = confusion_matrix(/* truth_labels */, /* predicted_labels */, /* label_order
↪*/, /* normalize */);
out.info("confusion_matrix", value);
```

Expected output

```
confusion_matrix returned ConfusionTable
```

copy_feature_route

API Signature

- `copy_feature_route(feature_route:  RangeView::Int) -> std::vector<std::int64_t>`

Notes

Namespace-level API function.

Inputs

- `feature_route` (`RangeView::Int`) Input feature data used by this call.

Output

- Returns `std::vector<std::int64_t>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = copy_feature_route(/* feature_route */);
out.info("copy_feature_route", value);
```

Expected output

```
copy_feature_route returned std::vector<std::int64_t>
```

copy_range_values

API Signature

- `copy_range_values(values:  RangeView::Of<T>) -> std::vector<T>`

Notes

Namespace-level API function.

Inputs

- `values` (`RangeView::Of<T>`) Method argument; see signature type for accepted form.

Output

- Returns `std::vector<T>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = copy_range_values(/* values */);
out.info("copy_range_values", value);
```

Expected output

```
copy_range_values returned std::vector<T>
```

dependence_curve

API Signature

- `dependence_curve(spec: const ModelSpec&, train: const LabeledBatch&, eval: const FeatureBatch&, feature_index: std::int64_t, grid_points: std::int64_t = 32) -> DependenceCurve`

Notes

Namespace-level API function.

Inputs

- `eval` (`const FeatureBatch&`) Method argument; see signature type for accepted form.
- `feature_index` (`std::int64_t`) Input feature data used by this call.
- `grid_points` (`std::int64_t`), default: `32` Method argument; see signature type for accepted form.
- `spec` (`const ModelSpec&`) Method argument; see signature type for accepted form.
- `train` (`const LabeledBatch&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `DependenceCurve`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = dependence_curve(/* spec */, /* train */, /* eval */, /* feature_index */,␣
↪/* grid_points */);
out.info("dependence_curve", value);
```

Expected output

```
dependence_curve returned DependenceCurve
```

dependence_surface

API Signature

- `dependence_surface(spec: const ModelSpec&, train: const LabeledBatch&, eval: const FeatureBatch&, feature_a: std::int64_t, feature_b: std::int64_t, grid_points_a: std::int64_t = 16, grid_points_b: std::int64_t = 16) -> DependenceSurface`

Notes

Namespace-level API function.

Inputs

- `eval` (`const FeatureBatch&`) Method argument; see signature type for accepted form.
- `feature_a` (`std::int64_t`) Input feature data used by this call.
- `feature_b` (`std::int64_t`) Input feature data used by this call.
- `grid_points_a` (`std::int64_t`), default: `16` Method argument; see signature type for accepted form.
- `grid_points_b` (`std::int64_t`), default: `16` Method argument; see signature type for accepted form.
- `spec` (`const ModelSpec&`) Method argument; see signature type for accepted form.
- `train` (`const LabeledBatch&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `DependenceSurface`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = dependence_surface(/* spec */, /* train */, /* eval */, /* feature_a */, /
↪* feature_b */, /* grid_points_a */, /* grid_points_b */);
out.info("dependence_surface", value);
```

Expected output

```
dependence_surface returned DependenceSurface
```

encoded_param_table

API Signature

- `encoded_param_table(report:  const TuneReport&) -> Series`

Notes

Namespace-level API function.

Inputs

- `report` (`const TuneReport&`) Method argument; see signature type for accepted form.

Output

- Returns `Series`.

Options

Option semantics used by this method:

- `TuneReport` (`struct`):
  - `std::vector<Trial> trials{}` Option field used by this method.
  - `std::int64_t best_index = -1` Option field used by this method.
  - `std::string best_name{}` Option field used by this method.
  - `MetricKind objective_metric = MetricKind::Accuracy` Selects optimization or evaluation objective.
  - `TuneObjectivePolicy objective_policy = TuneObjectivePolicy::Auto` Selects optimization or evaluation objective.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = encoded_param_table(/* report */);
out.info("encoded_param_table", value);
```

Expected output

```
encoded_param_table returned Series
```

has_best_trial

API Signature

- has_best_trial(report: const TuneReport&) -> bool

Notes

Boolean introspection helper that reports whether a capability/state is available.

Inputs

- report (const TuneReport&) Method argument; see signature type for accepted form.

Output

- Returns bool.

Options

Option semantics used by this method:

- TuneReport (struct):
  - std::vector<Trial> trials{} Option field used by this method.
  - std::int64_t best_index = -1 Option field used by this method.
  - std::string best_name{} Option field used by this method.
  - MetricKind objective_metric = MetricKind::Accuracy Selects optimization or evaluation objective.
  - TuneObjectivePolicy objective_policy = TuneObjectivePolicy::Auto Selects optimization or evaluation objective.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = has_best_trial(/* report */);
out.info("has_best_trial", value);
```

Expected output

```
has_best_trial: true
```

has_ledger

API Signature

- has_ledger(report: const TuneReport&) -> bool

Notes

Boolean introspection helper that reports whether a capability/state is available.

Inputs

- report (const TuneReport&) Method argument; see signature type for accepted form.

Output

- Returns bool.

Options

Option semantics used by this method:

- TuneReport (struct):
  - std::vector<Trial> trials{} Option field used by this method.
  - std::int64_t best_index = -1 Option field used by this method.
  - std::string best_name{} Option field used by this method.
  - MetricKind objective_metric = MetricKind::Accuracy Selects optimization or evaluation objective.
  - TuneObjectivePolicy objective_policy = TuneObjectivePolicy::Auto Selects optimization or evaluation objective.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = has_ledger(/* report */);
out.info("has_ledger", value);
```

Expected output

```
has_ledger: true
```

LinearRegressorModel::dump_state

API Signature

- LinearRegressorModel::dump_state() -> std::vector<Series>

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::vector<Series>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = LinearRegressorModel::dump_state();
out.info("dump_state", value);
```

Expected output

```
dump_state returned std::vector<Series>
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
```

(continues on next page)

(continued from previous page)

```
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state returned std::vector<Series>
```

`LinearRegressorModel::feature_count`

API Signature

- `LinearRegressorModel::feature_count() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = LinearRegressorModel::feature_count();
out.info("feature_count", value);
```

Expected output

```
feature_count: <numeric_result>
```

Example 2 (Demo Pattern)

```
  options.note_policy = DataNotePolicy::Hash;
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
```

(continues on next page)

(continued from previous page)

```
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

Example 3 (Demo Pattern)

```
}

Config i64() {
  return Config{}.with_dtype(ScalarType::Int64).with_platform(pluss::pCPU);
}

LabeledBatch slice_rows(const LabeledBatch& batch, std::int64_t start, std::int64_t␣
↪count) {
  const Series idx = pluss::arange(start, start + count, i64());
  const Series x = batch.features.x.index_select(0, idx).reshape({count, batch.features.
↪feature_count()});
  const Series y = batch.y.index_select(0, idx).reshape({count});
  return LabeledBatch{
      .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
      .y = y,
  };
}

LabeledBatch make_class_batch() {
  std::vector<double> xv{};
  std::vector<double> yv{};
  xv.reserve(2 * 90);
  yv.reserve(90);
```

Expected output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

`LinearRegressorModel::infer`

API Signature

- `LinearRegressorModel::infer(x: const Csr&) -> Series`
- `LinearRegressorModel::infer(x: const Triplet&) -> Series`

Notes

Computes predictions from prepared inputs using the currently learned state.

Inputs

- `x` (`const Csr& | const Triplet&`) Input feature data used by this call.

Output

- Returns `Series`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = LinearRegressorModel::infer(/* x */);
out.info("infer", value);
```

Expected output

```
infer returned Series
```

Example 2 (Demo Pattern)

```
  train.tolerance = 1e-8;
  train.random.seed = 77;
  train.random.deterministic = true;
  if (!flow.learn(batch, train).ok()) {
    std::cerr << "flow learn failed\n";
    return 1;
  }

  auto pred_before_or = flow.infer(batch.features);
  if (!pred_before_or.ok()) {
    std::cerr << "flow infer before save failed: " << pred_before_or.status().message <<
↪"\n";
    return 1;
  }
  const std::string model_dir = "<repo_root>/PlussAI/examples/NodeJ/trained_models";
```

(continues on next page)

(continued from previous page)

```
  std::error_code mkdir_ec;
  std::filesystem::create_directories(model_dir, mkdir_ec);
  if (mkdir_ec) {
    std::cerr << "failed to create model directory: " << model_dir << " (" << mkdir_ec.
↪message() << ")\n";
    return 1;
  }
  const std::string bundle_path = model_dir + "/nodej_flow_bundle_demo.bin";
```

Expected output

```
libc++abi: terminating due to uncaught exception of type pluss::Error: DataReader::with_
↪sizes: explicit sizes are only supported for host-buffer inputs
Triggered at: <path> (DataReader &pluss::detail::DataReader::with_sizes(std::span<const␣
↪std::int64_t>))
Native stack trace:
  0  pluss::detail::DataReader::with_sizes(std::__1::span<long long const,␣
↪18446744073709551615ul>)(data_reader.h:250)
  1  pluss::codem::Series pluss::detail::DataReader::to_series<pluss::codem::Config␣
↪const&>(pluss::detail::DataReader::Option, pluss::codem::Config const&) const(data_
↪reader.h:295)
  2  pluss::codem::Series pluss::codem::value<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_value.h:13)
  3  pluss::codem::Series pluss::make_series<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_factory.h:960)
  4  main(nodej_flow_bundle_demo.cpp:13)
  5  start + 6992 [dyld]


[exit_code] -6
```

Example 3 (Demo Pattern)

```
}

void run_classifier_family(const std::string& name, const ModelSpec& spec, const␣
↪LabeledBatch& train) {
  auto model_or = make_classifier(spec);
  if (!require_ok(model_or, name + " make_classifier")) return;
  std::unique_ptr<ClassifierBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;

  auto pred_or = model->infer(train.features);
  if (!require_ok(pred_or, name + " infer")) return;
  auto metric_or = model->assess(train, AssessOptions{.metrics = {MetricKind::Accuracy,␣
↪MetricKind::LogLoss}});
  if (!require_ok(metric_or, name + " assess")) return;
  print_metric("  " + name + " (acc, logloss)", metric_or.value());
}
```

(continues on next page)

(continued from previous page)

```
void run_regressor_family(const std::string& name, const ModelSpec& spec, const␣
↪LabeledBatch& train) {
  auto model_or = make_regressor(spec);
  if (!require_ok(model_or, name + " make_regressor")) return;
  std::unique_ptr<RegressorBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;
```

Expected output

```
NodeJ model families demo
Classification families:
  linear (acc, logloss): 1 4.47877e-06
  linear_hinge (acc, logloss): 1 0.208572
  tree (acc, logloss): 1 9.99978e-13
  ensemble (acc, logloss): 1 1.00003e-12
  neighbor (acc, logloss): 1 9.99978e-13
  kernel (acc, logloss): 1 0.318438
  bayes (acc, logloss): 1 9.99978e-13
Regression families:
  linear (mse, r2): 1.52381e-19 1
  tree (mse, r2): 0 1
...
```

LinearRegressorModel::trained

API Signature

- `LinearRegressorModel::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = LinearRegressorModel::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

make_branch_transformer

API Signature

- make_branch_transformer(frame: const pluss::table::DataFrame&, data_options: const DataFrameBatchOptions&, branches: RangeView::Of<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer
- make_branch_transformer(frame: const pluss::table::DataFrame&, data_options: const DataFrameBatchOptions&, branches: std::initializer_list<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer
- make_branch_transformer(frame: const pluss::table::DataFrame&, branches: RangeView::Of<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer
- make_branch_transformer(frame: const pluss::table::DataFrame&, branches: std::initializer_list<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer
- make_branch_transformer(feature_order: RangeView::Of<std::string>, branches: RangeView::Of<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer
- make_branch_transformer(feature_order: std::initializer_list<std::string>, branches: std::initializer_list<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer

Notes

Namespace-level API function.

Inputs

- branches (RangeView::Of<workflow::feature::Branch> | std::initializer_list<workflow::feature::Branch List-like argument; pass values in the order expected by the signature.
- data_options (const DataFrameBatchOptions&) Input feature data used by this call.
- feature_order (RangeView::Of<std::string> | std::initializer_list<std::string>) Input feature data used by this call.
- frame (const pluss::table::DataFrame&) Input feature data used by this call.

- `remainder` (`prep::FeatureRemainderPolicy`), default: `prep::FeatureRemainderPolicy::PassThrough` Method argument; see signature type for accepted form.

Output

- Returns `::pluss::ai::nodej::prep::FeatureTransformer`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = make_branch_transformer(/* frame */, /* data_options */, /* branches */, /*
↪remainder */);
out.info("make_branch_transformer", value);
```

Expected output

```
make_branch_transformer returned ::pluss::ai::nodej::prep::FeatureTransformer
```

`make_cross_validation_split_options`

API Signature

- `make_cross_validation_split_options(options: const CrossValidationOptions&) -> SplitOptions`

Notes

Builds train/test index partitions according to split-plan options.

Inputs

- `options` (`const CrossValidationOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `SplitOptions`.

Options

Option semantics used by this method:

- `CrossValidationOptions` (`struct`):
  - `CVPlanKind plan = CVPlanKind::KFold` Controls fold/trial/count sizing.
  - `double holdout_fraction = 0.2` Option field used by this method.
  - `std::int64_t folds = 5` Controls fold/trial/count sizing.

- `bool stratified = false` Option field used by this method.
- `Series groups = Series::undefined()` Option field used by this method.
- `bool reliability_defaults = true` Option field used by this method.
- `bool shuffle = true` Option field used by this method.
- `std::uint64_t random_state = 0` Random seed for reproducible behavior.
- `bool deterministic = true` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = make_cross_validation_split_options(/* options */);
out.info("make_cross_validation_split_options", value);
```

Expected output

```
split completed
train/test partitions ready
```

make_feature_transformer

API Signature

- `make_feature_transformer(frame: const pluss::table::DataFrame&, data_options: const DataFrameBatchOptions&, branches: const std::vector<workflow::feature::Branch>&, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer`
- `make_feature_transformer(frame: const pluss::table::DataFrame&, data_options: const DataFrameBatchOptions&, branches: RangeView::Of<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer`
- `make_feature_transformer(frame: const pluss::table::DataFrame&, data_options: const DataFrameBatchOptions&, branches: std::initializer_list<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer`
- `make_feature_transformer(frame: const pluss::table::DataFrame&, branches: const std::vector<workflow::feature::Branch>&, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer`
- `make_feature_transformer(frame: const pluss::table::DataFrame&, branches: RangeView::Of<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer`
- `make_feature_transformer(frame: const pluss::table::DataFrame&, branches: std::initializer_list<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer`

- `make_feature_transformer(feature_order: const std::vector<std::string>&, branches: const std::vector<workflow::feature::Branch>&, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer`
- `make_feature_transformer(feature_order: RangeView::Of<std::string>, branches: RangeView::Of<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer`
- `make_feature_transformer(feature_order: std::initializer_list<std::string>, branches: std::initializer_list<workflow::feature::Branch>, remainder: prep::FeatureRemainderPolicy = prep::FeatureRemainderPolicy::PassThrough) -> ::pluss::ai::nodej::prep::FeatureTransformer`

Notes

Namespace-level API function.

Inputs

- `branches` (`RangeView::Of<workflow::feature::Branch> | const std::vector<workflow::feature::Branch>& | std::initializer_list<workflow::feature::Branch>`) List-like argument; pass values in the order expected by the signature.
- `data_options` (`const DataFrameBatchOptions&`) Input feature data used by this call.
- `feature_order` (`RangeView::Of<std::string> | const std::vector<std::string>& | std::initializer_list<std::string>`) Input feature data used by this call.
- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `remainder` (`prep::FeatureRemainderPolicy`), default: `prep::FeatureRemainderPolicy::PassThrough` Method argument; see signature type for accepted form.

Output

- Returns `::pluss::ai::nodej::prep::FeatureTransformer`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = make_feature_transformer(/* frame */, /* data_options */, /* branches */, /
↪* remainder */);
out.info("make_feature_transformer", value);
```

Expected output

```
make_feature_transformer returned ::pluss::ai::nodej::prep::FeatureTransformer
```

Example 2 (Demo Pattern)

```
  const auto categorical = housing.note_features().feature_names();

  pluss::ai::nodej::DataFrameBatchOptions data_options{};
  data_options.feature_names = {"median_income", "ocean_proximity", "target"};
  data_options.label_feature = "target";
  data_options.note_policy = pluss::ai::nodej::DataNotePolicy::Hash;
  data_options.layout = pluss::ai::nodej::ObservationLayout::Rows;

  auto preprocessing = lymph::make_feature_transformer(
      housing,
      data_options,
      std::vector<lymph::workflow::feature::Branch>{
          lymph::workflow::feature::compose(
              {
                  lymph::workflow::stage::simple_
↪imputer(lymph::prep::SimpleImputerOptions{
                      .strategy = lymph::prep::SimpleImputerStrategy::Mean,
                  }),
                  lymph::workflow::stage::standard_scaler(),
              },
              {"median_income"}),
          lymph::workflow::feature::one_hot_encoder(lymph::prep::OneHotEncoderOptions{},␣
↪categorical),
```

Expected output

```
numeric features: median_income target
categorical features: ocean_proximity
ordinal mapped names: ocean_proximity_ordinal
label mapped names: ocean_proximity_label
ordinal sample: ======================
     | ocean_proximity
----------------------
0    | 2
1    | 1
2    | 2
3    | 1
4    | 2
...
```

make_holdout_indices

API Signature

- make_holdout_indices(frame: const pluss::table::DataFrame&, options: const HoldoutSplitOptions&) -> IndexSplit

Notes

Namespace-level API function.

Inputs

- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `options` (`const HoldoutSplitOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `IndexSplit`.

Options

Option semantics used by this method:

- `HoldoutSplitOptions` (`struct`):
  - `double test_size = 0.2` Option field used by this method.
  - `Series stratify = Series::undefined()` Option field used by this method.
  - `std::string stratify_feature{}` Option field used by this method.
  - `std::uint64_t random_state = 0` Random seed for reproducible behavior.
  - `bool deterministic = true` Option field used by this method.
  - `bool shuffle = true` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = make_holdout_indices(/* frame */, /* options */);
out.info("make_holdout_indices", value);
```

Expected output

```
make_holdout_indices returned IndexSplit
```

make_holdout_split_options

API Signature

- `make_holdout_split_options(options:  const HoldoutSplitOptions&) -> SplitOptions`

Notes

Builds train/test index partitions according to split-plan options.

Inputs

- `options` (`const HoldoutSplitOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `SplitOptions`.

Options

Option semantics used by this method:

- `HoldoutSplitOptions` (`struct`):
  - `double test_size = 0.2` Option field used by this method.
  - `Series stratify = Series::undefined()` Option field used by this method.
  - `std::string stratify_feature{}` Option field used by this method.
  - `std::uint64_t random_state = 0` Random seed for reproducible behavior.
  - `bool deterministic = true` Option field used by this method.
  - `bool shuffle = true` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = make_holdout_split_options(/* options */);
out.info("make_holdout_split_options", value);
```

Expected output

```
split completed
train/test partitions ready
```

make_pipeline

API Signature

- `make_pipeline(spec: const ModelSpec&, stages: std::initializer_list<TransformSpec>) -> Workflow`
- `make_pipeline(model: WorkflowModel, stages: std::initializer_list<TransformSpec>) -> Workflow`

Notes

Namespace-level API function.

Inputs

- `model` (`WorkflowModel`) Method argument; see signature type for accepted form.
- `spec` (`const ModelSpec&`) Method argument; see signature type for accepted form.
- `stages` (`std::initializer_list<TransformSpec>`) List-like argument; pass values in the order expected by the signature.

Output

- Returns `Workflow`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = make_pipeline(/* spec */, /* stages */);
out.info("make_pipeline", value);
```

Expected output

```
make_pipeline returned Workflow
```

make_triplet

API Signature

- `make_triplet(rows: std::int64_t, cols: std::int64_t, row_indices: RangeView::Int, column_indices: RangeView::Int, values: RangeView::Of<double>) -> Triplet`
- `make_triplet(rows: std::int64_t, cols: std::int64_t, row_indices: std::initializer_list<std::int64_t>, column_indices: std::initializer_list<std::int64_t>, values: std::initializer_list<double>) -> Triplet`

Notes

Namespace-level API function.

Inputs

- `cols` (`std::int64_t`) Method argument; see signature type for accepted form.
- `column_indices` (`RangeView::Int | std::initializer_list<std::int64_t>`) List-like argument; pass values in the order expected by the signature.
- `row_indices` (`RangeView::Int | std::initializer_list<std::int64_t>`) List-like argument; pass values in the order expected by the signature.
- `rows` (`std::int64_t`) Method argument; see signature type for accepted form.
- `values` (`RangeView::Of<double> | std::initializer_list<double>`) List-like argument; pass values in the order expected by the signature.

Output

- Returns `Triplet`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = make_triplet(/* rows */, /* cols */, /* row_indices */, /* column_indices */
↪, /* values */);
out.info("make_triplet", value);
```

Expected output

```
make_triplet returned Triplet
```

`max_abs_diff`

API Signature

- `max_abs_diff(lhs:  const Series&, rhs:  const Series&) -> double`

Notes

Namespace-level API function.

Inputs

- `lhs` (`const Series&`) Pluss numeric vector/tensor input.
- `rhs` (`const Series&`) Pluss numeric vector/tensor input.

Output

- Returns `double`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = max_abs_diff(/* lhs */, /* rhs */);
out.info("max_abs_diff", value);
```

Expected output

```
max_abs_diff: <numeric_result>
```

Example 2 (Demo Pattern)

```
    const auto test_metrics = best.assess(test_x, test_y, assess_options);
    std::cout << "test metrics [rmse, r2, mae]: " << test_metrics.rmse << " " << test_
↪metrics.r2 << " " << test_metrics.mae << "\n";

    const Series pred = best.infer(test_x);
    const fs::path best_model_path = model_dir / "hour_best_workflow.bin";
    best.save(best_model_path.string(), train_x, train_y, train_options);
    auto restored = nja::Workflow::load(best_model_path.string());
    const Series pred_restored = restored.infer(test_x);
    const double restored_max_abs_diff = nja::max_abs_diff(pred, pred_restored);
    std::cout << "restored prediction max_abs_diff: " << restored_max_abs_diff << "\n";

    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});
    const Series y_pred = pred.to(pluss::pCPU, ScalarType::Float64).reshape({pred.
↪numel()});
    DataFrame pred_frame({
        {"y_true", Thread(y_true)},
        {"y_pred", Thread(y_pred)},
        {"residual", Thread(y_true.sub(y_pred))},
    });
    pred_frame.save_csv((out_dir / "hour_workflow_tuning_predictions.csv").string());
    pred_frame.plot()
        .scatter("y_true", "y_pred")
```

Expected output

```
CV objective metric: rmse
candidate=linear_ridge objective=102.872 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[102.87158] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.41650024] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[77.611594]
candidate=decision_tree objective=44.9727 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[44.972693] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.88843998] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[27.61723]
candidate=random_forest objective=81.1235 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[81.123538] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.63710967] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
```

(continues on next page)

(continued from previous page)

```
...
```

`MultiLinearModel::class_count`

API Signature

- `MultiLinearModel::class_count() -> std::int64_t`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = MultiLinearModel::class_count();
out.info("class_count", value);
```

Expected output

```
class_count: <numeric_result>
```

`MultiLinearModel::dump_state`

API Signature

- `MultiLinearModel::dump_state() -> std::vector<Series>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::vector<Series>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = MultiLinearModel::dump_state();
out.info("dump_state", value);
```

Expected output

```
dump_state returned std::vector<Series>
```

Example 2 (Demo Pattern)

```
  StatusOr<Series> test_maxabs_scaled_or = maxabs_scaler.map(test_simple_batch);
  if (!test_maxabs_scaled_or.ok()) {
    std::cerr << "MaxAbsScaler.map(test) failed: " << test_maxabs_scaled_or.status().
↪message << "\n";
    return 1;
  }
  Series test_maxabs_scaled = test_maxabs_scaled_or.move_value();

  // This section saves compact prep-state diagnostics.
  StatusOr<std::vector<Series>> simple_state_or = simple_imputer.dump_state();
  if (!simple_state_or.ok()) {
    std::cerr << "SimpleImputer.dump_state failed: " << simple_state_or.status().message
↪<< "\n";
    return 1;
  }
  std::vector<Series> simple_state = simple_state_or.move_value();

  StatusOr<std::vector<Series>> knn_state_or = knn_imputer.dump_state();
  if (!knn_state_or.ok()) {
    std::cerr << "NeighborImputer.dump_state failed: " << knn_state_or.status().message <
↪< "\n";
    return 1;
  }
  std::vector<Series> knn_state = knn_state_or.move_value();
```

Expected output

```
dump_state returned std::vector<Series>
```

`MultiLinearModel::feature_count`

API Signature

- `MultiLinearModel::feature_count() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = MultiLinearModel::feature_count();
out.info("feature_count", value);
```

Expected output

```
feature_count: <numeric_result>
```

Example 2 (Demo Pattern)

```
  options.note_policy = DataNotePolicy::Hash;
  options.thread_policy = DataThreadPolicy::CastTemporalToSeries;

  auto batch_or = labeled_batch_from_dataframe(frame, options);
  if (!require_ok(batch_or, "labeled_batch_from_dataframe")) return 1;

  const LabeledBatch& batch = batch_or.value();
  std::cout << "adapted batch: samples=" << batch.features.sample_count()
            << " features=" << batch.features.feature_count()
            << " weighted=" << (batch.sample_weights.defined() ? "yes" : "no") << "\n";

  ModelSpec spec{
      .task = TaskKind::Regression,
      .family = ModelFamily::Linear,
      .options = LinearFamilyOptions{
          .objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
```

(continues on next page)

(continued from previous page)

```
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

Example 3 (Demo Pattern)

```
}

Config i64() {
  return Config{}.with_dtype(ScalarType::Int64).with_platform(pluss::pCPU);
}

LabeledBatch slice_rows(const LabeledBatch& batch, std::int64_t start, std::int64_t␣
↪count) {
  const Series idx = pluss::arange(start, start + count, i64());
  const Series x = batch.features.x.index_select(0, idx).reshape({count, batch.features.
↪feature_count()});
  const Series y = batch.y.index_select(0, idx).reshape({count});
  return LabeledBatch{
      .features = FeatureBatch{.x = x, .layout = ObservationLayout::Rows},
      .y = y,
  };
}

LabeledBatch make_class_batch() {
  std::vector<double> xv{};
  std::vector<double> yv{};
  xv.reserve(2 * 90);
  yv.reserve(90);
```

Expected output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

MultiLinearModel::infer

API Signature

- `MultiLinearModel::infer(x: const Csr&) -> Series`
- `MultiLinearModel::infer(x: const Triplet&) -> Series`

Notes

Computes predictions from prepared inputs using the currently learned state.

Inputs

- `x` (`const Csr& | const Triplet&`) Input feature data used by this call.

Output

- Returns `Series`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = MultiLinearModel::infer(/* x */);
out.info("infer", value);
```

Expected output

```
infer returned Series
```

Example 2 (Demo Pattern)

```
  train.tolerance = 1e-8;
  train.random.seed = 77;
  train.random.deterministic = true;
  if (!flow.learn(batch, train).ok()) {
    std::cerr << "flow learn failed\n";
    return 1;
  }

  auto pred_before_or = flow.infer(batch.features);
  if (!pred_before_or.ok()) {
    std::cerr << "flow infer before save failed: " << pred_before_or.status().message <<
↪"\n";
    return 1;
  }
  const std::string model_dir = "<repo_root>/PlussAI/examples/NodeJ/trained_models";
  std::error_code mkdir_ec;
  std::filesystem::create_directories(model_dir, mkdir_ec);
  if (mkdir_ec) {
    std::cerr << "failed to create model directory: " << model_dir << " (" << mkdir_ec.
↪message() << ")\n";
    return 1;
  }
  const std::string bundle_path = model_dir + "/nodej_flow_bundle_demo.bin";
```

Expected output

```
libc++abi: terminating due to uncaught exception of type pluss::Error: DataReader::with_
↪sizes: explicit sizes are only supported for host-buffer inputs
Triggered at: <path> (DataReader &pluss::detail::DataReader::with_sizes(std::span<const␣
↪std::int64_t>))
Native stack trace:
  0  pluss::detail::DataReader::with_sizes(std::__1::span<long long const,␣
↪18446744073709551615ul>)(data_reader.h:250)
  1  pluss::codem::Series pluss::detail::DataReader::to_series<pluss::codem::Config␣
↪const&>(pluss::detail::DataReader::Option, pluss::codem::Config const&) const(data_
↪reader.h:295)
  2  pluss::codem::Series pluss::codem::value<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_value.h:13)
  3  pluss::codem::Series pluss::make_series<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_factory.h:960)
  4  main(nodej_flow_bundle_demo.cpp:13)
  5  start + 6992 [dyld]


[exit_code] -6
```

Example 3 (Demo Pattern)

```
}

void run_classifier_family(const std::string& name, const ModelSpec& spec, const␣
↪LabeledBatch& train) {
  auto model_or = make_classifier(spec);
  if (!require_ok(model_or, name + " make_classifier")) return;
  std::unique_ptr<ClassifierBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;

  auto pred_or = model->infer(train.features);
  if (!require_ok(pred_or, name + " infer")) return;
  auto metric_or = model->assess(train, AssessOptions{.metrics = {MetricKind::Accuracy,␣
↪MetricKind::LogLoss}});
  if (!require_ok(metric_or, name + " assess")) return;
  print_metric("  " + name + " (acc, logloss)", metric_or.value());
}

void run_regressor_family(const std::string& name, const ModelSpec& spec, const␣
↪LabeledBatch& train) {
  auto model_or = make_regressor(spec);
  if (!require_ok(model_or, name + " make_regressor")) return;
  std::unique_ptr<RegressorBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;
```

Expected output

```
NodeJ model families demo
Classification families:
  linear (acc, logloss): 1 4.47877e-06
  linear_hinge (acc, logloss): 1 0.208572
  tree (acc, logloss): 1 9.99978e-13
  ensemble (acc, logloss): 1 1.00003e-12
  neighbor (acc, logloss): 1 9.99978e-13
  kernel (acc, logloss): 1 0.318438
  bayes (acc, logloss): 1 9.99978e-13
Regression families:
  linear (mse, r2): 1.52381e-19 1
  tree (mse, r2): 0 1
...
```

MultiLinearModel::infer_scores

API Signature

- `MultiLinearModel::infer_scores(x: const Csr&) -> Series`
- `MultiLinearModel::infer_scores(x: const Triplet&) -> Series`

Notes

Computes predictions from prepared inputs using the currently learned state.

Inputs

- `x` (`const Csr& | const Triplet&`) Input feature data used by this call.

Output

- Returns `Series`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto value = MultiLinearModel::infer_scores(/* x */);
out.info("infer_scores", value);
```

Expected output

```
infer_scores returned Series
```

Example 2 (Demo Pattern)

```
  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }

  auto bounded_scores_or = bounded->infer_scores(batch.features);
  if (!require_ok(bounded_scores_or, "infer_scores(bounded)")) return 1;
  std::cout << "max |bounded score|: " << bounded_scores_or.value().abs().max().item
↪<double>() << "\n";

  std::cout << "NodeJ CV + tune demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪1.12262
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.169779
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.420312
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0248626
sweep trial objectives: 0 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0352983 0.743786
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 3 (Demo Pattern)

```
  }
  auto cls = cls_or.move_value();
  const auto learn_status = cls->learn(batch);
  if (!learn_status.ok()) {
    std::cerr << "learn failed: " << learn_status.message << '\n';
    return 1;
  }

  auto score_or = cls->infer_scores(batch.features);
  if (!score_or.ok()) {
    std::cerr << "infer_scores failed: " << score_or.status().message << '\n';
```

(continues on next page)

(continued from previous page)

```
    return 1;
  }
  std::cout << "scores shape: [" << score_or.value().sizes()[0] << ", " << score_or.
↪value().sizes()[1] << "]\n";

  pluss::ai::nodej::AssessOptions assess{};
  assess.metrics = {MetricKind::Accuracy, MetricKind::F1, MetricKind::LogLoss};
  auto metric_or = cls->assess(batch, assess);
  if (!metric_or.ok()) {
    std::cerr << "assess failed: " << metric_or.status().message << '\n';
    return 1;
```

Expected output

```
scores shape: [12, 3]
metrics [accuracy, f1, logloss]: 1 1 1.24859e-10
flow metrics [accuracy, f1, logloss]: 1 1 1.48016e-09
```

`MultiLinearModel::trained`

API Signature

- `MultiLinearModel::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = MultiLinearModel::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

objective_rank_series

API Signature

- `objective_rank_series(report:  const TuneReport&) -> Series`

Notes

Namespace-level API function.

Inputs

- `report` (`const TuneReport&`) Method argument; see signature type for accepted form.

Output

- Returns `Series`.

Options

Option semantics used by this method:

- `TuneReport` (`struct`):
  - `std::vector<Trial> trials{}` Option field used by this method.
  - `std::int64_t best_index = -1` Option field used by this method.
  - `std::string best_name{}` Option field used by this method.
  - `MetricKind objective_metric = MetricKind::Accuracy` Selects optimization or evaluation objective.
  - `TuneObjectivePolicy objective_policy = TuneObjectivePolicy::Auto` Selects optimization or evaluation objective.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = objective_rank_series(/* report */);
out.info("objective_rank_series", value);
```

Expected output

```
objective_rank_series returned Series
```

objective_series

API Signature

- `objective_series(report:  const TuneReport&) -> Series`

Notes

Namespace-level API function.

Inputs

- `report` (`const TuneReport&`) Method argument; see signature type for accepted form.

Output

- Returns `Series`.

Options

Option semantics used by this method:

- `TuneReport` (`struct`):
  - `std::vector<Trial> trials{}` Option field used by this method.
  - `std::int64_t best_index = -1` Option field used by this method.
  - `std::string best_name{}` Option field used by this method.
  - `MetricKind objective_metric = MetricKind::Accuracy` Selects optimization or evaluation objective.
  - `TuneObjectivePolicy objective_policy = TuneObjectivePolicy::Auto` Selects optimization or evaluation objective.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = objective_series(/* report */);
out.info("objective_series", value);
```

Expected output

```
objective_series returned Series
```

online_factory::encoded_param_table

API Signature

- `online_factory::encoded_param_table(result:  const SpecTuneResult&) -> Series`

Notes

Factory helper that constructs a configured model/workflow object.

Inputs

- `result` (`const SpecTuneResult&`) Method argument; see signature type for accepted form.

Output

- Returns `Series`.

Options

Option semantics used by this method:

- `SpecTuneResult` (`struct`):
  - `::pluss::ai::nodej::TuneReport report{}` Option field used by this method.
  - `ModelSpec best_spec{}` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = online_factory::encoded_param_table(/* result */);
out.info("encoded_param_table", value);
```

Expected output

```
encoded_param_table returned Series
```

`online_factory::has_ledger`

API Signature

- `online_factory::has_ledger(result:  const SpecTuneResult&) -> bool`

Notes

Boolean introspection helper that reports whether a capability/state is available.

Inputs

- `result` (`const SpecTuneResult&`) Method argument; see signature type for accepted form.

Output

- Returns `bool`.

Options

Option semantics used by this method:

- `SpecTuneResult` (`struct`):
  - `::pluss::ai::nodej::TuneReport report{}` Option field used by this method.
  - `ModelSpec best_spec{}` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = online_factory::has_ledger(/* result */);
out.info("has_ledger", value);
```

Expected output

```
has_ledger: true
```

online_factory::objective_rank_series

API Signature

- `online_factory::objective_rank_series(result:  const SpecTuneResult&) -> Series`

Notes

Factory helper that constructs a configured model/workflow object.

Inputs

- `result` (`const SpecTuneResult&`) Method argument; see signature type for accepted form.

Output

- Returns `Series`.

Options

Option semantics used by this method:

- `SpecTuneResult` (`struct`):
  - `::pluss::ai::nodej::TuneReport report{}` Option field used by this method.
  - `ModelSpec best_spec{}` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = online_factory::objective_rank_series(/* result */);
out.info("objective_rank_series", value);
```

Expected output

```
objective_rank_series returned Series
```

online_factory::objective_series

API Signature

- online_factory::objective_series(result: const SpecTuneResult&) -> Series

Notes

Factory helper that constructs a configured model/workflow object.

Inputs

- result (const SpecTuneResult&) Method argument; see signature type for accepted form.

Output

- Returns Series.

Options

Option semantics used by this method:

- SpecTuneResult (struct):
  - ::pluss::ai::nodej::TuneReport report{} Option field used by this method.
  - ModelSpec best_spec{} Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = online_factory::objective_series(/* result */);
out.info("objective_series", value);
```

Expected output

```
objective_series returned Series
```

online_factory::tune_ledger

API Signature

- online_factory::tune_ledger(result: const SpecTuneResult&) -> const ::pluss::ai::nodej::TuneLedger&

Notes

Runs parameter search and returns per-trial metrics plus best-trial selection.

Inputs

- `result` (`const SpecTuneResult&`) Method argument; see signature type for accepted form.

Output

- Returns `const ::pluss::ai::nodej::TuneLedger&`.

Options

Option semantics used by this method:

- `SpecTuneResult` (`struct`):
  - `::pluss::ai::nodej::TuneReport report{}` Option field used by this method.
  - `ModelSpec best_spec{}` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = online_factory::tune_ledger(/* result */);
out.info("tune_ledger", value);
```

Expected output

```
tune_ledger returned const ::pluss::ai::nodej::TuneLedger&
```

`OnlineLinearClassifier::core`

API Signature

- `OnlineLinearClassifier::core() -> ::pluss::ai::nodej::online::OnlineLinearClassifier&`
- `OnlineLinearClassifier::core() -> const ::pluss::ai::nodej::online::OnlineLinearClassifier&`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `::pluss::ai::nodej::online::OnlineLinearClassifier&`.
- Returns `const ::pluss::ai::nodej::online::OnlineLinearClassifier&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OnlineLinearClassifier::core();
out.info("core", value);
```

Expected output

```
core returned ::pluss::ai::nodej::online::OnlineLinearClassifier&
```

Example 2 (Demo Pattern)

```
  try {
    raw = DataFrame::load_csv(data_path.string(), {{"ncases", ncases}});
  } catch (const std::exception& ex) {
    std::cerr << "load_csv failed: " << ex.what() << "\n";
    return 1;
  }

  //========== schema normalization ========
  // This section gives consistent names and explicit core column dtypes.
  DataFrame frame = raw.rename_features({
      {"dteday", "date"},
      {"holiday", "is_holiday"},
      {"workingday", "is_workingday"},
      {"weathersit", "weather_condition"},
      {"hum", "humidity"},
      {"mnth", "month"},
      {"cnt", "total_count"},
      {"hr", "hour"},
      {"yr", "year"},
  });
```

Expected output

```
core returned ::pluss::ai::nodej::online::OnlineLinearClassifier&
```

OnlineLinearClassifier::OnlineLinearClassifier

API Signature

- `OnlineLinearClassifier(arg1: ::pluss::ai::nodej::online::OnlineLinearClassifier core) : core_(std::move(core)) -> auto`

Notes

Instance method on `self`.

Inputs

- `arg1` (`::pluss::ai::nodej::online::OnlineLinearClassifier core`) : `core_(std::move(core))` Method argument; see signature type for accepted form.

Output

- Returns method result for the selected overload.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OnlineLinearClassifier(/* arg1 */);
out.info("OnlineLinearClassifier", value);
```

Expected output

```
OnlineLinearClassifier returned result
```

`OnlineLinearClassifier::seen_samples`

API Signature

- `OnlineLinearClassifier::seen_samples() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OnlineLinearClassifier::seen_samples();
out.info("seen_samples", value);
```

Expected output

```
seen_samples: <numeric_result>
```

Example 2 (Demo Pattern)

```
  pluss::ai::nodej::AssessOptions assess{};
  assess.metrics = {MetricKind::Accuracy, MetricKind::LogLoss};
  auto metric_or = model.assess(full, assess);
  if (!metric_or.ok()) {
    std::cerr << "assess failed: " << metric_or.status().message << '\n';
    return 1;
  }

  std::cout << "online seen_samples=" << model.seen_samples() << '\n';
  std::cout << "online metrics [accuracy, logloss]: ";
  print_vec(metric_or.value());
  return 0;
}
```

Expected output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

Example 3 (Demo Pattern)

```
    warm_opts.max_iterations = 80;

    auto model = pluss::ai::lymph::online_factory::linear_classifier(online_options);
    model.learn_(warm, warm_opts).advance_(stream);

    const auto metrics = model.assess(full);
    const Series metric_values = metrics.accuracy.defined() ? metrics.accuracy : metrics.
↪values;

    std::cout << "lymph online seen_samples=" << model.seen_samples() << '\n';
    std::cout << "lymph online metrics [accuracy]: ";
    print_vec(metric_values);
    return 0;
  } catch (const std::exception& ex) {
    std::cerr << "lymph_online_demo failed: " << ex.what() << '\n';
    return 1;
  }
}
```

Expected output

```
lymph online seen_samples=1660
lymph online metrics [accuracy]: 0.9875
```

OnlineLinearClassifier::trained

API Signature

- `OnlineLinearClassifier::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OnlineLinearClassifier::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

OnlineLinearRegressor::core

API Signature

- `OnlineLinearRegressor::core() -> ::pluss::ai::nodej::online::OnlineLinearRegressor&`
- `OnlineLinearRegressor::core() -> const ::pluss::ai::nodej::online::OnlineLinearRegressor&`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `::pluss::ai::nodej::online::OnlineLinearRegressor&`.
- Returns `const ::pluss::ai::nodej::online::OnlineLinearRegressor&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OnlineLinearRegressor::core();
out.info("core", value);
```

Expected output

```
core returned ::pluss::ai::nodej::online::OnlineLinearRegressor&
```

Example 2 (Demo Pattern)

```
  try {
    raw = DataFrame::load_csv(data_path.string(), {{"ncases", ncases}});
  } catch (const std::exception& ex) {
    std::cerr << "load_csv failed: " << ex.what() << "\n";
    return 1;
  }

  //========== schema normalization ========
  // This section gives consistent names and explicit core column dtypes.
  DataFrame frame = raw.rename_features({
      {"dteday", "date"},
      {"holiday", "is_holiday"},
      {"workingday", "is_workingday"},
      {"weathersit", "weather_condition"},
      {"hum", "humidity"},
      {"mnth", "month"},
      {"cnt", "total_count"},
      {"hr", "hour"},
      {"yr", "year"},
  });
```

Expected output

```
core returned ::pluss::ai::nodej::online::OnlineLinearRegressor&
```

OnlineLinearRegressor::OnlineLinearRegressor

API Signature

- `OnlineLinearRegressor(arg1: ::pluss::ai::nodej::online::OnlineLinearRegressor core) : core_(std::move(core)) -> auto`

Notes

Instance method on `self`.

Inputs

- `arg1` (`::pluss::ai::nodej::online::OnlineLinearRegressor core) : core_(std::move(core))`) Method argument; see signature type for accepted form.

Output

- Returns method result for the selected overload.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OnlineLinearRegressor(/* arg1 */);
out.info("OnlineLinearRegressor", value);
```

Expected output

```
OnlineLinearRegressor returned result
```

OnlineLinearRegressor::seen_samples

API Signature

- `OnlineLinearRegressor::seen_samples() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OnlineLinearRegressor::seen_samples();
out.info("seen_samples", value);
```

Expected output

```
seen_samples: <numeric_result>
```

Example 2 (Demo Pattern)

```
  pluss::ai::nodej::AssessOptions assess{};
  assess.metrics = {MetricKind::Accuracy, MetricKind::LogLoss};
  auto metric_or = model.assess(full, assess);
  if (!metric_or.ok()) {
    std::cerr << "assess failed: " << metric_or.status().message << '\n';
    return 1;
  }

  std::cout << "online seen_samples=" << model.seen_samples() << '\n';
  std::cout << "online metrics [accuracy, logloss]: ";
  print_vec(metric_or.value());
  return 0;
}
```

Expected output

```
online seen_samples=1872
online metrics [accuracy, logloss]: 1 0.27341
```

Example 3 (Demo Pattern)

```
    warm_opts.max_iterations = 80;

    auto model = pluss::ai::lymph::online_factory::linear_classifier(online_options);
    model.learn_(warm, warm_opts).advance_(stream);

    const auto metrics = model.assess(full);
    const Series metric_values = metrics.accuracy.defined() ? metrics.accuracy : metrics.
↪values;
```

(continues on next page)

(continued from previous page)

```
    std::cout << "lymph online seen_samples=" << model.seen_samples() << '\n';
    std::cout << "lymph online metrics [accuracy]: ";
    print_vec(metric_values);
    return 0;
  } catch (const std::exception& ex) {
    std::cerr << "lymph_online_demo failed: " << ex.what() << '\n';
    return 1;
  }
}
```

Expected output

```
lymph online seen_samples=1660
lymph online metrics [accuracy]: 0.9875
```

`OnlineLinearRegressor::trained`

API Signature

- `OnlineLinearRegressor::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OnlineLinearRegressor::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

`OutlierIsolator::features`

API Signature

- `OutlierIsolator::features(names: RangeView::Of<std::string_view>) -> OutlierIsolator&`
- `OutlierIsolator::features(names: std::initializer_list<std::string_view>) -> OutlierIsolator&`

Notes

Instance method on `self`.

Inputs

- `names` (`RangeView::Of<std::string_view> | std::initializer_list<std::string_view>`) List-like argument; pass values in the order expected by the signature.

Output

- Returns `OutlierIsolator&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OutlierIsolator::features(/* names */);
out.info("features", value);
```

Expected output

```
features returned OutlierIsolator&
```

Example 2 (Demo Pattern)

```
    const auto split = nja::holdout_split(frame, nja::HoldoutSplitOptions{
                                                      .test_size = 0.2,
                                                      .random_state = 42,
                                                      .deterministic = true,
                                                      .shuffle = true,
                                                  });
    const DataFrame train_x = split.train_set.features(
        {"season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
↪"weather_condition", "temp", "atemp", "humidity", "windspeed"});
    const DataFrame test_x = split.test_set.features(
        {"season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
↪"weather_condition", "temp", "atemp", "humidity", "windspeed"});
```

(continues on next page)

(continued from previous page)

```
    const Series train_y = split.train_set.features("total_count").as_series();
    const Series test_y = split.test_set.features("total_count").as_series();

    TrainOptions train_options{};
    train_options.max_iterations = 2500;
    train_options.tolerance = 1e-8;
    train_options.random.seed = 7;
    train_options.random.deterministic = true;
```

Expected output

```
CV objective metric: rmse
candidate=linear_ridge objective=102.872 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[102.87158] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.41650024] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[77.611594]
candidate=decision_tree objective=44.9727 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[44.972693] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.88843998] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[27.61723]
candidate=random_forest objective=81.1235 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[81.123538] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.63710967] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
...
```

Example 3 (Demo Pattern)

```
  std::cout << "numeric features: ";
  for (const auto& n : numeric) std::cout << n << " ";
  std::cout << "\n";
  std::cout << "categorical features: ";
  for (const auto& n : categorical) std::cout << n << " ";
  std::cout << "\n";
  auto ord = lymph::prep::OrdinalEncoder({.include_unknown_bucket = true, .one_based =␣
↪true});
  ord.features({"ocean_proximity"});
  const auto ocean_ord = ord.learn_map(housing.features({"ocean_proximity"}));
  auto lbl = lymph::prep::LabelEncoder({.include_unknown_bucket = true, .one_based =␣
↪false});
  const auto ocean_lbl = lbl.learn_map(housing.features("ocean_proximity"));
  std::cout << "ordinal mapped names: ";
  for (const auto& name : ord.mapped_names_) std::cout << name << " ";
```

(continues on next page)

(continued from previous page)

```
  std::cout << "\n";
  std::cout << "label mapped names: ";
  for (const auto& name : lbl.mapped_names_) std::cout << name << " ";
  std::cout << "\n";
  std::cout << "ordinal sample: " << ocean_ord << "\n";
  std::cout << "label sample: " << ocean_lbl << "\n";
  std::cout << "pred: " << pred << "\n";
```

Expected output

```
numeric features: median_income target
categorical features: ocean_proximity
ordinal mapped names: ocean_proximity_ordinal
label mapped names: ocean_proximity_label
ordinal sample: ======================
     | ocean_proximity
----------------------
0    | 2
1    | 1
2    | 2
3    | 1
4    | 2
...
```

`OutlierIsolator::learned`

API Signature

- `OutlierIsolator::learned() -> bool`

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OutlierIsolator::learned();
out.info("learned", value);
```

Expected output

```
learned: true
```

`OutlierIsolator::summary`

API Signature

- `OutlierIsolator::summary() -> OutlierIsolatorSummary`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `OutlierIsolatorSummary`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = OutlierIsolator::summary();
out.info("summary", value);
```

Expected output

```
summary returned OutlierIsolatorSummary
```

Example 2 (Demo Pattern)

```
    //========== standalone lymph preprocessing API ========
    // This section demonstrates direct prep estimators (learn/map/learn_map)
    // without going through Workflow composition.
    auto simple_imp = lymph::prep::simple_imputer(lymph::prep::SimpleImputerOptions{
        .strategy = lymph::prep::SimpleImputerStrategy::Median,
    });
    const DataFrame train_simple_imputed = simple_imp.learn_map(train_x);
    const DataFrame test_simple_imputed = simple_imp.map(test_x);
```

(continues on next page)

(continued from previous page)

```
    const auto simple_imp_summary = simple_imp.summary();

    auto knn_imp = lymph::prep::neighbor_imputer(lymph::prep::NeighborImputerOptions{
        .neighbors = 7,
        .weighting = lymph::prep::NeighborImputerWeighting::Distance,
        .distance_floor = 1e-12,
        .empty_feature_fallback = 0.0,
    });
    const DataFrame train_knn_imputed = knn_imp.learn_map(train_x);
    const DataFrame test_knn_imputed = knn_imp.map(test_x);
    const auto knn_imp_summary = knn_imp.summary();

    auto iter_imp = lymph::prep::iterative_imputer(lymph::prep::IterativeImputerOptions{
```

Expected output

```
summary returned OutlierIsolatorSummary
```

`output_render`

API Signature

- `output_render(os: std::ostream&, summary: const CategoryIndexerSummary&) -> void`
- `output_render(os: std::ostream&, summary: const OutlierIsolatorSummary&) -> void`
- `output_render(os: std::ostream&, summary: const SimpleImputerSummary&) -> void`
- `output_render(os: std::ostream&, summary: const StandardScalerSummary&) -> void`
- `output_render(os: std::ostream&, summary: const TransformerSummary&) -> void`

Notes

Namespace-level API function.

Inputs

- `os` (`std::ostream&`) Method argument; see signature type for accepted form.
- `summary` (`const CategoryIndexerSummary& | const OutlierIsolatorSummary& | const SimpleImputerSummary& | const StandardScalerSummary& | const TransformerSummary&`) Target values aligned to input observations.

Output

- No direct return value; operation applies side effects to object state.

Options

Option semantics used by this method:

- `CategoryIndexerSummary` (`struct`):
  - `bool learned = false` Option field used by this method.
  - `std::int64_t feature_count = 0` Option field used by this method.

  - `bool include_unknown_bucket = true` Controls fold/trial/count sizing.
  - `bool one_based = true` Option field used by this method.
  - `std::vector<std::string> feature_names{}` Explicit list of feature columns to read from the input frame.
  - `std::vector<std::string> mapped_names{}` Option field used by this method.
  - `std::vector<std::vector<std::string>> categories{}` Option field used by this method.
- `OutlierIsolatorSummary` (`struct`):
  - `bool learned = false` Option field used by this method.
  - `std::int64_t feature_count = 0` Option field used by this method.
  - `std::int64_t estimators = 0` Option field used by this method.
  - `std::int64_t max_samples = 0` Option field used by this method.
  - `std::int64_t max_features = 0` Option field used by this method.
  - `bool bootstrap = false` Option field used by this method.
  - `bool contamination_auto = true` Controls fold/trial/count sizing.
  - `double contamination = 0.1` Option field used by this method.
  - `double offset = -0.5` Option field used by this method.
  - `std::vector<std::string> feature_names{}` Explicit list of feature columns to read from the input frame.
- `SimpleImputerSummary` (`struct`):
  - `SimpleImputerStrategy strategy = SimpleImputerStrategy::Mean` Option field used by this method.
  - `bool learned = false` Option field used by this method.
  - `std::int64_t feature_count = 0` Option field used by this method.
  - `double constant_value = 0.0` Option field used by this method.
  - `double empty_feature_fallback = 0.0` Option field used by this method.
  - `std::vector<std::string> feature_names{}` Explicit list of feature columns to read from the input frame.
  - `Series statistics = Series::undefined()` Option field used by this method.
- `StandardScalerSummary` (`struct`):
  - `std::vector<std::string> feature_names{}` Explicit list of feature columns to read from the input frame.
  - `Series mean = Series::undefined()` Option field used by this method.
  - `Series scale = Series::undefined()` Option field used by this method.
- `TransformerSummary` (`struct`):
  - `std::vector<std::string> feature_names{}` Explicit list of feature columns to read from the input frame.
  - `std::vector<Series> state{}` Option field used by this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
output_render(/* os */, /* summary */);
out.info("output_render completed");
```

Expected output

```
output_render completed
```

pipeline

API Signature

- `pipeline(spec: const ModelSpec&, stages: std::initializer_list<TransformSpec>) -> Workflow`
- `pipeline(model: WorkflowModel, stages: std::initializer_list<TransformSpec>) -> Workflow`

Notes

Namespace-level API function.

Inputs

- `model` (`WorkflowModel`) Method argument; see signature type for accepted form.
- `spec` (`const ModelSpec&`) Method argument; see signature type for accepted form.
- `stages` (`std::initializer_list<TransformSpec>`) List-like argument; pass values in the order expected by the signature.

Output

- Returns `Workflow`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = pipeline(/* spec */, /* stages */);
out.info("pipeline", value);
```

Expected output

```
pipeline returned Workflow
```

Example 2 (Demo Pattern)

```
  Series final_pred = final_pred_or.move_value();

  std::cout << "\n========== final holdout metrics ========\n";
  std::cout << "rmse=" << (final_metrics_vec.size() > 0 ? final_metrics_vec[0] :␣
↪std::numeric_limits<double>::quiet_NaN()) << "\n";
  std::cout << "r2=" << (final_metrics_vec.size() > 1 ? final_metrics_vec[1] :␣
↪std::numeric_limits<double>::quiet_NaN()) << "\n";
  std::cout << "mae=" << (final_metrics_vec.size() > 2 ? final_metrics_vec[2] :␣
↪std::numeric_limits<double>::quiet_NaN()) << "\n";

  //========== flow persistence with best spec ========
  // This section uses core Flow + io::save/io::load_and_restore for exact pipeline␣
↪persistence.
  StatusOr<Flow> final_flow_or = make_flow(best_spec);
  if (!final_flow_or.ok()) {
    std::cerr << "make_flow(best_spec) failed: " << final_flow_or.status().message << "\n
↪";
    return 1;
  }
  Flow final_flow = final_flow_or.move_value();

  Status flow_learn_status = final_flow.learn(train_batch, train_options);
  if (!flow_learn_status.ok()) {
    std::cerr << "final_flow.learn failed: " << flow_learn_status.message << "\n";
    return 1;
  }
```

Expected output

```
pipeline returned Workflow
```

prep::CategoryIndexer::learn

API Signature

- `prep::CategoryIndexer::learn(frame: const pluss::table::DataFrame&, options: const TrainOptions&) -> prep::CategoryIndexer&`
- `prep::CategoryIndexer::learn(feature: const pluss::table::Thread&, options: const TrainOptions&) -> prep::CategoryIndexer&`

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

- `feature` (`const pluss::table::Thread&`) Input feature data used by this call.
- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `options` (`const TrainOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `prep::CategoryIndexer&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = prep::CategoryIndexer::learn(/* frame */, /* options */);
out.info("learn", value);
```

Expected output

```
learn returned prep::CategoryIndexer&
```

Example 2 (Demo Pattern)

```
  auto bounded_or = make_classifier(linear_logistic_spec());
  if (!require_ok(bounded_or, "make_classifier(linear bounded)")) return 1;
  auto bounded = bounded_or.move_value();

  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }

  auto bounded_scores_or = bounded->infer_scores(batch.features);
  if (!require_ok(bounded_scores_or, "infer_scores(bounded)")) return 1;
  std::cout << "max |bounded score|: " << bounded_scores_or.value().abs().max().item
↪<double>() << "\n";

  std::cout << "NodeJ CV + tune demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪1.12262
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.169779
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
```

(continues on next page)

(continued from previous page)

```
↪0.420312
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0248626
sweep trial objectives: 0 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0352983 0.743786
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 3 (Demo Pattern)

```
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
  }

  auto metrics_or = reg->assess(batch, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metrics_or, "assess")) return 1;
  print_metrics("linear metrics [mse, r2]", metrics_or.value());

  std::cout << "NodeJ DataFrame bridge demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

prep::CategoryIndexer::learn_map

API Signature

- prep::CategoryIndexer::learn_map(frame: const pluss::table::DataFrame&, learn_options: const TrainOptions&, infer_options: const InferOptions&) -> pluss::table::DataFrame
- prep::CategoryIndexer::learn_map(feature: const pluss::table::Thread&, learn_options: const TrainOptions&, infer_options: const InferOptions&) -> Series

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

- `feature` (`const pluss::table::Thread&`) Input feature data used by this call.
- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `infer_options` (`const InferOptions&`) Configuration object controlling algorithm behavior and safety defaults.
- `learn_options` (`const TrainOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `Series`.
- Returns `pluss::table::DataFrame`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = prep::CategoryIndexer::learn_map(/* frame */, /* learn_options */, /* infer_
↪options */);
out.info("learn_map", value);
```

Expected output

```
learn_map returned pluss::table::DataFrame
```

Example 2 (Demo Pattern)

```
  if (!require_ok(fold_or, "fold_plan")) return 1;
  std::cout << "kfold folds=" << fold_or.value().folds.size() << "\n";

  SimpleImputer imputer(SimpleImputerOptions{
      .strategy = SimpleImputerStrategy::Median,
      .constant_value = 0.0,
      .empty_feature_fallback = 0.0,
  });
  auto imp_or = imputer.learn_map(dense);
  if (!require_ok(imp_or, "SimpleImputer.learn_map")) return 1;
  const Series x_imputed = imp_or.move_value();
  print_shape("imputed", x_imputed);

  StandardScaler zscore{};
```

(continues on next page)

(continued from previous page)

```
  auto z_or = zscore.learn_map(FeatureBatch{.x = x_imputed, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(z_or, "StandardScaler.learn_map")) return 1;
  const Series x_z = z_or.move_value();
  print_shape("standardized", x_z);

  MinMaxScaler minmax{};
  auto mm_or = minmax.learn_map(FeatureBatch{.x = x_imputed, .layout =␣
↪ObservationLayout::Rows});
```

Expected output

```
NodeJ split + prep demo
holdout train idx count=4 test idx count=2
strat holdout train idx count=4 test idx count=2
kfold folds=3
imputed shape=[6, 3]
standardized shape=[6, 3]
minmax shape=[6, 3]
robust shape=[6, 3]
neighbor-imputed shape=[6, 3]
iterative-imputed shape=[6, 3]
vector-normalized shape=[6, 3]
maxabs shape=[6, 3]
...
```

Example 3 (Demo Pattern)

```
  std::cout << "numeric features: ";
  for (const auto& n : numeric) std::cout << n << " ";
  std::cout << "\n";
  std::cout << "categorical features: ";
  for (const auto& n : categorical) std::cout << n << " ";
  std::cout << "\n";
  auto ord = lymph::prep::OrdinalEncoder({.include_unknown_bucket = true, .one_based =␣
↪true});
  ord.features({"ocean_proximity"});
  const auto ocean_ord = ord.learn_map(housing.features({"ocean_proximity"}));
  auto lbl = lymph::prep::LabelEncoder({.include_unknown_bucket = true, .one_based =␣
↪false});
  const auto ocean_lbl = lbl.learn_map(housing.features("ocean_proximity"));
  std::cout << "ordinal mapped names: ";
  for (const auto& name : ord.mapped_names_) std::cout << name << " ";
  std::cout << "\n";
  std::cout << "label mapped names: ";
  for (const auto& name : lbl.mapped_names_) std::cout << name << " ";
  std::cout << "\n";
  std::cout << "ordinal sample: " << ocean_ord << "\n";
  std::cout << "label sample: " << ocean_lbl << "\n";
  std::cout << "pred: " << pred << "\n";
  std::cout << "mse: " << metrics.mse << " r2: " << metrics.r2 << "\n";
```

Expected output

```
numeric features: median_income target
categorical features: ocean_proximity
ordinal mapped names: ocean_proximity_ordinal
label mapped names: ocean_proximity_label
ordinal sample: ======================
     | ocean_proximity
----------------------
0    | 2
1    | 1
2    | 2
3    | 1
4    | 2
...
```

prep::CategoryIndexer::map

API Signature

- `prep::CategoryIndexer::map(frame: const pluss::table::DataFrame&, options: const InferOptions&) -> pluss::table::DataFrame`
- `prep::CategoryIndexer::map(feature: const pluss::table::Thread&, options: const InferOptions&) -> Series`

Notes

Applies preprocessing/transformation behavior configured by the current options.

Inputs

- `feature` (`const pluss::table::Thread&`) Input feature data used by this call.
- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `options` (`const InferOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `Series`.
- Returns `pluss::table::DataFrame`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto value = prep::CategoryIndexer::map(/* frame */, /* options */);
out.info("map", value);
```

Expected output

```
map returned pluss::table::DataFrame
```

Example 2 (Demo Pattern)

```
  if (!require_ok(poly_or, "PolynomialLift.learn_map")) return 1;
  print_shape("polynomial-lift", poly_or.value());

  const Series x_query = pluss::make_series(std::vector<double>{
                                              7.0, 5.0, 1.0,
                                              8.0, 5.5, 0.0},
                                            {2, 3},
                                            f64());
  auto mapped_or = zscore.map(FeatureBatch{.x = x_query, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(mapped_or, "StandardScaler.map")) return 1;
  print_shape("standardized query", mapped_or.value());

  const Series categories = pluss::make_series(std::vector<double>{
                                                 0.0, 1.0,
                                                 1.0, 1.0,
                                                 2.0, 0.0,
                                                 1.0, 2.0,
                                                 0.0, 2.0,
                                                 2.0, 1.0},
                                               {6, 2},
                                               f64());
```

Expected output

```
NodeJ split + prep demo
holdout train idx count=4 test idx count=2
strat holdout train idx count=4 test idx count=2
kfold folds=3
imputed shape=[6, 3]
standardized shape=[6, 3]
minmax shape=[6, 3]
robust shape=[6, 3]
neighbor-imputed shape=[6, 3]
iterative-imputed shape=[6, 3]
vector-normalized shape=[6, 3]
maxabs shape=[6, 3]
...
```

Example 3 (Demo Pattern)

```
    std::cerr << "make_transformer(kmeans) failed: " << kmeans_or.status().message << "\n
↪";
    return 1;
  }
  auto kmeans = kmeans_or.move_value();
```

(continues on next page)

(continued from previous page)

```
  if (!kmeans->learn(features).ok()) {
    std::cerr << "kmeans learn failed\n";
    return 1;
  }
  auto labels_or = kmeans->map(features);
  if (!labels_or.ok()) {
    std::cerr << "kmeans map failed: " << labels_or.status().message << "\n";
    return 1;
  }
  const auto labels = labels_or.value().reshape({60}).to(pluss::pCPU,␣
↪pluss::ScalarType::Float64).to_vec<double>();
  std::cout << "kmeans labels (first 10): ";
  for (int i = 0; i < 10; ++i) std::cout << labels[static_cast<std::size_t>(i)] << " ";
  std::cout << "\n";

  std::vector<double> pca_vals{};
  for (int i = 0; i < 80; ++i) {
    const double x0 = -2.0 + 4.0 * static_cast<double>(i) / 79.0;
```

Expected output

```
kmeans labels (first 10): 0 0 0 0 0 0 0 0 0 0
pca output shape: [80, 2]
nodej_unsupervised_demo complete
```

prep::OutlierIsolator::filter_inliers

API Signature

- `prep::OutlierIsolator::filter_inliers(frame: const pluss::table::DataFrame&, options: const InferOptions&) -> pluss::table::DataFrame`

Notes

Instance method on `self`.

Inputs

- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `options` (`const InferOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `pluss::table::DataFrame`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = prep::OutlierIsolator::filter_inliers(/* frame */, /* options */);
out.info("filter_inliers", value);
```

Expected output

```
filter_inliers returned pluss::table::DataFrame
```

prep::OutlierIsolator::infer

API Signature

- `prep::OutlierIsolator::infer(frame: const pluss::table::DataFrame&, options: const InferOptions&) -> pluss::table::Thread`

Notes

Computes predictions from prepared inputs using the currently learned state.

Inputs

- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `options` (`const InferOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `pluss::table::Thread`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = prep::OutlierIsolator::infer(/* frame */, /* options */);
out.info("infer", value);
```

Expected output

```
infer returned pluss::table::Thread
```

Example 2 (Demo Pattern)

```
  train.tolerance = 1e-8;
  train.random.seed = 77;
  train.random.deterministic = true;
  if (!flow.learn(batch, train).ok()) {
    std::cerr << "flow learn failed\n";
    return 1;
  }

  auto pred_before_or = flow.infer(batch.features);
  if (!pred_before_or.ok()) {
    std::cerr << "flow infer before save failed: " << pred_before_or.status().message <<
↪"\n";
    return 1;
  }
  const std::string model_dir = "<repo_root>/PlussAI/examples/NodeJ/trained_models";
  std::error_code mkdir_ec;
  std::filesystem::create_directories(model_dir, mkdir_ec);
  if (mkdir_ec) {
    std::cerr << "failed to create model directory: " << model_dir << " (" << mkdir_ec.
↪message() << ")\n";
    return 1;
  }
  const std::string bundle_path = model_dir + "/nodej_flow_bundle_demo.bin";
```

Expected output

```
libc++abi: terminating due to uncaught exception of type pluss::Error: DataReader::with_
↪sizes: explicit sizes are only supported for host-buffer inputs
Triggered at: <path> (DataReader &pluss::detail::DataReader::with_sizes(std::span<const␣
↪std::int64_t>))
Native stack trace:
  0  pluss::detail::DataReader::with_sizes(std::__1::span<long long const,␣
↪18446744073709551615ul>)(data_reader.h:250)
  1  pluss::codem::Series pluss::detail::DataReader::to_series<pluss::codem::Config␣
↪const&>(pluss::detail::DataReader::Option, pluss::codem::Config const&) const(data_
↪reader.h:295)
  2  pluss::codem::Series pluss::codem::value<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_value.h:13)
  3  pluss::codem::Series pluss::make_series<pluss::codem::Config const&>
↪(pluss::detail::DataReader, pluss::detail::DataReader::Option, pluss::codem::Config␣
↪const&)(series_factory.h:960)
  4  main(nodej_flow_bundle_demo.cpp:13)
  5  start + 6992 [dyld]


[exit_code] -6
```

Example 3 (Demo Pattern)

```
}

void run_classifier_family(const std::string& name, const ModelSpec& spec, const␣
↪LabeledBatch& train) {
  auto model_or = make_classifier(spec);
  if (!require_ok(model_or, name + " make_classifier")) return;
  std::unique_ptr<ClassifierBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;

  auto pred_or = model->infer(train.features);
  if (!require_ok(pred_or, name + " infer")) return;
  auto metric_or = model->assess(train, AssessOptions{.metrics = {MetricKind::Accuracy,␣
↪MetricKind::LogLoss}});
  if (!require_ok(metric_or, name + " assess")) return;
  print_metric("  " + name + " (acc, logloss)", metric_or.value());
}

void run_regressor_family(const std::string& name, const ModelSpec& spec, const␣
↪LabeledBatch& train) {
  auto model_or = make_regressor(spec);
  if (!require_ok(model_or, name + " make_regressor")) return;
  std::unique_ptr<RegressorBase> model = model_or.move_value();
  if (!require_status(model->learn(train), name + " learn")) return;
```

Expected output

```
NodeJ model families demo
Classification families:
  linear (acc, logloss): 1 4.47877e-06
  linear_hinge (acc, logloss): 1 0.208572
  tree (acc, logloss): 1 9.99978e-13
  ensemble (acc, logloss): 1 1.00003e-12
  neighbor (acc, logloss): 1 9.99978e-13
  kernel (acc, logloss): 1 0.318438
  bayes (acc, logloss): 1 9.99978e-13
Regression families:
  linear (mse, r2): 1.52381e-19 1
  tree (mse, r2): 0 1
...
```

prep::OutlierIsolator::learn

API Signature

- prep::OutlierIsolator::learn(frame: const pluss::table::DataFrame&, options: const TrainOptions&) -> prep::OutlierIsolator&

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `options` (`const TrainOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `prep::OutlierIsolator&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = prep::OutlierIsolator::learn(/* frame */, /* options */);
out.info("learn", value);
```

Expected output

```
learn returned prep::OutlierIsolator&
```

Example 2 (Demo Pattern)

```
  auto bounded_or = make_classifier(linear_logistic_spec());
  if (!require_ok(bounded_or, "make_classifier(linear bounded)")) return 1;
  auto bounded = bounded_or.move_value();

  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }

  auto bounded_scores_or = bounded->infer_scores(batch.features);
  if (!require_ok(bounded_scores_or, "infer_scores(bounded)")) return 1;
  std::cout << "max |bounded score|: " << bounded_scores_or.value().abs().max().item
↪<double>() << "\n";

  std::cout << "NodeJ CV + tune demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪1.12262
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.169779
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.420312
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0248626
sweep trial objectives: 0 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0352983 0.743786
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 3 (Demo Pattern)

```
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
  }

  auto metrics_or = reg->assess(batch, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metrics_or, "assess")) return 1;
  print_metrics("linear metrics [mse, r2]", metrics_or.value());

  std::cout << "NodeJ DataFrame bridge demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

prep::OutlierIsolator::learn_infer

API Signature

- prep::OutlierIsolator::learn_infer(frame: const pluss::table::DataFrame&, learn_options: const TrainOptions&, infer_options: const InferOptions&) -> pluss::table::Thread

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

- frame (const pluss::table::DataFrame&) Input feature data used by this call.
- infer_options (const InferOptions&) Configuration object controlling algorithm behavior and safety defaults.
- learn_options (const TrainOptions&) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns pluss::table::Thread.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = prep::OutlierIsolator::learn_infer(/* frame */, /* learn_options */, /*␣
↪infer_options */);
out.info("learn_infer", value);
```

Expected output

```
learn_infer returned pluss::table::Thread
```

prep::OutlierIsolator::score_samples

API Signature

- prep::OutlierIsolator::score_samples(frame: const pluss::table::DataFrame&, options: const InferOptions&) -> pluss::table::Thread

Notes

Instance method on self.

Inputs

- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `options` (`const InferOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `pluss::table::Thread`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = prep::OutlierIsolator::score_samples(/* frame */, /* options */);
out.info("score_samples", value);
```

Expected output

```
score_samples returned pluss::table::Thread
```

prep::SimpleImputer::learn

API Signature

- `prep::SimpleImputer::learn(frame: const pluss::table::DataFrame&, options: const TrainOptions&) -> prep::SimpleImputer&`

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `options` (`const TrainOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `prep::SimpleImputer&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = prep::SimpleImputer::learn(/* frame */, /* options */);
out.info("learn", value);
```

Expected output

```
learn returned prep::SimpleImputer&
```

Example 2 (Demo Pattern)

```
  auto bounded_or = make_classifier(linear_logistic_spec());
  if (!require_ok(bounded_or, "make_classifier(linear bounded)")) return 1;
  auto bounded = bounded_or.move_value();

  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }

  auto bounded_scores_or = bounded->infer_scores(batch.features);
  if (!require_ok(bounded_scores_or, "infer_scores(bounded)")) return 1;
  std::cout << "max |bounded score|: " << bounded_scores_or.value().abs().max().item
↪<double>() << "\n";

  std::cout << "NodeJ CV + tune demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪1.12262
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.169779
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.420312
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0248626
sweep trial objectives: 0 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0352983 0.743786
regression quantile sweep best index: 0
```

(continues on next page)

(continued from previous page)

```
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 3 (Demo Pattern)

```
          .regularization_strength = 1e-8,
          .fit_intercept = true,
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
  }

  auto metrics_or = reg->assess(batch, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metrics_or, "assess")) return 1;
  print_metrics("linear metrics [mse, r2]", metrics_or.value());

  std::cout << "NodeJ DataFrame bridge demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

prep::SimpleImputer::learn_map

API Signature

- prep::SimpleImputer::learn_map(frame: const pluss::table::DataFrame&, learn_options: const TrainOptions&, infer_options: const InferOptions&) -> pluss::table::DataFrame

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `infer_options` (`const InferOptions&`) Configuration object controlling algorithm behavior and safety defaults.
- `learn_options` (`const TrainOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `pluss::table::DataFrame`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = prep::SimpleImputer::learn_map(/* frame */, /* learn_options */, /* infer_
↪options */);
out.info("learn_map", value);
```

Expected output

```
learn_map returned pluss::table::DataFrame
```

Example 2 (Demo Pattern)

```
  if (!require_ok(fold_or, "fold_plan")) return 1;
  std::cout << "kfold folds=" << fold_or.value().folds.size() << "\n";

  SimpleImputer imputer(SimpleImputerOptions{
      .strategy = SimpleImputerStrategy::Median,
      .constant_value = 0.0,
      .empty_feature_fallback = 0.0,
  });
  auto imp_or = imputer.learn_map(dense);
  if (!require_ok(imp_or, "SimpleImputer.learn_map")) return 1;
  const Series x_imputed = imp_or.move_value();
  print_shape("imputed", x_imputed);

  StandardScaler zscore{};
  auto z_or = zscore.learn_map(FeatureBatch{.x = x_imputed, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(z_or, "StandardScaler.learn_map")) return 1;
```

(continues on next page)

(continued from previous page)

```
  const Series x_z = z_or.move_value();
  print_shape("standardized", x_z);

  MinMaxScaler minmax{};
  auto mm_or = minmax.learn_map(FeatureBatch{.x = x_imputed, .layout =␣
↪ObservationLayout::Rows});
```

Expected output

```
NodeJ split + prep demo
holdout train idx count=4 test idx count=2
strat holdout train idx count=4 test idx count=2
kfold folds=3
imputed shape=[6, 3]
standardized shape=[6, 3]
minmax shape=[6, 3]
robust shape=[6, 3]
neighbor-imputed shape=[6, 3]
iterative-imputed shape=[6, 3]
vector-normalized shape=[6, 3]
maxabs shape=[6, 3]
...
```

Example 3 (Demo Pattern)

```
  std::cout << "numeric features: ";
  for (const auto& n : numeric) std::cout << n << " ";
  std::cout << "\n";
  std::cout << "categorical features: ";
  for (const auto& n : categorical) std::cout << n << " ";
  std::cout << "\n";
  auto ord = lymph::prep::OrdinalEncoder({.include_unknown_bucket = true, .one_based =␣
↪true});
  ord.features({"ocean_proximity"});
  const auto ocean_ord = ord.learn_map(housing.features({"ocean_proximity"}));
  auto lbl = lymph::prep::LabelEncoder({.include_unknown_bucket = true, .one_based =␣
↪false});
  const auto ocean_lbl = lbl.learn_map(housing.features("ocean_proximity"));
  std::cout << "ordinal mapped names: ";
  for (const auto& name : ord.mapped_names_) std::cout << name << " ";
  std::cout << "\n";
  std::cout << "label mapped names: ";
  for (const auto& name : lbl.mapped_names_) std::cout << name << " ";
  std::cout << "\n";
  std::cout << "ordinal sample: " << ocean_ord << "\n";
  std::cout << "label sample: " << ocean_lbl << "\n";
  std::cout << "pred: " << pred << "\n";
  std::cout << "mse: " << metrics.mse << " r2: " << metrics.r2 << "\n";
```

Expected output

```
numeric features: median_income target
categorical features: ocean_proximity
ordinal mapped names: ocean_proximity_ordinal
label mapped names: ocean_proximity_label
ordinal sample: ======================
     | ocean_proximity
----------------------
0    | 2
1    | 1
2    | 2
3    | 1
4    | 2
...
```

prep::SimpleImputer::map

API Signature

- `prep::SimpleImputer::map(frame: const pluss::table::DataFrame&, options: const InferOptions&) -> pluss::table::DataFrame`

Notes

Applies preprocessing/transformation behavior configured by the current options.

Inputs

- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `options` (`const InferOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `pluss::table::DataFrame`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto value = prep::SimpleImputer::map(/* frame */, /* options */);
out.info("map", value);
```

Expected output

```
map returned pluss::table::DataFrame
```

Example 2 (Demo Pattern)

```
  if (!require_ok(poly_or, "PolynomialLift.learn_map")) return 1;
  print_shape("polynomial-lift", poly_or.value());

  const Series x_query = pluss::make_series(std::vector<double>{
                                              7.0, 5.0, 1.0,
                                              8.0, 5.5, 0.0},
                                            {2, 3},
                                            f64());
  auto mapped_or = zscore.map(FeatureBatch{.x = x_query, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(mapped_or, "StandardScaler.map")) return 1;
  print_shape("standardized query", mapped_or.value());

  const Series categories = pluss::make_series(std::vector<double>{
                                                 0.0, 1.0,
                                                 1.0, 1.0,
                                                 2.0, 0.0,
                                                 1.0, 2.0,
                                                 0.0, 2.0,
                                                 2.0, 1.0},
                                               {6, 2},
                                               f64());
```

Expected output

```
NodeJ split + prep demo
holdout train idx count=4 test idx count=2
strat holdout train idx count=4 test idx count=2
kfold folds=3
imputed shape=[6, 3]
standardized shape=[6, 3]
minmax shape=[6, 3]
robust shape=[6, 3]
neighbor-imputed shape=[6, 3]
iterative-imputed shape=[6, 3]
vector-normalized shape=[6, 3]
maxabs shape=[6, 3]
...
```

Example 3 (Demo Pattern)

```
    std::cerr << "make_transformer(kmeans) failed: " << kmeans_or.status().message << "\n
↪";
    return 1;
  }
  auto kmeans = kmeans_or.move_value();
  if (!kmeans->learn(features).ok()) {
    std::cerr << "kmeans learn failed\n";
    return 1;
  }
  auto labels_or = kmeans->map(features);
```

(continues on next page)

(continued from previous page)

```
  if (!labels_or.ok()) {
    std::cerr << "kmeans map failed: " << labels_or.status().message << "\n";
    return 1;
  }
  const auto labels = labels_or.value().reshape({60}).to(pluss::pCPU,␣
↪pluss::ScalarType::Float64).to_vec<double>();
  std::cout << "kmeans labels (first 10): ";
  for (int i = 0; i < 10; ++i) std::cout << labels[static_cast<std::size_t>(i)] << " ";
  std::cout << "\n";

  std::vector<double> pca_vals{};
  for (int i = 0; i < 80; ++i) {
    const double x0 = -2.0 + 4.0 * static_cast<double>(i) / 79.0;
```

Expected output

```
kmeans labels (first 10): 0 0 0 0 0 0 0 0 0 0
pca output shape: [80, 2]
nodej_unsupervised_demo complete
```

prep::StandardScaler::learn

API Signature

- `prep::StandardScaler::learn(frame: const pluss::table::DataFrame&, options: const TrainOptions&) -> prep::StandardScaler&`

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `options` (`const TrainOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `prep::StandardScaler&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = prep::StandardScaler::learn(/* frame */, /* options */);
out.info("learn", value);
```

Expected output

```
learn returned prep::StandardScaler&
```

Example 2 (Demo Pattern)

```
  auto bounded_or = make_classifier(linear_logistic_spec());
  if (!require_ok(bounded_or, "make_classifier(linear bounded)")) return 1;
  auto bounded = bounded_or.move_value();

  TrainOptions bounded_train = train;
  bounded_train.parameter_lower_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  bounded_train.parameter_upper_bounds = pluss::make_series(std::vector<double>{0.0}, {1}
↪, f64());
  if (!bounded->learn(batch, bounded_train).ok()) {
    std::cerr << "bounded linear learn failed\n";
    return 1;
  }

  auto bounded_scores_or = bounded->infer_scores(batch.features);
  if (!require_ok(bounded_scores_or, "infer_scores(bounded)")) return 1;
  std::cout << "max |bounded score|: " << bounded_scores_or.value().abs().max().item
↪<double>() << "\n";

  std::cout << "NodeJ CV + tune demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ CV + tune demo
linear mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪1.12262
linear std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.169779
kernel mean metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 1 1 1 1 1 1␣
↪0.420312
kernel std metrics [acc, bal_acc, precision, recall, f1, roc_auc, logloss]: 0 0 0 0 0 0␣
↪0.0248626
sweep trial objectives: 0 1
sweep best index: 1
probe best index: 1
regression quantile sweep objectives: 0.0352983 0.743786
regression quantile sweep best index: 0
max |bounded score|: 0
NodeJ CV + tune demo completed.
```

Example 3 (Demo Pattern)

```
          .regularization_strength = 1e-8,
          .fit_intercept = true,
```

(continues on next page)

(continued from previous page)

```
      },
  };

  auto reg_or = make_regressor(spec);
  if (!require_ok(reg_or, "make_regressor")) return 1;
  auto reg = reg_or.move_value();
  Status st = reg->learn(batch);
  if (!st.ok()) {
    std::cerr << "learn failed: " << st.message << "\n";
    return 1;
  }

  auto metrics_or = reg->assess(batch, AssessOptions{.metrics = {MetricKind::Mse,␣
↪MetricKind::R2}});
  if (!require_ok(metrics_or, "assess")) return 1;
  print_metrics("linear metrics [mse, r2]", metrics_or.value());

  std::cout << "NodeJ DataFrame bridge demo completed.\n";
  return 0;
}
```

Expected output

```
NodeJ DataFrame bridge demo
adapted batch: samples=6 features=4 weighted=yes
linear metrics [mse, r2]: 1.51681e-17 1
NodeJ DataFrame bridge demo completed.
```

prep::StandardScaler::learn_map

API Signature

- `prep::StandardScaler::learn_map(frame: const pluss::table::DataFrame&, learn_options: const TrainOptions&, infer_options: const InferOptions&) -> pluss::table::DataFrame`

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `infer_options` (`const InferOptions&`) Configuration object controlling algorithm behavior and safety defaults.
- `learn_options` (`const TrainOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `pluss::table::DataFrame`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = prep::StandardScaler::learn_map(/* frame */, /* learn_options */, /* infer_
↪options */);
out.info("learn_map", value);
```

Expected output

```
learn_map returned pluss::table::DataFrame
```

Example 2 (Demo Pattern)

```
  if (!require_ok(fold_or, "fold_plan")) return 1;
  std::cout << "kfold folds=" << fold_or.value().folds.size() << "\n";

  SimpleImputer imputer(SimpleImputerOptions{
      .strategy = SimpleImputerStrategy::Median,
      .constant_value = 0.0,
      .empty_feature_fallback = 0.0,
  });
  auto imp_or = imputer.learn_map(dense);
  if (!require_ok(imp_or, "SimpleImputer.learn_map")) return 1;
  const Series x_imputed = imp_or.move_value();
  print_shape("imputed", x_imputed);

  StandardScaler zscore{};
  auto z_or = zscore.learn_map(FeatureBatch{.x = x_imputed, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(z_or, "StandardScaler.learn_map")) return 1;
  const Series x_z = z_or.move_value();
  print_shape("standardized", x_z);

  MinMaxScaler minmax{};
  auto mm_or = minmax.learn_map(FeatureBatch{.x = x_imputed, .layout =␣
↪ObservationLayout::Rows});
```

Expected output

```
NodeJ split + prep demo
holdout train idx count=4 test idx count=2
strat holdout train idx count=4 test idx count=2
kfold folds=3
```

(continues on next page)

(continued from previous page)

```
imputed shape=[6, 3]
standardized shape=[6, 3]
minmax shape=[6, 3]
robust shape=[6, 3]
neighbor-imputed shape=[6, 3]
iterative-imputed shape=[6, 3]
vector-normalized shape=[6, 3]
maxabs shape=[6, 3]
...
```

Example 3 (Demo Pattern)

```cpp
  std::cout << "numeric features: ";
  for (const auto& n : numeric) std::cout << n << " ";
  std::cout << "\n";
  std::cout << "categorical features: ";
  for (const auto& n : categorical) std::cout << n << " ";
  std::cout << "\n";
  auto ord = lymph::prep::OrdinalEncoder({.include_unknown_bucket = true, .one_based =␣
↪true});
  ord.features({"ocean_proximity"});
  const auto ocean_ord = ord.learn_map(housing.features({"ocean_proximity"}));
  auto lbl = lymph::prep::LabelEncoder({.include_unknown_bucket = true, .one_based =␣
↪false});
  const auto ocean_lbl = lbl.learn_map(housing.features("ocean_proximity"));
  std::cout << "ordinal mapped names: ";
  for (const auto& name : ord.mapped_names_) std::cout << name << " ";
  std::cout << "\n";
  std::cout << "label mapped names: ";
  for (const auto& name : lbl.mapped_names_) std::cout << name << " ";
  std::cout << "\n";
  std::cout << "ordinal sample: " << ocean_ord << "\n";
  std::cout << "label sample: " << ocean_lbl << "\n";
  std::cout << "pred: " << pred << "\n";
  std::cout << "mse: " << metrics.mse << " r2: " << metrics.r2 << "\n";
```

Expected output

```
numeric features: median_income target
categorical features: ocean_proximity
ordinal mapped names: ocean_proximity_ordinal
label mapped names: ocean_proximity_label
ordinal sample: ======================
     | ocean_proximity
----------------------
0    | 2
1    | 1
2    | 2
3    | 1
4    | 2
...
```

prep::StandardScaler::map

API Signature

- `prep::StandardScaler::map(frame: const pluss::table::DataFrame&, options: const InferOptions&) -> pluss::table::DataFrame`

Notes

Applies preprocessing/transformation behavior configured by the current options.

Inputs

- `frame` (`const pluss::table::DataFrame&`) Input feature data used by this call.
- `options` (`const InferOptions&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `pluss::table::DataFrame`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = prep::StandardScaler::map(/* frame */, /* options */);
out.info("map", value);
```

Expected output

```
map returned pluss::table::DataFrame
```

Example 2 (Demo Pattern)

```
  if (!require_ok(poly_or, "PolynomialLift.learn_map")) return 1;
  print_shape("polynomial-lift", poly_or.value());

  const Series x_query = pluss::make_series(std::vector<double>{
                                                7.0, 5.0, 1.0,
                                                8.0, 5.5, 0.0},
                                            {2, 3},
                                            f64());
  auto mapped_or = zscore.map(FeatureBatch{.x = x_query, .layout =␣
↪ObservationLayout::Rows});
  if (!require_ok(mapped_or, "StandardScaler.map")) return 1;
  print_shape("standardized query", mapped_or.value());

  const Series categories = pluss::make_series(std::vector<double>{
```

(continues on next page)

(continued from previous page)

```
                                                    0.0, 1.0,
                                                    1.0, 1.0,
                                                    2.0, 0.0,
                                                    1.0, 2.0,
                                                    0.0, 2.0,
                                                    2.0, 1.0},
                                                {6, 2},
                                                f64());
```

Expected output

```
NodeJ split + prep demo
holdout train idx count=4 test idx count=2
strat holdout train idx count=4 test idx count=2
kfold folds=3
imputed shape=[6, 3]
standardized shape=[6, 3]
minmax shape=[6, 3]
robust shape=[6, 3]
neighbor-imputed shape=[6, 3]
iterative-imputed shape=[6, 3]
vector-normalized shape=[6, 3]
maxabs shape=[6, 3]
...
```

Example 3 (Demo Pattern)

```
    std::cerr << "make_transformer(kmeans) failed: " << kmeans_or.status().message << "\n
↪";
    return 1;
  }
  auto kmeans = kmeans_or.move_value();
  if (!kmeans->learn(features).ok()) {
    std::cerr << "kmeans learn failed\n";
    return 1;
  }
  auto labels_or = kmeans->map(features);
  if (!labels_or.ok()) {
    std::cerr << "kmeans map failed: " << labels_or.status().message << "\n";
    return 1;
  }
  const auto labels = labels_or.value().reshape({60}).to(pluss::pCPU,␣
↪pluss::ScalarType::Float64).to_vec<double>();
  std::cout << "kmeans labels (first 10): ";
  for (int i = 0; i < 10; ++i) std::cout << labels[static_cast<std::size_t>(i)] << " ";
  std::cout << "\n";

  std::vector<double> pca_vals{};
  for (int i = 0; i < 80; ++i) {
    const double x0 = -2.0 + 4.0 * static_cast<double>(i) / 79.0;
```

Expected output

```
kmeans labels (first 10): 0 0 0 0 0 0 0 0 0 0
pca output shape: [80, 2]
nodej_unsupervised_demo complete
```

simple_imputer_strategy_name

API Signature

- `simple_imputer_strategy_name(strategy: SimpleImputerStrategy) -> const char*`

Notes

Applies preprocessing/transformation behavior configured by the current options.

Inputs

- `strategy` (`SimpleImputerStrategy`) Target values aligned to input observations.

Output

- Returns `const char*`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```cpp
pluss::OutputEngine out;
auto value = simple_imputer_strategy_name(/* strategy */);
out.info("simple_imputer_strategy_name", value);
```

Expected output

```
simple_imputer_strategy_name returned const char*
```

SimpleImputer::features

API Signature

- `SimpleImputer::features(names: RangeView::Of<std::string_view>) -> SimpleImputer&`
- `SimpleImputer::features(names: std::initializer_list<std::string_view>) -> SimpleImputer&`

Notes

Instance method on `self`.

Inputs

- `names` (`RangeView::Of<std::string_view> | std::initializer_list<std::string_view>`) List-like argument; pass values in the order expected by the signature.

Output

- Returns `SimpleImputer&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = SimpleImputer::features(/* names */);
out.info("features", value);
```

Expected output

```
features returned SimpleImputer&
```

Example 2 (Demo Pattern)

```
    const auto split = nja::holdout_split(frame, nja::HoldoutSplitOptions{
                                                     .test_size = 0.2,
                                                     .random_state = 42,
                                                     .deterministic = true,
                                                     .shuffle = true,
                                                 });
    const DataFrame train_x = split.train_set.features(
        {"season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
↪"weather_condition", "temp", "atemp", "humidity", "windspeed"});
    const DataFrame test_x = split.test_set.features(
        {"season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
↪"weather_condition", "temp", "atemp", "humidity", "windspeed"});
    const Series train_y = split.train_set.features("total_count").as_series();
    const Series test_y = split.test_set.features("total_count").as_series();

    TrainOptions train_options{};
    train_options.max_iterations = 2500;
    train_options.tolerance = 1e-8;
    train_options.random.seed = 7;
    train_options.random.deterministic = true;
```

Expected output

```
CV objective metric: rmse
candidate=linear_ridge objective=102.872 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
```

(continues on next page)

(continued from previous page)

```
data=[102.87158] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.41650024] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[77.611594]
candidate=decision_tree objective=44.9727 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[44.972693] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.88843998] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[27.61723]
candidate=random_forest objective=81.1235 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[81.123538] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.63710967] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
...
```

Example 3 (Demo Pattern)

```
  std::cout << "numeric features: ";
  for (const auto& n : numeric) std::cout << n << " ";
  std::cout << "\n";
  std::cout << "categorical features: ";
  for (const auto& n : categorical) std::cout << n << " ";
  std::cout << "\n";
  auto ord = lymph::prep::OrdinalEncoder({.include_unknown_bucket = true, .one_based =␣
↪true});
  ord.features({"ocean_proximity"});
  const auto ocean_ord = ord.learn_map(housing.features({"ocean_proximity"}));
  auto lbl = lymph::prep::LabelEncoder({.include_unknown_bucket = true, .one_based =␣
↪false});
  const auto ocean_lbl = lbl.learn_map(housing.features("ocean_proximity"));
  std::cout << "ordinal mapped names: ";
  for (const auto& name : ord.mapped_names_) std::cout << name << " ";
  std::cout << "\n";
  std::cout << "label mapped names: ";
  for (const auto& name : lbl.mapped_names_) std::cout << name << " ";
  std::cout << "\n";
  std::cout << "ordinal sample: " << ocean_ord << "\n";
  std::cout << "label sample: " << ocean_lbl << "\n";
  std::cout << "pred: " << pred << "\n";
```

Expected output

```
numeric features: median_income target
categorical features: ocean_proximity
ordinal mapped names: ocean_proximity_ordinal
label mapped names: ocean_proximity_label
```

(continues on next page)

(continued from previous page)

```
ordinal sample: ======================
     | ocean_proximity
----------------------
0    | 2
1    | 1
2    | 2
3    | 1
4    | 2
...
```

SimpleImputer::learned

API Signature

- `SimpleImputer::learned() -> bool`

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = SimpleImputer::learned();
out.info("learned", value);
```

Expected output

```
learned: true
```

SimpleImputer::summary

API Signature

- `SimpleImputer::summary() -> SimpleImputerSummary`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `SimpleImputerSummary`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = SimpleImputer::summary();
out.info("summary", value);
```

Expected output

```
summary returned SimpleImputerSummary
```

Example 2 (Demo Pattern)

```
    //========== standalone lymph preprocessing API ========
    // This section demonstrates direct prep estimators (learn/map/learn_map)
    // without going through Workflow composition.
    auto simple_imp = lymph::prep::simple_imputer(lymph::prep::SimpleImputerOptions{
        .strategy = lymph::prep::SimpleImputerStrategy::Median,
    });
    const DataFrame train_simple_imputed = simple_imp.learn_map(train_x);
    const DataFrame test_simple_imputed = simple_imp.map(test_x);
    const auto simple_imp_summary = simple_imp.summary();

    auto knn_imp = lymph::prep::neighbor_imputer(lymph::prep::NeighborImputerOptions{
        .neighbors = 7,
        .weighting = lymph::prep::NeighborImputerWeighting::Distance,
        .distance_floor = 1e-12,
        .empty_feature_fallback = 0.0,
    });
    const DataFrame train_knn_imputed = knn_imp.learn_map(train_x);
    const DataFrame test_knn_imputed = knn_imp.map(test_x);
    const auto knn_imp_summary = knn_imp.summary();

    auto iter_imp = lymph::prep::iterative_imputer(lymph::prep::IterativeImputerOptions{
```

Expected output

```
summary returned SimpleImputerSummary
```

StandardScaler::features

API Signature

- `StandardScaler::features(names: RangeView::Of<std::string_view>) -> StandardScaler&`
- `StandardScaler::features(names: std::initializer_list<std::string_view>) -> StandardScaler&`

Notes

Instance method on `self`.

Inputs

- `names` (`RangeView::Of<std::string_view> | std::initializer_list<std::string_view>`) List-like argument; pass values in the order expected by the signature.

Output

- Returns `StandardScaler&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = StandardScaler::features(/* names */);
out.info("features", value);
```

Expected output

```
features returned StandardScaler&
```

Example 2 (Demo Pattern)

```
    const auto split = nja::holdout_split(frame, nja::HoldoutSplitOptions{
                                                     .test_size = 0.2,
                                                     .random_state = 42,
                                                     .deterministic = true,
                                                     .shuffle = true,
                                                 });
    const DataFrame train_x = split.train_set.features(
        {"season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
↪"weather_condition", "temp", "atemp", "humidity", "windspeed"});
    const DataFrame test_x = split.test_set.features(
```

(continues on next page)

(continued from previous page)

```
        {"season", "year", "month", "hour", "is_holiday", "weekday", "is_workingday",
↪"weather_condition", "temp", "atemp", "humidity", "windspeed"});
    const Series train_y = split.train_set.features("total_count").as_series();
    const Series test_y = split.test_set.features("total_count").as_series();

    TrainOptions train_options{};
    train_options.max_iterations = 2500;
    train_options.tolerance = 1e-8;
    train_options.random.seed = 7;
    train_options.random.deterministic = true;
```

Expected output

```
CV objective metric: rmse
candidate=linear_ridge objective=102.872 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[102.87158] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.41650024] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[77.611594]
candidate=decision_tree objective=44.9727 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[44.972693] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.88843998] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[27.61723]
candidate=random_forest objective=81.1235 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[81.123538] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.63710967] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
...
```

Example 3 (Demo Pattern)

```
  std::cout << "numeric features: ";
  for (const auto& n : numeric) std::cout << n << " ";
  std::cout << "\n";
  std::cout << "categorical features: ";
  for (const auto& n : categorical) std::cout << n << " ";
  std::cout << "\n";
  auto ord = lymph::prep::OrdinalEncoder({.include_unknown_bucket = true, .one_based =␣
↪true});
  ord.features({"ocean_proximity"});
  const auto ocean_ord = ord.learn_map(housing.features({"ocean_proximity"}));
  auto lbl = lymph::prep::LabelEncoder({.include_unknown_bucket = true, .one_based =␣
↪false});
  const auto ocean_lbl = lbl.learn_map(housing.features("ocean_proximity"));
```

(continues on next page)

(continued from previous page)

```
  std::cout << "ordinal mapped names: ";
  for (const auto& name : ord.mapped_names_) std::cout << name << " ";
  std::cout << "\n";
  std::cout << "label mapped names: ";
  for (const auto& name : lbl.mapped_names_) std::cout << name << " ";
  std::cout << "\n";
  std::cout << "ordinal sample: " << ocean_ord << "\n";
  std::cout << "label sample: " << ocean_lbl << "\n";
  std::cout << "pred: " << pred << "\n";
```

Expected output

```
numeric features: median_income target
categorical features: ocean_proximity
ordinal mapped names: ocean_proximity_ordinal
label mapped names: ocean_proximity_label
ordinal sample: ======================
     | ocean_proximity
----------------------
0    | 2
1    | 1
2    | 2
3    | 1
4    | 2
...
```

`StandardScaler::learned`

API Signature

- `StandardScaler::learned() -> bool`

Notes

Fits model/transform state from the provided training data. Call this before inference-oriented methods.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = StandardScaler::learned();
out.info("learned", value);
```

Expected output

```
learned: true
```

StandardScaler::summary

API Signature

- `StandardScaler::summary() -> StandardScalerSummary`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `StandardScalerSummary`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = StandardScaler::summary();
out.info("summary", value);
```

Expected output

```
summary returned StandardScalerSummary
```

Example 2 (Demo Pattern)

```
    //========== standalone lymph preprocessing API ========
    // This section demonstrates direct prep estimators (learn/map/learn_map)
    // without going through Workflow composition.
    auto simple_imp = lymph::prep::simple_imputer(lymph::prep::SimpleImputerOptions{
```

(continues on next page)

(continued from previous page)

```
        .strategy = lymph::prep::SimpleImputerStrategy::Median,
    });
    const DataFrame train_simple_imputed = simple_imp.learn_map(train_x);
    const DataFrame test_simple_imputed = simple_imp.map(test_x);
    const auto simple_imp_summary = simple_imp.summary();

    auto knn_imp = lymph::prep::neighbor_imputer(lymph::prep::NeighborImputerOptions{
        .neighbors = 7,
        .weighting = lymph::prep::NeighborImputerWeighting::Distance,
        .distance_floor = 1e-12,
        .empty_feature_fallback = 0.0,
    });
    const DataFrame train_knn_imputed = knn_imp.learn_map(train_x);
    const DataFrame test_knn_imputed = knn_imp.map(test_x);
    const auto knn_imp_summary = knn_imp.summary();

    auto iter_imp = lymph::prep::iterative_imputer(lymph::prep::IterativeImputerOptions{
```

Expected output

```
summary returned StandardScalerSummary
```

take_features

API Signature

- `take_features(batch: const FeatureBatch&, feature_indices: RangeView::Int) -> FeatureBatch`
- `take_features(batch: const FeatureBatch&, feature_indices: std::initializer_list<std::int64_t>) -> FeatureBatch`

Notes

Namespace-level API function.

Inputs

- `batch` (`const FeatureBatch&`) Input feature data used by this call.
- `feature_indices` (`RangeView::Int | std::initializer_list<std::int64_t>`) Input feature data used by this call.

Output

- Returns `FeatureBatch`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = take_features(/* batch */, /* feature_indices */);
out.info("take_features", value);
```

Expected output

```
take_features returned FeatureBatch
```

temporal_features

API Signature

- temporal_features(frame: const pluss::table::DataFrame&) -> std::vector<std::string>

Notes

Namespace-level API function.

Inputs

- frame (const pluss::table::DataFrame&) Input feature data used by this call.

Output

- Returns std::vector<std::string>.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = temporal_features(/* frame */);
out.info("temporal_features", value);
```

Expected output

```
temporal_features returned std::vector<std::string>
```

to_csr

API Signature

- to_csr(sparse: const Triplet&) -> Csr

Notes

Namespace-level API function.

Inputs

- `sparse` (`const Triplet&`) Method argument; see signature type for accepted form.

Output

- Returns `Csr`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = to_csr(/* sparse */);
out.info("to_csr", value);
```

Expected output

```
to_csr returned Csr
```

`tune_ledger`

API Signature

- `tune_ledger(report:  const TuneReport&) -> const TuneLedger&`

Notes

Runs parameter search and returns per-trial metrics plus best-trial selection.

Inputs

- `report` (`const TuneReport&`) Method argument; see signature type for accepted form.

Output

- Returns `const TuneLedger&`.

Options

Option semantics used by this method:

- `TuneReport` (`struct`):
  - `std::vector<Trial> trials{}` Option field used by this method.
  - `std::int64_t best_index = -1` Option field used by this method.

- `std::string best_name{}` Option field used by this method.
- `MetricKind objective_metric = MetricKind::Accuracy` Selects optimization or evaluation objective.
- `TuneObjectivePolicy objective_policy = TuneObjectivePolicy::Auto` Selects optimization or evaluation objective.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = tune_ledger(/* report */);
out.info("tune_ledger", value);
```

Expected output

```
tune_ledger returned const TuneLedger&
```

variance_prune

API Signature

- `variance_prune(batch: const FeatureBatch&, variance_floor: double = 1e-12) -> VariancePrune`

Notes

Namespace-level API function.

Inputs

- `batch` (`const FeatureBatch&`) Input feature data used by this call.
- `variance_floor` (`double`), default: `1e-12` Method argument; see signature type for accepted form.

Output

- Returns `VariancePrune`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = variance_prune(/* batch */, /* variance_floor */);
out.info("variance_prune", value);
```

Expected output

```
variance_prune returned VariancePrune
```

Workflow::coeff

API Signature

- `Workflow::coeff() -> Series`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `Series`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::coeff();
out.info("coeff", value);
```

Expected output

```
coeff returned Series
```

Example 2 (Demo Pattern)

```
                lymph::prep::SimpleImputerOptions{.strategy =␣
↪lymph::prep::SimpleImputerStrategy::Median}),
            lymph::workflow::stage::standard_scaler(),
        },
    };
    linear_workflow.learn_(train_x, train_y, train_options);
    const auto linear_metrics = linear_workflow.assess(test_x, test_y, assess_options);
    std::cout << "linear test metrics [rmse, r2, mae]: " << linear_metrics.rmse << " " <
↪< linear_metrics.r2 << " "
              << linear_metrics.mae << "\n";
    std::cout << "linear params: intercept=" << linear_workflow.intercept() << " coeff="
↪<< linear_workflow.coeff() << "\n";

    const Series linear_pred_series = linear_workflow.infer(test_x);
```

(continues on next page)

(continued from previous page)

```
    const fs::path linear_model_path = model_dir / "bike_sharing_linear.bin";
    linear_workflow.save(linear_model_path.string(), train_x, train_y, train_options);
    const auto linear_restored = lymph::Workflow::load(linear_model_path.string());
    const Series linear_pred_restored_series = linear_restored.infer(test_x);
    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});

    const Series linear_pred = linear_pred_series.to(pluss::pCPU, ScalarType::Float64).
↪reshape({linear_pred_series.numel()});
    const double linear_max_abs_diff = lymph::max_abs_diff(linear_pred_series, linear_
↪pred_restored_series);
    std::cout << "linear restored prediction max_abs_diff: " << linear_max_abs_diff << "\
↪n";
```

Expected output

```
coeff returned Series
```

Workflow::dependence_curve

API Signature

- `Workflow::dependence_curve(train: const LabeledBatch&, eval: const FeatureBatch&, feature_index: std::int64_t, grid_points: std::int64_t = 32) -> DependenceCurve`

Notes

Instance method on `self`.

Inputs

- `eval` (`const FeatureBatch&`) Method argument; see signature type for accepted form.
- `feature_index` (`std::int64_t`) Input feature data used by this call.
- `grid_points` (`std::int64_t`), default: `32` Method argument; see signature type for accepted form.
- `train` (`const LabeledBatch&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `DependenceCurve`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::dependence_curve(/* train */, /* eval */, /* feature_index */, /*␣
```

(continues on next page)

(continued from previous page)

```
↪grid_points */);
out.info("dependence_curve", value);
```

Expected output

```
dependence_curve returned DependenceCurve
```

Workflow::dependence_surface

API Signature

- `Workflow::dependence_surface(train: const LabeledBatch&, eval: const FeatureBatch&, feature_a: std::int64_t, feature_b: std::int64_t, grid_points_a: std::int64_t = 16, grid_points_b: std::int64_t = 16) -> DependenceSurface`

Notes

Instance method on `self`.

Inputs

- `eval` (`const FeatureBatch&`) Method argument; see signature type for accepted form.
- `feature_a` (`std::int64_t`) Input feature data used by this call.
- `feature_b` (`std::int64_t`) Input feature data used by this call.
- `grid_points_a` (`std::int64_t`), default: `16` Method argument; see signature type for accepted form.
- `grid_points_b` (`std::int64_t`), default: `16` Method argument; see signature type for accepted form.
- `train` (`const LabeledBatch&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `DependenceSurface`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::dependence_surface(/* train */, /* eval */, /* feature_a */, /*␣
↪feature_b */, /* grid_points_a */, /* grid_points_b */);
out.info("dependence_surface", value);
```

Expected output

```
dependence_surface returned DependenceSurface
```

Workflow::dump_model_state

API Signature

- `Workflow::dump_model_state() -> std::vector<Series>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::vector<Series>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::dump_model_state();
out.info("dump_model_state", value);
```

Expected output

```
dump_model_state returned std::vector<Series>
```

Workflow::dump_transform_states

API Signature

- `Workflow::dump_transform_states() -> std::vector<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::vector<std::vector<Series>>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::dump_transform_states();
out.info("dump_transform_states", value);
```

Expected output

```
dump_transform_states returned std::vector<std::vector<Series>>
```

`Workflow::feature_transformer`

API Signature

- `Workflow::feature_transformer(transformer: prep::FeatureTransformer) -> Workflow&`

Notes

Instance method on `self`.

Inputs

- `transformer` (`prep::FeatureTransformer`) Method argument; see signature type for accepted form.

Output

- Returns `Workflow&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::feature_transformer(/* transformer */);
out.info("feature_transformer", value);
```

Expected output

```
feature_transformer returned Workflow&
```

Example 2 (Demo Pattern)

```
      lymph::prep::FeatureRemainderPolicy::PassThrough);

  auto reg = lymph::linear_factory::linear_
↪regression(pluss::ai::nodej::LinearFamilyOptions{
      .objective = pluss::ai::nodej::LinearObjective::LeastSquares,
      .regularization = pluss::ai::nodej::RegularizationKind::None,
      .regularization_strength = 0.0,
      .fit_intercept = true,
  });
  reg.feature_transformer(std::move(preprocessing));
  reg.learn_(housing, data_options);

  const auto pred = reg.infer(housing, data_options);
  const auto metrics = reg.assess(housing,
                                  data_options,
                                  pluss::ai::nodej::AssessOptions{
                                      .metrics = {pluss::ai::nodej::MetricKind::Mse,␣
↪pluss::ai::nodej::MetricKind::R2},
                                  });

  std::cout << "numeric features: ";
  for (const auto& n : numeric) std::cout << n << " ";
  std::cout << "\n";
```

Expected output

```
numeric features: median_income target
categorical features: ocean_proximity
ordinal mapped names: ocean_proximity_ordinal
label mapped names: ocean_proximity_label
ordinal sample: ======================
     | ocean_proximity
----------------------
0    | 2
1    | 1
2    | 2
3    | 1
4    | 2
...
```

`Workflow::flow_summary`

API Signature

- `Workflow::flow_summary() -> std::string`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::string`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::flow_summary();
out.info("flow_summary", value);
```

Expected output

```
flow_summary returned std::string
```

`Workflow::function_transformer`

API Signature

- `Workflow::function_transformer(transformer: prep::FunctionTransformer) -> Workflow&`

Notes

Instance method on `self`.

Inputs

- `transformer` (`prep::FunctionTransformer`) Method argument; see signature type for accepted form.

Output

- Returns `Workflow&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::function_transformer(/* transformer */);
out.info("function_transformer", value);
```

Expected output

```
function_transformer returned Workflow&
```

Workflow::graph_info

API Signature

- `Workflow::graph_info() -> FlowGraphInfo`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `FlowGraphInfo`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::graph_info();
out.info("graph_info", value);
```

Expected output

```
graph_info returned FlowGraphInfo
```

Workflow::has_model

API Signature

- `Workflow::has_model() -> bool`

Notes

Boolean introspection helper that reports whether a capability/state is available.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::has_model();
out.info("has_model", value);
```

Expected output

```
has_model: true
```

`Workflow::has_model_spec`

API Signature

- `Workflow::has_model_spec() -> bool`

Notes

Boolean introspection helper that reports whether a capability/state is available.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::has_model_spec();
out.info("has_model_spec", value);
```

Expected output

```
has_model_spec: true
```

Workflow::has_opaque_transforms

API Signature

- `Workflow::has_opaque_transforms() -> bool`

Notes

Boolean introspection helper that reports whether a capability/state is available.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::has_opaque_transforms();
out.info("has_opaque_transforms", value);
```

Expected output

```
has_opaque_transforms: true
```

Workflow::intercept

API Signature

- `Workflow::intercept() -> Series`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `Series`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::intercept();
out.info("intercept", value);
```

Expected output

```
intercept returned Series
```

Example 2 (Demo Pattern)

```
                lymph::prep::SimpleImputerOptions{.strategy =␣
↪lymph::prep::SimpleImputerStrategy::Median}),
            lymph::workflow::stage::standard_scaler(),
        },
    };
    linear_workflow.learn_(train_x, train_y, train_options);
    const auto linear_metrics = linear_workflow.assess(test_x, test_y, assess_options);
    std::cout << "linear test metrics [rmse, r2, mae]: " << linear_metrics.rmse << " " <
↪< linear_metrics.r2 << " "
              << linear_metrics.mae << "\n";
    std::cout << "linear params: intercept=" << linear_workflow.intercept() << " coeff="
↪<< linear_workflow.coeff() << "\n";

    const Series linear_pred_series = linear_workflow.infer(test_x);
    const fs::path linear_model_path = model_dir / "bike_sharing_linear.bin";
    linear_workflow.save(linear_model_path.string(), train_x, train_y, train_options);
    const auto linear_restored = lymph::Workflow::load(linear_model_path.string());
    const Series linear_pred_restored_series = linear_restored.infer(test_x);
    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});

    const Series linear_pred = linear_pred_series.to(pluss::pCPU, ScalarType::Float64).
↪reshape({linear_pred_series.numel()});
    const double linear_max_abs_diff = lymph::max_abs_diff(linear_pred_series, linear_
↪pred_restored_series);
    std::cout << "linear restored prediction max_abs_diff: " << linear_max_abs_diff << "\
↪n";
```

Expected output

```
intercept returned Series
```

`Workflow::load`

API Signature

- `Workflow::load(path:  const std::string&) -> Workflow`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `path` (`const std::string&`) Path to persisted model/data artifact.

Output

- Returns `Workflow`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::load(/* path */);
out.info("load", value);
```

Expected output

```
load returned Workflow
```

Example 2 (Demo Pattern)

```
    auto best = std::move(tuned.best_workflow);
    best.learn_(train_x, train_y, train_options);
    const auto test_metrics = best.assess(test_x, test_y, assess_options);
    std::cout << "test metrics [rmse, r2, mae]: " << test_metrics.rmse << " " << test_
↪metrics.r2 << " " << test_metrics.mae << "\n";

    const Series pred = best.infer(test_x);
    const fs::path best_model_path = model_dir / "hour_best_workflow.bin";
    best.save(best_model_path.string(), train_x, train_y, train_options);
    auto restored = nja::Workflow::load(best_model_path.string());
    const Series pred_restored = restored.infer(test_x);
    const double restored_max_abs_diff = nja::max_abs_diff(pred, pred_restored);
    std::cout << "restored prediction max_abs_diff: " << restored_max_abs_diff << "\n";

    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});
    const Series y_pred = pred.to(pluss::pCPU, ScalarType::Float64).reshape({pred.
```

(continues on next page)

(continued from previous page)

```
↪numel()});
    DataFrame pred_frame({
        {"y_true", Thread(y_true)},
        {"y_pred", Thread(y_pred)},
        {"residual", Thread(y_true.sub(y_pred))},
    });
    pred_frame.save_csv((out_dir / "hour_workflow_tuning_predictions.csv").string());
```

Expected output

```
CV objective metric: rmse
candidate=linear_ridge objective=102.872 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[102.87158] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.41650024] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[77.611594]
candidate=decision_tree objective=44.9727 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[44.972693] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.88843998] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[27.61723]
candidate=random_forest objective=81.1235 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[81.123538] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.63710967] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
...
```

`Workflow::model_spec`

API Signature

- `Workflow::model_spec() -> const std::optional<ModelSpec>&`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `const std::optional<ModelSpec>&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::model_spec();
out.info("model_spec", value);
```

Expected output

```
model_spec returned const std::optional<ModelSpec>&
```

`Workflow::nested_param`

API Signature

- `Workflow::nested_param(path:  std::string_view) -> double`

Notes

Instance method on `self`.

Inputs

- `path` (`std::string_view`) Path to persisted model/data artifact.

Output

- Returns `double`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::nested_param(/* path */);
out.info("nested_param", value);
```

Expected output

```
nested_param: <numeric_result>
```

Workflow::nested_param_keys

API Signature

- `Workflow::nested_param_keys() -> std::vector<std::string>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::vector<std::string>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::nested_param_keys();
out.info("nested_param_keys", value);
```

Expected output

```
nested_param_keys returned std::vector<std::string>
```

Workflow::restore

API Signature

- `Workflow::restore(bundle:  const io::FlowBundle&) -> Workflow`

Notes

Instance method on `self`.

Inputs

- `bundle` (`const io::FlowBundle&`) Method argument; see signature type for accepted form.

Output

- Returns `Workflow`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::restore(/* bundle */);
out.info("restore", value);
```

Expected output

```
restore returned Workflow
```

`Workflow::save_blueprint`

API Signature

- `Workflow::save_blueprint() -> std::string`

Notes

Persists the current fitted artifact so it can be loaded for inference later.

Inputs

None.

Output

- Returns `std::string`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::save_blueprint();
out.info("save_blueprint", value);
```

Expected output

```
save_blueprint returned std::string
```

`Workflow::stage_count`

API Signature

- `Workflow::stage_count() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::stage_count();
out.info("stage_count", value);
```

Expected output

```
stage_count: <numeric_result>
```

`Workflow::task_kind`

API Signature

- `Workflow::task_kind() -> TaskKind`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `TaskKind`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::task_kind();
out.info("task_kind", value);
```

Expected output

```
task_kind returned TaskKind
```

`Workflow::trained`

API Signature

- `Workflow::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

`Workflow::transform_specs`

API Signature

- `Workflow::transform_specs() -> const std::vector<TransformSpec>&`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `const std::vector<TransformSpec>&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::transform_specs();
out.info("transform_specs", value);
```

Expected output

```
transform_specs returned const std::vector<TransformSpec>&
```

`Workflow::workflow_model`

API Signature

- `Workflow::workflow_model() -> const WorkflowModel&`
- `Workflow::workflow_model() -> WorkflowModel&`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `WorkflowModel&`.
- Returns `const WorkflowModel&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = Workflow::workflow_model();
out.info("workflow_model", value);
```

Expected output

```
workflow_model returned const WorkflowModel&
```

`WorkflowModel::coefficients`

API Signature

- `WorkflowModel::coefficients() -> std::vector<double>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::vector<double>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::coefficients();
out.info("coefficients", value);
```

Expected output

```
coefficients returned std::vector<double>
```

WorkflowModel::dependence_curve

API Signature

- `WorkflowModel::dependence_curve(train: const LabeledBatch&, eval: const FeatureBatch&, feature_index: std::int64_t, grid_points: std::int64_t = 32) -> DependenceCurve`

Notes

Instance method on `self`.

Inputs

- `eval` (`const FeatureBatch&`) Method argument; see signature type for accepted form.
- `feature_index` (`std::int64_t`) Input feature data used by this call.
- `grid_points` (`std::int64_t`), default: `32` Method argument; see signature type for accepted form.
- `train` (`const LabeledBatch&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `DependenceCurve`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::dependence_curve(/* train */, /* eval */, /* feature_index */
↪, /* grid_points */);
out.info("dependence_curve", value);
```

Expected output

```
dependence_curve returned DependenceCurve
```

WorkflowModel::dependence_surface

API Signature

- `WorkflowModel::dependence_surface(train: const LabeledBatch&, eval: const FeatureBatch&, feature_a: std::int64_t, feature_b: std::int64_t, grid_points_a: std::int64_t = 16, grid_points_b: std::int64_t = 16) -> DependenceSurface`

Notes

Instance method on `self`.

Inputs

- `eval` (`const FeatureBatch&`) Method argument; see signature type for accepted form.
- `feature_a` (`std::int64_t`) Input feature data used by this call.
- `feature_b` (`std::int64_t`) Input feature data used by this call.
- `grid_points_a` (`std::int64_t`), default: `16` Method argument; see signature type for accepted form.
- `grid_points_b` (`std::int64_t`), default: `16` Method argument; see signature type for accepted form.
- `train` (`const LabeledBatch&`) Configuration object controlling algorithm behavior and safety defaults.

Output

- Returns `DependenceSurface`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::dependence_surface(/* train */, /* eval */, /* feature_a */,␣
↪/* feature_b */, /* grid_points_a */, /* grid_points_b */);
out.info("dependence_surface", value);
```

Expected output

```
dependence_surface returned DependenceSurface
```

`WorkflowModel::dump_model_state`

API Signature

- `WorkflowModel::dump_model_state() -> std::vector<Series>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::vector<Series>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::dump_model_state();
out.info("dump_model_state", value);
```

Expected output

```
dump_model_state returned std::vector<Series>
```

`WorkflowModel::dump_transform_states`

API Signature

- `WorkflowModel::dump_transform_states() -> std::vector<std::vector<Series>>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::vector<std::vector<Series>>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::dump_transform_states();
out.info("dump_transform_states", value);
```

Expected output

```
dump_transform_states returned std::vector<std::vector<Series>>
```

`WorkflowModel::feature_transformer`

API Signature

- `WorkflowModel::feature_transformer(transformer: prep::FeatureTransformer) -> WorkflowModel&`

Notes

Instance method on `self`.

Inputs

- `transformer` (`prep::FeatureTransformer`) Method argument; see signature type for accepted form.

Output

- Returns `WorkflowModel&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::feature_transformer(/* transformer */);
out.info("feature_transformer", value);
```

Expected output

```
feature_transformer returned WorkflowModel&
```

Example 2 (Demo Pattern)

```
      lymph::prep::FeatureRemainderPolicy::PassThrough);

  auto reg = lymph::linear_factory::linear_
↪regression(pluss::ai::nodej::LinearFamilyOptions{
      .objective = pluss::ai::nodej::LinearObjective::LeastSquares,
      .regularization = pluss::ai::nodej::RegularizationKind::None,
      .regularization_strength = 0.0,
      .fit_intercept = true,
  });
  reg.feature_transformer(std::move(preprocessing));
  reg.learn_(housing, data_options);

  const auto pred = reg.infer(housing, data_options);
  const auto metrics = reg.assess(housing,
                                  data_options,
                                  pluss::ai::nodej::AssessOptions{
```

(continues on next page)

(continued from previous page)

```
                                    .metrics = {pluss::ai::nodej::MetricKind::Mse,␣
↪pluss::ai::nodej::MetricKind::R2},
                                });

  std::cout << "numeric features: ";
  for (const auto& n : numeric) std::cout << n << " ";
  std::cout << "\n";
```

Expected output

```
numeric features: median_income target
categorical features: ocean_proximity
ordinal mapped names: ocean_proximity_ordinal
label mapped names: ocean_proximity_label
ordinal sample: ======================
     | ocean_proximity
----------------------
0    | 2
1    | 1
2    | 2
3    | 1
4    | 2
...
```

WorkflowModel::flow_summary

API Signature

- `WorkflowModel::flow_summary() -> std::string`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::string`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::flow_summary();
out.info("flow_summary", value);
```

Expected output

```
flow_summary returned std::string
```

WorkflowModel::function_transformer

API Signature

- `WorkflowModel::function_transformer(transformer: prep::FunctionTransformer) -> WorkflowModel&`

Notes

Instance method on `self`.

Inputs

- `transformer` (`prep::FunctionTransformer`) Method argument; see signature type for accepted form.

Output

- Returns `WorkflowModel&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::function_transformer(/* transformer */);
out.info("function_transformer", value);
```

Expected output

```
function_transformer returned WorkflowModel&
```

WorkflowModel::graph_info

API Signature

- `WorkflowModel::graph_info() -> FlowGraphInfo`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `FlowGraphInfo`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::graph_info();
out.info("graph_info", value);
```

Expected output

```
graph_info returned FlowGraphInfo
```

`WorkflowModel::has_model`

API Signature

- `WorkflowModel::has_model() -> bool`

Notes

Boolean introspection helper that reports whether a capability/state is available.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::has_model();
out.info("has_model", value);
```

Expected output

```
has_model: true
```

WorkflowModel::has_model_spec

API Signature

- `WorkflowModel::has_model_spec() -> bool`

Notes

Boolean introspection helper that reports whether a capability/state is available.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::has_model_spec();
out.info("has_model_spec", value);
```

Expected output

```
has_model_spec: true
```

WorkflowModel::has_opaque_transforms

API Signature

- `WorkflowModel::has_opaque_transforms() -> bool`

Notes

Boolean introspection helper that reports whether a capability/state is available.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::has_opaque_transforms();
out.info("has_opaque_transforms", value);
```

Expected output

```
has_opaque_transforms: true
```

`WorkflowModel::intercept_term`

API Signature

- `WorkflowModel::intercept_term() -> double`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `double`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::intercept_term();
out.info("intercept_term", value);
```

Expected output

```
intercept_term: <numeric_result>
```

WorkflowModel::linear_parameters

API Signature

- `WorkflowModel::linear_parameters() -> LinearParameters`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `LinearParameters`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::linear_parameters();
out.info("linear_parameters", value);
```

Expected output

```
linear_parameters returned LinearParameters
```

WorkflowModel::load

API Signature

- `WorkflowModel::load(path:  const std::string&) -> WorkflowModel`

Notes

Loads a previously persisted artifact and restores its inference-ready state.

Inputs

- `path` (`const std::string&`) Path to persisted model/data artifact.

Output

- Returns `WorkflowModel`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::load(/* path */);
out.info("load", value);
```

Expected output

```
load returned WorkflowModel
```

Example 2 (Demo Pattern)

```
    auto best = std::move(tuned.best_workflow);
    best.learn_(train_x, train_y, train_options);
    const auto test_metrics = best.assess(test_x, test_y, assess_options);
    std::cout << "test metrics [rmse, r2, mae]: " << test_metrics.rmse << " " << test_
↪metrics.r2 << " " << test_metrics.mae << "\n";

    const Series pred = best.infer(test_x);
    const fs::path best_model_path = model_dir / "hour_best_workflow.bin";
    best.save(best_model_path.string(), train_x, train_y, train_options);
    auto restored = nja::Workflow::load(best_model_path.string());
    const Series pred_restored = restored.infer(test_x);
    const double restored_max_abs_diff = nja::max_abs_diff(pred, pred_restored);
    std::cout << "restored prediction max_abs_diff: " << restored_max_abs_diff << "\n";

    const Series y_true = test_y.to(pluss::pCPU, ScalarType::Float64).reshape({test_y.
↪numel()});
    const Series y_pred = pred.to(pluss::pCPU, ScalarType::Float64).reshape({pred.
↪numel()});
    DataFrame pred_frame({
        {"y_true", Thread(y_true)},
        {"y_pred", Thread(y_pred)},
        {"residual", Thread(y_true.sub(y_pred))},
```

(continues on next page)

(continued from previous page)

```
    });
    pred_frame.save_csv((out_dir / "hour_workflow_tuning_predictions.csv").string());
```

Expected output

```
CV objective metric: rmse
candidate=linear_ridge objective=102.872 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[102.87158] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.41650024] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[77.611594]
candidate=decision_tree objective=44.9727 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[44.972693] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.88843998] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[27.61723]
candidate=random_forest objective=81.1235 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[81.123538] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.63710967] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
...
```

`WorkflowModel::model_spec`

API Signature

- `WorkflowModel::model_spec() -> const std::optional<ModelSpec>&`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `const std::optional<ModelSpec>&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::model_spec();
out.info("model_spec", value);
```

Expected output

```
model_spec returned const std::optional<ModelSpec>&
```

WorkflowModel::nested_param

API Signature

- `WorkflowModel::nested_param(path:  std::string_view) -> double`

Notes

Instance method on `self`.

Inputs

- `path` (`std::string_view`) Path to persisted model/data artifact.

Output

- Returns `double`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::nested_param(/* path */);
out.info("nested_param", value);
```

Expected output

```
nested_param: <numeric_result>
```

WorkflowModel::nested_param_keys

API Signature

- `WorkflowModel::nested_param_keys() -> std::vector<std::string>`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `std::vector<std::string>`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::nested_param_keys();
out.info("nested_param_keys", value);
```

Expected output

```
nested_param_keys returned std::vector<std::string>
```

`WorkflowModel::restore`

API Signature

- `WorkflowModel::restore(bundle:  const io::FlowBundle&) -> WorkflowModel`

Notes

Instance method on `self`.

Inputs

- `bundle` (`const io::FlowBundle&`) Method argument; see signature type for accepted form.

Output

- Returns `WorkflowModel`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::restore(/* bundle */);
out.info("restore", value);
```

Expected output

```
restore returned WorkflowModel
```

`WorkflowModel::save_blueprint`

API Signature

- `WorkflowModel::save_blueprint() -> std::string`

Notes

Persists the current fitted artifact so it can be loaded for inference later.

Inputs

None.

Output

- Returns `std::string`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::save_blueprint();
out.info("save_blueprint", value);
```

Expected output

```
save_blueprint returned std::string
```

`WorkflowModel::stage_count`

API Signature

- `WorkflowModel::stage_count() -> std::int64_t`

Notes

Reports a cardinality/count statistic from the current object state.

Inputs

None.

Output

- Returns `std::int64_t`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::stage_count();
out.info("stage_count", value);
```

Expected output

```
stage_count: <numeric_result>
```

`WorkflowModel::task_kind`

API Signature

- `WorkflowModel::task_kind() -> TaskKind`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `TaskKind`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::task_kind();
out.info("task_kind", value);
```

Expected output

```
task_kind returned TaskKind
```

WorkflowModel::trained

API Signature

- `WorkflowModel::trained() -> bool`

Notes

Boolean state check for object readiness.

Inputs

None.

Output

- Returns `bool`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::trained();
out.info("trained", value);
```

Expected output

```
trained: true
```

WorkflowModel::transform_specs

API Signature

- `WorkflowModel::transform_specs() -> const std::vector<TransformSpec>&`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `const std::vector<TransformSpec>&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::transform_specs();
out.info("transform_specs", value);
```

Expected output

```
transform_specs returned const std::vector<TransformSpec>&
```

`WorkflowModel::workflow`

API Signature

- `WorkflowModel::workflow() -> const Workflow&`
- `WorkflowModel::workflow() -> Workflow&`

Notes

Instance method on `self`.

Inputs

None.

Output

- Returns `Workflow&`.
- Returns `const Workflow&`.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel::workflow();
out.info("workflow", value);
```

Expected output

```
workflow returned const Workflow&
```

Example 2 (Demo Pattern)

```
    {
      auto wf = nja::Workflow{
          .model = nja::linear_factory::linear_regression(LinearFamilyOptions{          .
↪objective = LinearObjective::LeastSquares,
          .regularization = RegularizationKind::L2,
          .regularization_strength = 1e-6,
          .fit_intercept = true,
      }),
          .stages = {
              nja::workflow::stage::simple_imputer(nja::prep::SimpleImputerOptions{.
↪strategy = nja::prep::SimpleImputerStrategy::Median}),
              nja::workflow::stage::standard_scaler(),
          },
      };
      candidates.emplace_back("linear_ridge", std::move(wf));
    }
    {
      auto wf = nja::Workflow{
          .model = nja::tree_factory::decision_tree_regressor(TreeFamilyOptions{        ␣
↪ .criterion = TreeCriterion::Mse,
          .max_depth = 18,
          .min_samples_leaf = 8,
          .min_samples_split = 24,
      }),
```

Expected output

```
CV objective metric: rmse
candidate=linear_ridge objective=102.872 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[102.87158] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.41650024] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[77.611594]
candidate=decision_tree objective=44.9727 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[44.972693] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
```

(continues on next page)

(continued from previous page)

```
data=[0.88843998] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
data=[27.61723]
candidate=random_forest objective=81.1235 cv_rmse=pluss::Series(shape=[1], strides=[1],␣
↪dtype=float64, platform=cpu, numel=1, contiguous=true)
data=[81.123538] cv_r2=pluss::Series(shape=[1], strides=[1], dtype=float64, platform=cpu,
↪ numel=1, contiguous=true)
data=[0.63710967] cv_mae=pluss::Series(shape=[1], strides=[1], dtype=float64,␣
↪platform=cpu, numel=1, contiguous=true)
...
```

Example 3 (Demo Pattern)

```
  data_options.feature_names = {"median_income", "ocean_proximity", "target"};
  data_options.label_feature = "target";
  data_options.note_policy = pluss::ai::nodej::DataNotePolicy::Hash;
  data_options.layout = pluss::ai::nodej::ObservationLayout::Rows;

  auto preprocessing = lymph::make_feature_transformer(
      housing,
      data_options,
      std::vector<lymph::workflow::feature::Branch>{
          lymph::workflow::feature::compose(
              {
                  lymph::workflow::stage::simple_
↪imputer(lymph::prep::SimpleImputerOptions{
                      .strategy = lymph::prep::SimpleImputerStrategy::Mean,
                  }),
                  lymph::workflow::stage::standard_scaler(),
              },
              {"median_income"}),
          lymph::workflow::feature::one_hot_encoder(lymph::prep::OneHotEncoderOptions{},␣
↪categorical),
      },
      lymph::prep::FeatureRemainderPolicy::PassThrough);
```

Expected output

```
numeric features: median_income target
categorical features: ocean_proximity
ordinal mapped names: ocean_proximity_ordinal
label mapped names: ocean_proximity_label
ordinal sample: ======================
     | ocean_proximity
----------------------
0    | 2
1    | 1
2    | 2
3    | 1
4    | 2
...
```

`WorkflowModel::WorkflowModel`

API Signature

- `WorkflowModel(arg1: const ModelSpec& spec) : workflow_(resolve(make_flow(spec), "make_flow")) -> auto`
- `WorkflowModel(arg1: Workflow workflow) : workflow_(std::move(workflow)) -> auto`

Notes

Instance method on `self`.

Inputs

- `arg1` (`Workflow workflow) : workflow_(std::move(workflow) | const ModelSpec& spec) : workflow_(resolve(make_flow(spec), "make_flow")`) Method argument; see signature type for accepted form.

Output

- Returns method result for the selected overload.

Options

No explicit option struct/enum parameters were detected for this method.

Examples

Example 1 (Minimal Usage)

```
pluss::OutputEngine out;
auto value = WorkflowModel(/* arg1 */);
out.info("WorkflowModel", value);
```

Expected output

```
WorkflowModel returned result
```

CHAPTER

SIX

BUILD OUTPUTS

- HTML root: `PlussAI/docs/sphinx/_build/html/index.html`
- PDF output: `PlussAI/docs/sphinx/_build/latex/plussai_nodej_lymph_api_docs.pdf`

Build commands:

```
sphinx-build -b html PlussAI/docs/sphinx PlussAI/docs/sphinx/_build/html
sphinx-build -M latexpdf PlussAI/docs/sphinx PlussAI/docs/sphinx/_build
```